GRAU EN ENGINYERIA INFORMÀTICA

102753 Visualització Gràfica Interactiva



3.3+.

3. Primitives Gràfiques

Enric Martí Gòdia

Departament de Ciències de la Computació

Continguts

1. Sistemes de Coordenades
2. *Vertex Buffer Object* (VBO)
3. *Vertex Array Object* (VAO)
4. *Element Buffer Object* (EBO)
5. Pas de valors als shaders
6. Instancing
7. Propietats de Dibuix
8. Primitives Gràfiques
9. Malla. Classe `Mesh`
10. Model. Classe `Model`
11. Importació models 3D. Llibreria Assimp

1. Sistemes de Coordenades

OpenGL treballa en coordenades homogènies *(x,y,z,w)*. L'espai 3D correspon a *w=1* .

Coordenades 3D: Si *w<>1*, el punt 3D *(x',y',z')* que correspon a la coordenada *(x,y,z,w)* seria:

$$(x',y',z') = \left(\frac{x}{w}, \frac{y}{w}, \frac{z}{w}\right)$$

Coordenades 2D: El punt 2D *(x,y)* correspon al punt en coordenades homogènies:

$$\left.\begin{array}{l} z = 0 \\ w = 1 \end{array}\right\} \Rightarrow \quad (x, y, 0, 1)$$

1. Sistemes de Coordenades

Varis nivells per a dibuixar en OpenGL:

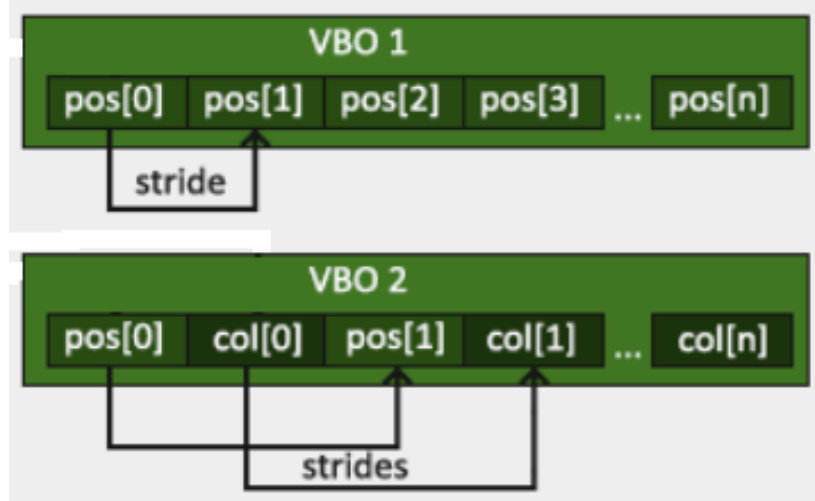
- Buffers OpenGL (VBO, VAO, EBO)
  - Pas de valors als shaders
  - Propietats de dibuix
- Primitives Gràfiques que utilitzen buffers OpenGL
- Classes: Mesh i Model per a importar models 3D mitjançant llibreria Assimp

2. Vertex Buffer Object (VBO)

Objecte OpenGL corresponent a un buffer de memòria d'alta velocitat en placa gràfica que conté informació de models 3D.

Objecte que crea un espai de memòria en la GPU per enviar geometria del model 3D.

S'utilitza per emmagatzemar una característica del model 3D: vèrtex, color, vector normal, textura, etc.



2. Vertex Buffer Object (VBO)

Enviar geometria de CPU és un procés lent. En GL 1.1, per a cada objecte cal enviar la geometria (vèrtexs, color, vectors normals, textura) al hardware gràfic i pintar.

Amb els VBO, podem emmagatzemar la geometria directament en la memòria del hardware gràfic (GPU's).

L'avantatge és que per a pintar les còpies, transferim la geometria de la primitiva una sola vegada, i pintem la còpia amb la geometria carregada a les GPU's.

2. Vertex Buffer Object (VBO)

Passos i comandes:

1. Generar buffer VBO amb identificador → `glGenBuffers()`
2. Activar el buffer VBO a configurar o visualitzar → `glBindBuffer()`
3. Reservar, i transferir dades al VBO → `glBufferData()`
4. Com interpretar les dades del VBO → `glVertexAttribPointer()`
5. Dibuixar l'objecte contingut en el VBO → `glDrawArrays():`
6. Eliminar buffer VBO amb identificador → `glDeleteBuffers()`:

2. Vertex Buffer Object (VBO)

```
void glGenBuffers(GLsizei n, GLuint *buffers);
```

Acció: Genera un vector d'identificadors de VBO i els propis VBO.

```
void glDeleteBuffers(GLsize n, const GLuint *buffers);
```

Acció: Elimina els buffers VBO i el vector d'identificadors.

Paràmetres:

`n`: Nombre de buffers a generar o eliminar.

`buffers`: Vector on es guarden els identificadors de VBO.

2. Vertex Buffer Object (VBO)

```
void glBindBuffer(GLenum target, GLuint buffer);
```

Acció: Activa el VBO identificat per `buffer`.

Paràmetres:

`target`: Tipus d'informació. Pot tenir entre altres valors:

`GL_ARRAY_BUFFER`: Atributs de vèrtexs (VBO).

`GL_ELEMENT_ARRAY_BUFFER`: Índexs a vèrtexs (EBO).

`buffer`: Identificador del *Vertex Buffer Object*. Si té el valor 0, desactiva el VBO que estigui activat.

2. Vertex Buffer Object (VBO)

```
void glBufferData(GLenum target, GLsizeiptr size, const GLvoid *data, GLenum usage);
```

Acció: Crea i inicialitza les dades d'un VBO. Per introduir el primer atribut de vèrtex.

Paràmetres:

`target`: Tipus d'informació. Pot tenir entre altres valors:

- `GL_ARRAY_BUFFER`: Atributs de vèrtexs.
- `GL_ELEMENT_ARRAY_BUFFER`: Índexs a vèrtexs.
- `GL_TEXTURE_BUFFER`: Buffer amb dades de textura.

`size`: Especifica el tamany de les dades a introduir.

`data`: Apuntador a taula que conté els valors a introduir.

`usage`: Especifica el tipus de dades a magatzemar. Pot tenir els valors: `GL_STREAM_DRAW`, `GL_STREAM_READ`, `GL_STREAM_COPY`, `GL_STATIC_DRAW`, `GL_STATIC_READ`, `GL_STATIC_COPY`, `GL_DYNAMIC_DRAW`, `GL_DYNAMIC_READ`, o `GL_DYNAMIC_COPY`.

2. Vertex Buffer Object (VBO)

```
void glVertexAttribPointer(Gluint index, Glint size, Glenum type,
Glboolean normalized, Glsizei strde, const , const GLvoid
*pointer);
```

Acció: Defineix el tipus d'atribut de vèrtex sobre un vector VBO.

Paràmetres:

`index`: Índex de l'atribut a definir.

`size`: nombre de components de l'atribut (valors 2, 3 o 4).

`type`: Tipus de cada component del vector VBO:

`GL_INT`, `GL_UNSIGNED_INT`, `GL_FLOAT`, `GL_DOUBLE` entre altres.

`normalized`: Si dades de punt fixe (*fixed-point*) ha de ser normalitzats (`GL_TRUE`) o no (`GL_FALSE`). Usualment `GL_FALSE`.

`stride`: Desplaçament entre atributs dins vector VBO. Si consecutius, valor 0.

`pointer`: Desplaçament de la primera component de l'atribut. Si vector amb un únic atribut, val 0.

2. Vertex Buffer Object (VBO)

```
void glDrawArrays(GLenum mode, Glint first, GLsizei count);
```

Acció: Dibuixa l'objecte contingut en el VBO, segons la primitiva a dibuixar.

Paràmetres:

`mode`: Especifica quina primitiva es vol dibuixar. Pot tenir els valors: `GL_POINTS`, `GL_LINE_STRIP`, `GL_LINE_LOOP`, `GL_LINES`, `GL_TRIANGLE_STRIP`, `GL_TRIANGLE_FAN`, `GL_TRIANGLES`, `GL_QUAD_STRIP`, `GL_QUADS`, `GL_PATCHES` i `GL_POLYGON`.

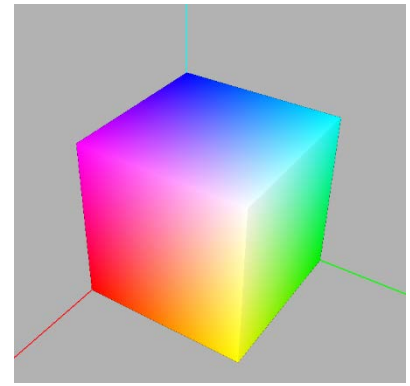
`GL_LINE_STRIP_ADJACENCY`, `GL_LINES_ADJACENCY`, `GL_TRIANGLE_STRIP_ADJACENCY` i `GL_TRIANGLES_ADJACENCY` per a OpenGL 3.3 o superior.

`first`: Index d'inici dins el VBO.

`count`: Nombre de punts a dibuixar.

2. Vertex Buffer Object (VBO)

Exemple: Dibuix del cub per triangles segons el model de color RGB.

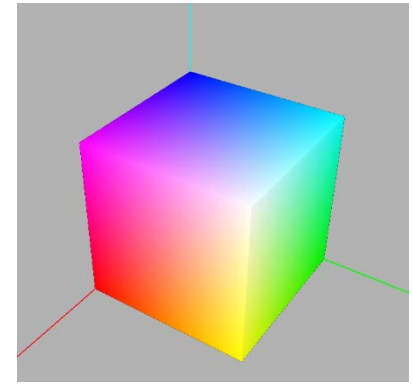


Diferents solucions:

1. Vectors estàtics, única comanda (GL1.1)
2. Vectors dinàmics, comandes separades

2.1. VBO: Vectors estàtics, única comanda

Exemple: Dibuix del cub segons el model de color RGB.



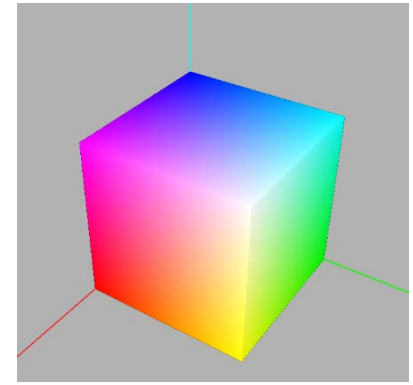
```
// Crear VBO, transferir-hi geometria del cub i dibuixar (tants cops com còpies del cub)
void glutSolidCubeRGB(GLdouble dsize) {
GLdouble size = dSize * 0.5f;

// Vertex Coord Array
GLdouble vertices = {size,size,size,-size,size,size,-size,-size,size, // v0-v1-v2 (front)
-size, -size,  size,  size, -size,  size,  size,  size,  size,      // v2-v3-v0
 size,  size,  size,  size, -size,  size,  size, -size, -size,      // v0-v3-v4 (right)
 size, -size, -size,  size,  size, -size,  size,  size,  size,      // v4-v5-v0
 size,  size,  size,  size,  size, -size, -size,  size, -size,      // v0-v5-v6 (top)
-size,  size, -size, -size,  size,  size,  size,  size,  size,      // v6-v1-v0
-size,  size,  size, -size,  size, -size, -size, -size, -size,      // v1-v6-v7 (left)
-size, -size, -size, -size, -size,  size, -size,  size,  size,      // v7-v2-v1
-size, -size, -size,  size, -size, -size,  size, -size,  size,      // v7-v4-v3 (bottom)
 size, -size,  size, -size, -size,  size, -size, -size, -size,      // v3-v2-v7
 size, -size, -size, -size, -size, -size, -size,  size, -size,      // v4-v7-v6 (back)
-size,  size, -size,  size,  size, -size,  size, -size, -size };    // v6-v5-v4

//Normal array
GLdouble normals[] = {  0,  0,  1,  0,  0,  1,  0,  0,  1,   // v0-v1-v2 (front)
   0,  0,  1,  0,  0,  1,  0,  0,  1,   // v2-v3-v0
   1,  0,  0,  1,  0,  0,  1,  0,  0,   // v0-v3-v4 (right)
   1,  0,  0,  1,  0,  0,  1,  0,  0,   // v4-v5-v0
   0,  1,  0,  0,  1,  0,  0,  1,  0,   // v0-v5-v6 (top)
   0,  1,  0,  0,  1,  0,  0,  1,  0,   // v6-v1-v0
  -1,  0,  0, -1,  0,  0, -1,  0,  0,   // v1-v6-v7 (left)
  -1,  0,  0, -1,  0,  0, -1,  0,  0,   // v7-v2-v1
   0, -1,  0,  0, -1,  0,  0, -1,  0,   // v7-v4-v3 (bottom)
   0, -1,  0,  0, -1,  0,  0, -1,  0,   // v3-v2-v7
   0,  0, -1,  0,  0, -1,  0,  0, -1,   // v4-v7-v6 (back)
   0,  0, -1,  0,  0, -1,  0,  0, -1 }; // v6-v5-v4
{…]
```

2.1. VBO: Vectors estàtics, única comanda

Exemple: Dibuix del cub segons el model de color RGB.



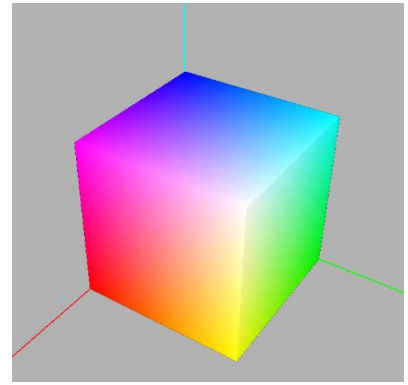
```
// Crear VBO, transferir-hi geometria del cub i dibuixar (tants cops com còpies del cub)
void glutSolidCubeRGB(GLdouble dsize) {
{…]


// Color array
GLdouble colors= { 1, 1, 1, 1, 0, 1, 1, 1, 0, 0, 1, 1,   // v0-v1-v2 (front)
0, 0, 1, 1, 1, 0, 1, 1, 1, 1, 1, 1,   // v2-v3-v0
1, 1, 1, 1, 1, 0, 1, 1, 1, 0, 0, 1,   // v0-v3-v4 (right)
1, 0, 0, 1, 1, 1, 0, 1, 1, 1, 1, 1,   // v4-v5-v0
1, 1, 1, 1, 1, 1, 0, 1, 0, 1, 0, 1,   // v0-v5-v6 (top)
0, 1, 0, 1, 0, 1, 1, 1, 1, 1, 1, 1,   // v6-v1-v0
0, 1, 1, 1, 0, 1, 0, 1, 0, 0, 0, 1,   // v1-v6-v7 (left)
0, 0, 0, 1, 0, 0, 1, 1, 0, 1, 1, 1,   // v7-v2-v1
0, 0, 0, 1, 1, 0, 0, 1, 1, 0, 1, 1,   // v7-v4-v3 (bottom)
1, 0, 1, 1, 0, 0, 1, 1, 0, 0, 0, 1,   // v3-v2-v7
1, 0, 0, 1, 0, 0, 0, 1, 0, 1, 0, 1,   // v4-v7-v6 (back)
0, 1, 0, 1, 1, 1, 0, 1, 1, 0, 0, 1 }; // v6-v5-v4

// Texture Array
GLdouble textures[] = { 1, 1, 1, 0, 0, 0,      // v0-v1-v2 (front)
1, 0, 0, 1, 1, 1,      // v2-v3-v0
1, 1, 0, 1, 0, 0,      // v0-v3-v4 (right)
0, 0, 1, 0, 1, 1,      // v4-v5-v0
1, 1, 1, 0, 0, 1,      // v0-v5-v6 (top)
0, 1, 1, 0, 1, 1,      // v6-v1-v0
1, 1, 0, 0, 1, 0,      // v1-v6-v7 (left)
0, 0, 0, 1, 1, 1,      // v7-v2-v1
0, 0, 0, 1, 1, 0,      // v7-v4-v3 (bottom)
1, 0, 1, 1, 0, 0,      // v3-v2-v7
0, 0, 1, 0, 1, 0,      // v4-v7-v6 (back)
0, 1, 0, 0, 1, 1 };    // v6-v5-v4
{…]
```

2.1. VBO: Vectors estàtics, única comanda

Exemple: Dibuix del cub segons el model de color RGB.



```
// Crear VBO, transferir-hi geometria del cub i dibuixar (tants cops com còpies del cub)
void glutSolidCubeRGB(GLdouble dsize) {

{…]
glGenBuffers(1, &vboId);
glBindBuffer(GL_ARRAY_BUFFER, vboId);                // Activar l'objecte VBO

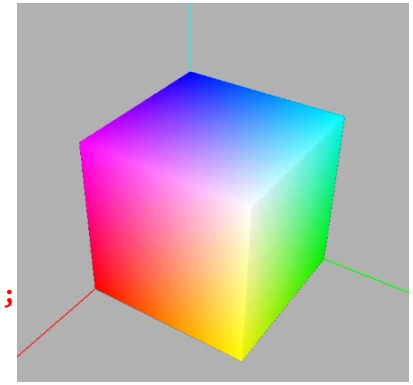
// Reservar buffer per a l'objecte VBO
glBufferData(GL_ARRAY_BUFFER, sizeof(vertices) + sizeof(normals) + sizeof(colors) + sizeof(textures), 0, GL_STATIC_DRAW);

// Atribut vèrtexs. Copia del vector de vèrtexs a la posició 0
glBufferSubData(GL_ARRAY_BUFFER, 0, sizeof(vertices), vertices);
// Atribut normals. Copia del vector de normals darrera el vector de vèrtexs
glBufferSubData(GL_ARRAY_BUFFER, sizeof(vertices), sizeof(normals), normals);
// Atribut color. Copia del vector de color darrera el vector de normals
glBufferSubData(GL_ARRAY_BUFFER, sizeof(vertices) + sizeof(normals), sizeof(colors), colors);
// Atribut textures. Copia del vector de textires darrera el vector de colors
glBufferSubData(GL_ARRAY_BUFFER, sizeof(vertices) + sizeof(normals) + sizeof(colors), sizeof(textures), textures);

glBindBuffer(GL_ARRAY_BUFFER, vboId);                // Activació VBO
glEnableClientState(GL_TEXTURE_COORD_ARRAY);         // Activació vector textures
glEnableClientState(GL_NORMAL_ARRAY);                // Activació vector normals
glEnableClientState(GL_COLOR_ARRAY);                 // Activació vector color
glEnableClientState(GL_VERTEX_ARRAY);                // Activació vector vèrtexs
{…]
```

2.1. VBO: Vectors estàtics, única comanda

Exemple: Dibuix del cub segons el model de color RGB.



```
// Crear VBO, transferir-hi geometria del cub i dibuixar (tants cops com còpies del cub)
void glutSolidCubeRGB(GLdouble dsize) {

{…]
// Abans de dibuixar especificar els offsets per a cada array (vector)
glTexCoordPointer(2, GL_DOUBLE, 0, (void*)(sizeof(vertices) + sizeof(normals) + sizeof(colors)));
glColorPointer(3, GL_DOUBLE, 0, (void*)(sizeof(vertices) + sizeof(normals)));
glNormalPointer(GL_DOUBLE, 0, (void*)sizeof(vertices));
glVertexPointer(3, GL_DOUBLE, 0, 0);

glDrawArrays(GL_TRIANGLES, 0, 36);                    // Dibuixar el cub

glDisableClientState(GL_TEXTURE_COORD_ARRAY);         // Desactivació vector textures
glDisableClientState(GL_NORMAL_ARRAY);                // Desactivació vector normals
glDisableClientState(GL_COLOR_ARRAY);                 // Desactivació vector color
glDisableClientState(GL_VERTEX_ARRAY);                // Desactivació vector vèrtexs


glBindBuffer(GL_ARRAY_BUFFER,0);          // Desactivar l'objecte VBO
glDeleteBuffers(1,&vboId);                // Eliminar l'objecte VBO
}
```

2.1. VBO: Vectors estàtics, única comanda

Si la geometria (nombre de vèrtexs) és coneguda i fixe, es pot utilitzar vectors estàtics.

Però amb funcions com l'esfera, cilindre, el nombre de vèrtexs depèn del nombre de particions pel que és complicat utilitzar vectors estàtics.

SOLUCIÓ: Utilitzar vectors dinàmics (tipus `vector`).

```
#include <vector>                       // Incloure llibreria vector (en stdafx.h en entorn MFC)

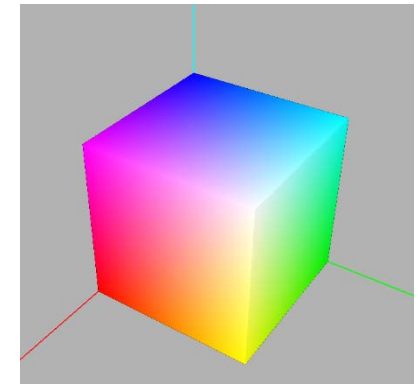
std::vector <double> vertices, normals, colors, textures;
vertices.resize(0);  normals.resize(0);  colors.resize(0); textures.resize(0); // Reinicialitza vectors

vertices.push_back(-size); vertices.push_back(-size); vertices.push_back(-size); // vertices[0]
vormals.push_back(0.0);    normals.push_back(0.0);    normals.push_back(1.0);    // normals[0]
colors.push_back(1.0);     colors.push_back(1.1);     colors.push_back(1.1);     // colors[0]
textures.push_back(1.0);   textures.push_back(1.1);

std::vector <int>::size_type ncv = vertices.size(); // Tamany de CubeVertices
```

2.2. VBO: Vectors dinàmics, única comanda

Exemple: Dibuix del cub segons el model de color RGB.



```
// Crear VBO, transferir-hi geometria del cub i dibuixar (tants cops com còpies del cub)
void glutSolidCubeRGB(GLdouble dsize) {
GLdouble size = dSize * 0.5f;
#include <vector>

// Definició vectors dinàmics per a vèrtexs, i color.
std::vector <double> vertices, normals, colors, textures;
vertices.resize(0);  normals.resize(0);  colors.resize(0); textures.resize(0); // Reinicialitza vectors

vertices.push_back(-size); vertices.push_back(-size); vertices.push_back(-size); // vertices[0]
normals.push_back(0.0);    normals.push_back(0.0);    normals.push_back(1.0);    // normals[0]
colors.push_back(1.0);     colors.push_back(1.1);     colors.push_back(1.1);     // colors[0]
textures.push_back(1.0);   textures.push_back(1.1);                              // textures[0]

             [...]

vertices.push_back(size);  vertices.push_back(size);  vertices.push_back(size);  // vertices[23]
normals.push_back(0.0);    normals.push_back(0.0);    normals.push_back(1.0);    // normals[23]
colors.push_back(1.0);     colors.push_back(0.0);     colors.push_back(0.0);     // colors[23]
textures.push_back(1.0);   textures.push_back(1.0);                              // textures[23]

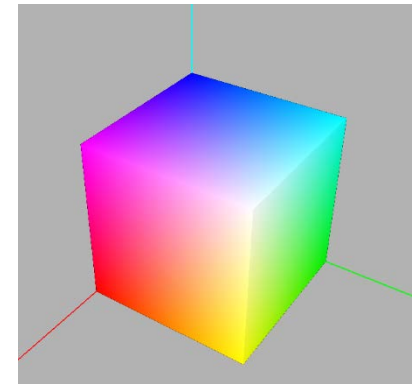

std::vector <int>::size_type ncv = vertices.size();       // Tamany de vertices
glGenBuffers(1, &vboId);
glBindBuffer(GL_ARRAY_BUFFER, vboId);                     // Activar l'objecte VBO

// Reservar buffer per a l'objecte VBO
glBufferData(GL_ARRAY_BUFFER, (vertices.size() + normals.size() + colors.size() + textures.size()), 0, GL_STATIC_DRAW);

{…]
```

2.2. VBO: Vectors dinàmics, única comanda

Exemple: Dibuix del cub segons el model de color RGB.



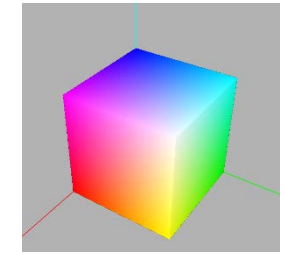
```
// Crear VBO, transferir-hi geometria del cub i dibuixar (tants cops com còpies del cub)
void glutSolidCubeRGB(GLdouble dsize) {
{…]
// Atribut vèrtexs. Copia del vector de vèrtexs a la posició 0
glBufferSubData(GL_ARRAY_BUFFER, 0, vertices.size(), vertices);
// Atribut normals. Copia del vector de normals darrera el vector de vèrtexs
glBufferSubData(GL_ARRAY_BUFFER, vertices.size(), normals.size(), normals);
// Atribut color. Copia del vector de color darrera el vector de normals
glBufferSubData(GL_ARRAY_BUFFER, (vertices.size() + normals.size()), colors.size(), colors);
// Atribut textures. Copia del vector de textires darrera el vector de colors
glBufferSubData(GL_ARRAY_BUFFER, (vertices.size() + normals.size() + colors.size()), textures.size(), textures);

glEnableClientState(GL_TEXTURE_COORD_ARRAY);            // Activació vector textures
glEnableClientState(GL_NORMAL_ARRAY);                   // Activació vector normals
glEnableClientState(GL_COLOR_ARRAY);                    // Activació vector color
glEnableClientState(GL_VERTEX_ARRAY);                   // Activació vector vèrtexs
// Abans de dibuixar especificar els offsets per a cada array (vector)
glTexCoordPointer(2, GL_DOUBLE, 0, (void*)(vertices.size() + normal.size() + colors.size()));
glColorPointer(3, GL_DOUBLE, 0, (void*)(vertices.size() + normal.size()));
glNormalPointer(GL_DOUBLE, 0, (void*)vertices.size());
glVertexPointer(3, GL_DOUBLE, 0, 0);

glDrawArrays(GLTRIANGLES,0,ncv/3);                      // Dibuix de l'objecte

glDisableClientState(GL_TEXTURE_COORD_ARRAY);           // Desactivació vector textures
glDisableClientState(GL_NORMAL_ARRAY);                  // Desactivació vector normals
glDisableClientState(GL_COLOR_ARRAY);                   // Desactivació vector color
glDisableClientState(GL_VERTEX_ARRAY);                  // Desactivació vector vèrtexs
glBindBuffer(GL_ARRAY_BUFFER,0);                        // Desactivar l'objecte VBO
glDeleteBuffers(1,&vboId);                              // Eliminar l'objecte VBO
}
```

2.2. VBO: Vectors dinàmics, única comanda



Exemple: Dibuix del cub segons el model de color RGB.

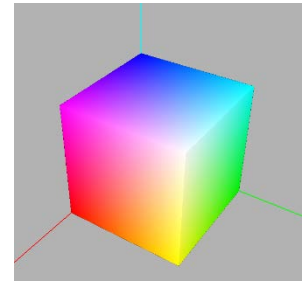
Problema: Cada cop que es dibuixa un cub, ni que sigui el mateix, s'envia la mateixa geometria a la placa

Més òptim: Amb vectors dinàmics, comandes separades:

```
void glutSolidCubeRGB(GLdouble dsize)
{
  GLuint vboId = 0;
  vboId = loadglutSolidCubeRGB_VBO(dsize);      // Carregar geometria a la placa gràfica
  drawSolidCubeRGB(vboId);          // Dibuix de l'objecte (amb la geometria en placa)
  deleteVBO(vboId);                 // Eliminar VBO
}
```

2.2. VBO: Vectors dinàmics, comandes separades

Exemple: Dibuix del cub segons el model de color RGB.

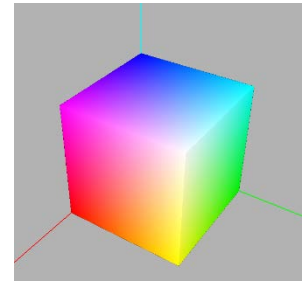


```
// Crear VBO, transferir-hi geometria del cub (un sol cop)
GLuint loadglutSolidCubeRGB_VBO(GLdouble dsize) {
GLdouble size = dSize * 0.5f;              Gluint vboId;
// Vertex Coord Array
std::vector<double> vertices = {size,size,size,-size,size,size,-size,-size,size, // v0-v1-v2 (front)
-size, -size,  size,  size, -size,  size,  size,  size,  size,      // v2-v3-v0
 size,  size,  size,  size, -size,  size,  size, -size, -size,      // v0-v3-v4 (right)
 size, -size, -size,  size,  size, -size,  size,  size,  size,      // v4-v5-v0
 size,  size,  size,  size,  size, -size, -size,  size, -size,      // v0-v5-v6 (top)
-size,  size, -size, -size,  size,  size,  size,  size,  size,      // v6-v1-v0
-size,  size,  size, -size,  size, -size, -size, -size, -size,      // v1-v6-v7 (left)
-size, -size, -size, -size, -size,  size, -size,  size,  size,      // v7-v2-v1
-size, -size, -size,  size, -size, -size,  size, -size,  size,      // v7-v4-v3 (bottom)
 size, -size,  size, -size, -size,  size, -size, -size, -size,      // v3-v2-v7
 size, -size, -size, -size, -size, -size, -size,  size, -size,      // v4-v7-v6 (back)
-size,  size, -size,  size,  size, -size,  size, -size, -size };    // v6-v5-v4

//Normal array
std::vector<double> normals[] = {  0,  0,  1,  0,  0,  1,  0,  0,  1,   // v0-v1-v2 (front)
   0,  0,  1,  0,  0,  1,  0,  0,  1,   // v2-v3-v0
   1,  0,  0,  1,  0,  0,  1,  0,  0,   // v0-v3-v4 (right)
   1,  0,  0,  1,  0,  0,  1,  0,  0,   // v4-v5-v0
   0,  1,  0,  0,  1,  0,  0,  1,  0,   // v0-v5-v6 (top)
   0,  1,  0,  0,  1,  0,  0,  1,  0,   // v6-v1-v0
  -1,  0,  0, -1,  0,  0, -1,  0,  0,   // v1-v6-v7 (left)
  -1,  0,  0, -1,  0,  0, -1,  0,  0,   // v7-v2-v1
   0, -1,  0,  0, -1,  0,  0, -1,  0,   // v7-v4-v3 (bottom)
   0, -1,  0,  0, -1,  0,  0, -1,  0,   // v3-v2-v7
   0,  0, -1,  0,  0, -1,  0,  0, -1,   // v4-v7-v6 (back)
   0,  0, -1,  0,  0, -1,  0,  0, -1 }; // v6-v5-v4
[…]
```

2.2. VBO: Vectors dinàmics, comandes separades

Exemple: Dibuix del cub segons el model de color RGB.

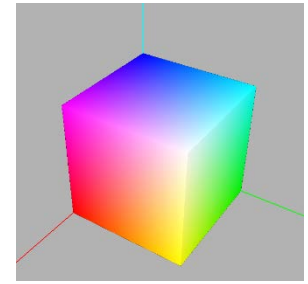


```
// Crear VBO, transferir-hi geometria del cub (un sol cop)
GLuint loadglutSolidCubeRGB_VBO(GLdouble dsize) {
[…]
// Color array
std::vector<double> colors= { 1, 1, 1, 1, 0, 1, 1, 1, 0, 0, 1, 1,   // v0-v1-v2 (front)
0, 0, 1, 1, 1, 0, 1, 1, 1, 1, 1, 1,   // v2-v3-v0
1, 1, 1, 1, 1, 0, 1, 1, 1, 0, 0, 1,   // v0-v3-v4 (right)
1, 0, 0, 1, 1, 1, 0, 1, 1, 1, 1, 1,   // v4-v5-v0
1, 1, 1, 1, 1, 1, 0, 1, 0, 1, 0, 1,   // v0-v5-v6 (top)
0, 1, 0, 1, 0, 1, 1, 1, 1, 1, 1, 1,   // v6-v1-v0
0, 1, 1, 1, 0, 1, 0, 1, 0, 0, 0, 1,   // v1-v6-v7 (left)
0, 0, 0, 1, 0, 0, 1, 1, 0, 1, 1, 1,   // v7-v2-v1
0, 0, 0, 1, 1, 0, 0, 1, 1, 0, 1, 1,   // v7-v4-v3 (bottom)
1, 0, 1, 1, 0, 0, 1, 1, 0, 0, 0, 1,   // v3-v2-v7
1, 0, 0, 1, 0, 0, 0, 1, 0, 1, 0, 1,   // v4-v7-v6 (back)
0, 1, 0, 1, 1, 1, 0, 1, 1, 0, 0, 1 }; // v6-v5-v4

// Texture Array
std::vector<double> textures[] = { 1, 1, 1, 0, 0, 0,      // v0-v1-v2 (front)
1, 0, 0, 1, 1, 1,      // v2-v3-v0
1, 1, 0, 1, 0, 0,      // v0-v3-v4 (right)
0, 0, 1, 0, 1, 1,      // v4-v5-v0
1, 1, 1, 0, 0, 1,      // v0-v5-v6 (top)
0, 1, 1, 0, 1, 1,      // v6-v1-v0
1, 1, 0, 0, 1, 0,      // v1-v6-v7 (left)
0, 0, 0, 1, 1, 1,      // v7-v2-v1
0, 0, 0, 1, 1, 0,      // v7-v4-v3 (bottom)
1, 0, 1, 1, 0, 0,      // v3-v2-v7
0, 0, 1, 0, 1, 0,      // v4-v7-v6 (back)
0, 1, 0, 0, 1, 1 };    // v6-v5-v4
[…]
```

2.2. VBO: Vectors dinàmics, comandes separades

Exemple: Dibuix del cub segons el model de color RGB.



```
// Crear VBO, transferir-hi geometria del cub (un sol cop)
GLuint loadglutSolidCubeRGB_VBO(GLdouble dsize) {
[…]
std::vector <int>::size_type ncv = CubeVertices.size();   // Tamany de CubeVertices
glGenBuffers(1, &vboId);
glBindBuffer(GL_ARRAY_BUFFER, vboId);       // Activar l'objecte VBO

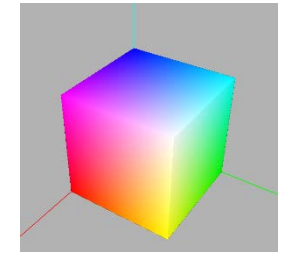
// Reservar buffer per a l'objecte VBO
glBufferData(GL_ARRAY_BUFFER, (CubeVertices.size() + CubeColors.size()) * sizeof(double), 0, GL_STATIC_DRAW);
// Atribut vèrtexs. Copia del vector de vèrtexs a la posició 0
glBufferSubData(GL_ARRAY_BUFFER, 0, vertices.size(), vertices);
// Atribut normals. Copia del vector de normals darrera el vector de vèrtexs
glBufferSubData(GL_ARRAY_BUFFER, vertices.size(), normals.size(), normals);
// Atribut color. Copia del vector de color darrera el vector de normals
glBufferSubData(GL_ARRAY_BUFFER, (vertices.size() + normals.size()), colors.size(), colors);
// Atribut textures. Copia del vector de textires darrera el vector de colors
glBufferSubData(GL_ARRAY_BUFFER, (vertices.size() + normals.size() + colors.size()), textures.size(), textures);

glEnableClientState(GL_TEXTURE_COORD_ARRAY);             // Activació vector textures
glEnableClientState(GL_NORMAL_ARRAY);                    // Activació vector normals
glEnableClientState(GL_COLOR_ARRAY);                     // Activació vector color
glEnableClientState(GL_VERTEX_ARRAY);                    // Activació vector vèrtexs
// Abans de dibuixar especificar els offsets per a cada array (vector)
glTexCoordPointer(2, GL_DOUBLE, 0, (void*)(vertices.size() + normal.size() + colors.size()));
glColorPointer(3, GL_DOUBLE, 0, (void*)(vertices.size() + normal.size()));
glNormalPointer(GL_DOUBLE, 0, (void*)vertices.size());
glVertexPointer(3, GL_DOUBLE, 0, 0);

glBindBuffer(GL_ARRAY_BUFFER,0);      // Desactivar l'objecte VBO
return vboId;
}
```

2.2. VBO: Vectors dinàmics, comandes separades

Exemple: Dibuix del cub segons el model de color RGB.



```
void drawSolidCubeRGB(Gluint vboId) {
if (vboId != -1) {      glBindBuffer(GL_ARRAY_BUFFER, vboId);          // Activar l'objecte VBO
                        glDrawArrays(GL_TRIANGLES, 0, 36);             // Dibuixar el cub
                        glBindBuffer(GL_ARRAY_BUFFER,0);               // Desactivar l'objecte VBO
                }

}


void deleteVBO(Gluint vboId) {
if (vboId != -1) {      glBindBuffer(GL_ARRAY_BUFFER,0);               // Desactivar l'objecte VBO
                        glDeleteBuffers(1,&vboId);                     // Eliminar l'objecte VBO
                }
}
```

2. Vertex Buffer Object (VBO)

Exemple: Dibuix d'una matriu de cubs. Amb única comanda:

```
for (i = 0; i < 10; i++)
    for (j = 0; j < 10; j++)
        for (k = 0; k < 10; k++)
        {    modelMatrix = glm::translate(matriuTG, vec3(i * 15.0f, j * 15.0f, k * 15.0f));
             // Pas ModelViewMatrix a shader
             glUniformMatrix4fv(glGetUniformLocation(sh_programID,"modelMatrix"), 1, GL_FALSE,&ModelMatrix[0][0]);
             // Pas NormalMatrix a shader
             NormalMatrix = transpose(inverse(MatriuVista * ModelMatrix));
             glUniformMatrix4fv(glGetUniformLocation(sh_programID,"normalMatrix"),1,GL_FALSE,&NormalMatrix[0][0]);
             glutSolidCubeRGB(5.0);
        }

ModelMatrix=matriuTG;
for (i = 0; i < 10; i++)
    for (j = 0; j < 10; j++)
    {    ModelMatrix = glm::translate(ModelMatrix, vec3(i * 15.0f, j * 15.0f, 200.0));
         // Pas ModelViewMatrix a shader
         glUniformMatrix4fv(glGetUniformLocation(sh_programID,"modelMatrix"),1,GL_FALSE,&ModelMatrix[0][0]);
         // Pas NormalMatrix a shader
         NormalMatrix = transpose(inverse(MatriuVista * ModelMatrix));
         glUniformMatrix4fv(glGetUniformLocation(sh_programID,"normalMatrix"),1,GL_FALSE,&NormalMatrix[0][0]);
         glutSolidTeapot();
    }
```

Per a cada còpia, es transfereixen els vectors de geometria (vèrtexs, normals, color, textures) a la GPU i es dibuixa → **MOLT LENT**

2. Vertex Buffer Object (VBO)

Exemple: Dibuix d'una matriu de cubs. Amb comandes separades:

```
GLuint = loadglutSolidCubeRGB_VBO(5.0);
for (i = 0; i < 10; i++)
    for (j = 0; j < 10; j++)
        for (k = 0; k < 10; k++)
        {     modelMatrix = glm::translate(matriuTG, vec3(i * 15.0f, j * 15.0f, k * 15.0f));
              // Pas ModelViewMatrix a shader
              glUniformMatrix4fv(glGetUniformLocation(sh_programID,"modelMatrix"),1,GL_FALSE,&ModelMatrix[0][0]);
              // Pas NormalMatrix a shader
              NormalMatrix = transpose(inverse(MatriuVista * ModelMatrix));
              glUniformMatrix4fv(glGetUniformLocation(sh_programID,"normalMatrix"),1,GL_FALSE,&NormalMatrix[0][0]);
              drawSolidCubeRGB(vboId);
        }
deleteVBO(vboId);           // Eliminar VBO Cub RGB

ModelMatrix=matriuTG;
vboId = loadglutSolidTeapot_VBO();
for (i = 0; i < 10; i++)
    for (j = 0; j < 10; j++)
    {    ModelMatrix = glm::translate(ModelMatrix, vec3(i * 15.0f, j * 15.0f, 200.0));
         // Pas ModelViewMatrix a shader
         glUniformMatrix4fv(glGetUniformLocation(sh_programID,"modelMatrix"),1,GL_FALSE,&ModelMatrix[0][0]);
         // Pas NormalMatrix a shader
         NormalMatrix = transpose(inverse(MatriuVista * ModelMatrix));
         glUniformMatrix4fv(glGetUniformLocation(sh_programID,"normalMatrix"),1,GL_FALSE,&NormalMatrix[0][0]);
         drawSolidTeapot(vboId);
    }
deleteVBO(vboId);           // Eliminar VBO Tetera
```

Es transfereixen els vectors de geometria (vèrtexs, normals, color, textures) a la GPU un únic cop i es dibuixa cada còpia amb la geometria en la GPU.

2. Vertex Buffer Object (VBO)

PROBLEMA: Només podem visualitzar un VBO a la vegada.

En una escena normal pot haver cents de models 3D diferents a dibuixar, pel que si per a cada model hem de fer configurar (vèrtex, color, normals, etc.) i dibuixar en VBO's de forma seqüencial, el procés és tediós i lent.



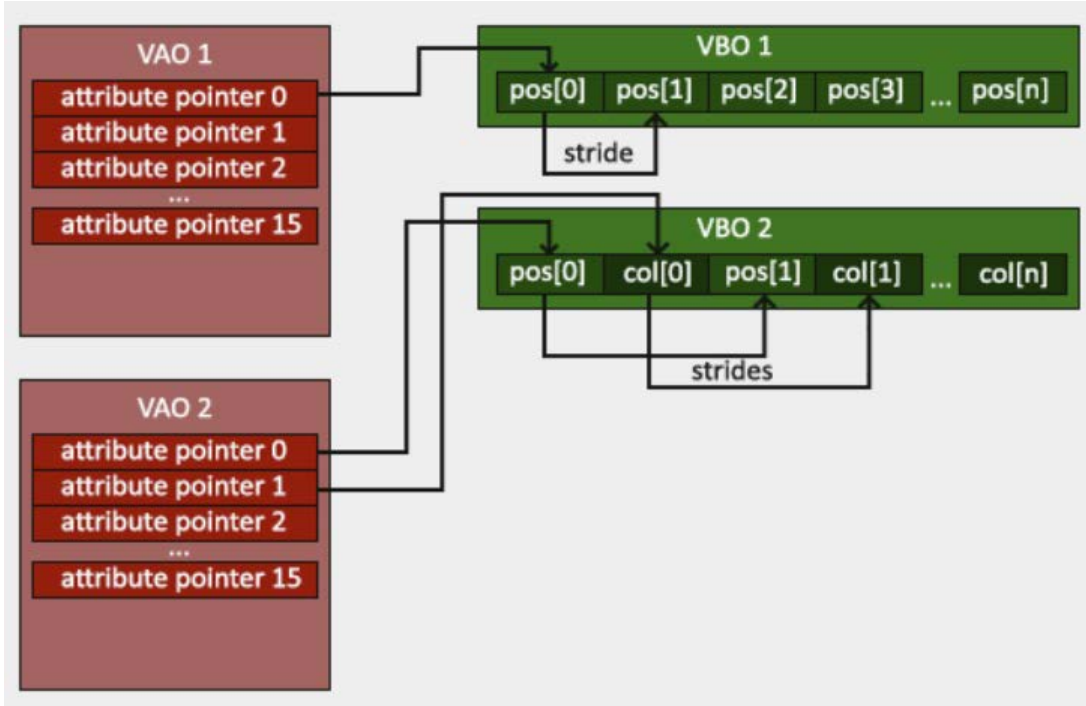


SOLUCIÓ: VERTEX ARRAY OBJECT (VAO). Tenir diferents models 3D carregats en GPU i dibuixar en cada moment el que interessa.

3. Vertex Array Object (VAO)

Objecte OpenGL que conté un VBO, per a emmagatzemar els atributs d'un model 3D (vèrtexs, normals, color, textures) en el hardware gràfic.

Es creen i configuren varis VAO's (un per objecte) en hardware gràfic un sol cop i dibuixem en cada moment el que volem.



3. Vertex Array Object (VAO)

Passos i comandes:

- **Generar VAO's amb identificador → `glGenVertexArrays()`**
- **Activar el VAO a configurar o visualitzar → `glBindVertexArray()`**
- Generar buffer VBO amb identificador → `glGenBuffers()`
- Activar el buffer VBO a configurar o visualitzar → `glBindBuffer()`
- Reservar, i transferir dades al VBO → `glBufferData()`
- Com interpretar les dades del VBO → `glVertexAttribPointer()`
- **Activar atribut al VAO → `glEnableVertexAttribArray()`**
- **Dibuixar l'objecte contingut en el VAO → `glDrawArrays()`**
- **Desactivar atribut al VAO → `glDisableVertexAttribArray()`**
- **Eliminar VAO's amb identificador → `glDeleteVertexArrays()`**

3. Vertex Array Object (VAO)

```
void glGenVertexArrays(Glsize n, Gluint *arrays);
```

Acció: Genera un vector de noms de VAO's.

```
void glDeleteVertexArrays(Glsize n, Gluint *arrays);
```

Acció: Esborra els VAO's del vector noms i el propi vector.

Paràmetres:

`n`: Nombre de posicions del vector de noms.

`arrays`: Vector sobre el que es generen els noms.

3. Vertex Array Object (VAO)

```
void glBindVertexArray(GLuint array);
```

Acció: Activa el VAO identificat per array.

Paràmetres:

`array`: Identificador del VAO a activar. Si té el valor 0, desactiva el VAO que estigui activat.

3. Vertex Array Object (VAO)

```
void gEnableVertexAttribArray(Gluint index);
```

Acció: Activa un atribut (contingut en un VBO) del Vertex Array per a que s'utilitzi en el renderitzat del VAO.

```
void gDisableVertexAttribArray(Gluint index);
```

Acció: Desactiva un atribut (contingut en un VBO) del Vertex Array.

Paràmetres:

`index`: Índex del atribut del VAO a activar o desactivar. Usualment pren els valors (0,1,2,3) per a vèrtexs, normals, textures, colors.

3. Vertex Array Object (VAO)

```
void glDrawArrays(GLenum mode, Glint first, GLsizei count);
```

Acció: Dibuixa l'objecte contingut en el VAO, segons la primitiva a dibuixar.

Paràmetres:

`mode`: Especifica quina primitiva es vol dibuixar. Pot tenir els valors: `GL_POINTS`, `GL_LINE_STRIP`, `GL_LINE_LOOP`, `GL_LINES`, `GL_TRIANGLE_STRIP`, `GL_TRIANGLE_FAN`, `GL_TRIANGLES`, `GL_QUAD_STRIP`, `GL_QUADS`, `GL_PATCHES` i `GL_POLYGON`.

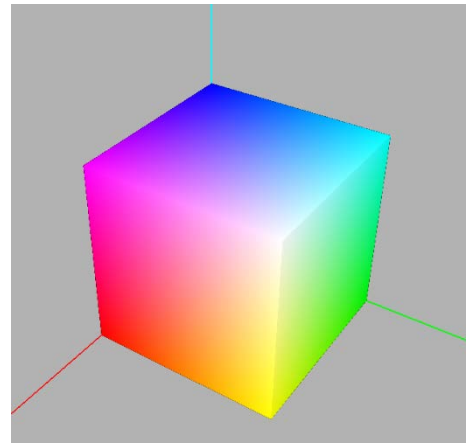
`GL_LINE_STRIP_ADJACENCY`, `GL_LINES_ADJACENCY`, `GL_TRIANGLE_STRIP_ADJACENCY` i `GL_TRIANGLES_ADJACENCY` per a OpenGL 3.3 o superior.

`first`: Index d'inici dins el VBO.

`count`: Nombre de punts a dibuixar.

3. Vertex Array Object (VAO)

Exemple: Dibuix del cub segons el model de color RGB.



Diferents solucions amb vectors dinàmics i comandes separades:

1. Una comanda de dibuix per objecte (`draw*()`)
2. Única comanda de dibuix per a tots els objectes

3.1. VAO: Comanda draw*() per objecte

Una comanda de dibuix pròpia per a cada objecte → funció draw*()

PASSOS:

1. Crear estructura CVAO que contingui totes les variables de VAO

```
// --------------- VGI: Estructura LLista VAO
struct CVAO
{            GLuint vaoId;            // Identificador del VAO
             GLuint vboId;            // Identificador del VBO
             GLint nVertexs;          // Nombre de vèrtexs del VAO
};
```

2. Crear vector de tipus CVAO (`VAOList[]`) on guardar paràmetres de cada objecte a visualitzar.

```
//--------------- VGI: Màxim tamany vector VAOList
#define MAX_SIZE_VAOID 45

// VAOList: Vector d'identificadors de Vertex Buffer Objects per a primitives glut_geometry.
// Cada posició del vector correspon a una primitiva: 0: CUBE_SKYBOX, 1: GLUT_CUBE, 2:GLUT_CUBE_RGB, 3:GLUT_SPHERE, 4: GLUT_CONE,
// 5: GLUT_CYLINDER, 6: GLUT_TORUS, 7: GLUT_DODECAHEDRON, 8: GLUT_OCTAHEDRON, 9: GLUT_TETRAEDRON, 10: GLUT_ICOSAHEDRON,
// 11: GLUT_RHOMBICDODECAHEDRON,  12: GLUT_SIERPINSKISPONGE, 13: GLU_TEAPOT, 14: GLU_CYLINDER, 15: GLU_DISK, 16: GLU_PARTIALDISK,
// 17: GLU_SPHERE, 18: CRV_POLYLINE, 19: CRV_LEMNISCATA2D, 20: CRV_LEMNISCATA3D, 21: CRV_BSPLINE, 22: CRV_BEZIER, 23:SUP_BSPLINE,
// 24: SUP_BEZIER, 25: GLUT_LINES, 26: GLUT_TRIANGLES, 27: GLU_CILINDRE_SENCER, 28: GLUT_SQUARE, 29: MAR_FRACTAL_VAO,
// 30: O_FRACTAL_VAO, 31: FIT_3DS, 32: FIT_3DS2, 33: FIT_OBJ, 34:FIT_OBJ2, 35:GLUT_USER1, 36: GLUT_USER2, 37: GLUT_USER3,
// 38: GLUT_USER4, 39: GLUT_USER5, 40: GLUT_USER6. 41: GLUT_USER7, 42: GLUT_USER8, 43: GLUT_USER9, 44: GLUT_USER10.
CVAO VAOList[MAX_SIZE_VAOID];
```

3.1. VAO: Comanda draw*() per objecte

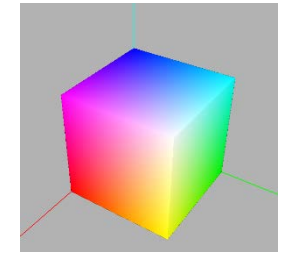
2. Funcions per a inicialitzar vector (`initVAOList()`), configurar un element del vector (`SetVAOList()`) i esborrar un element del vector (`deleteVAOList()`)

```
void initVAOList()
{  int i;
   for (i = 0; i < MAX_SIZE_VAOID; i++){
      VAOList[i].vaoId = 0;
      VAOList[i].vboId = 0;
      VAOList[i].nVertexs = 0;
      }
}
void Set_VAOList(GLint k, CVAO auxVAO)
{  VAOList[k].vaoId = auxVAO.vaoId;
   VAOList[k].vboId = auxVAO.vboId;
   VAOList[k].nVertexs = auxVAO.nVertexs;
}

void deleteVAOList(GLint k)
{ if (VAOList[k].vaoId != 0)  ç{
   glBindVertexArray(VAOList[k].vaoId); // Activació VAO
   glDisableVertexAttribArray(0); glDisableVertexAttribArray(1); glDisableVertexAttribArray(2);
   glDisableVertexAttribArray(3);
   // It is good idea to release VBOs with ID 0 after use.
   glBindBuffer(GL_ARRAY_BUFFER, 0);
   glDeleteBuffers(1, &VAOList[k].vboId); // Esborrar buffer VBO
   // Unbind and delete VAO
   glBindVertexArray(0);
   glDeleteVertexArrays(1, &VAOList[k].vaoId);
   VAOList[k].vboId = 0; VAOList[k].vaoId = 0; VAOList[k].nVertexs = 0; // Eliminar valors de la posició VAOList[]    }
}
```

3.1. VAO: Comanda draw*() per objecte

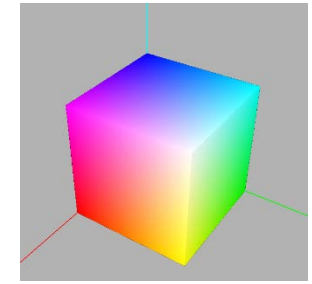
Exemple: Dibuix del cub segons el model de color RGB.



```
void glutSolidCubeRGB(GLdouble dsize)
{
  Set_VAOList(GLUT_CUBE_RGB, loadglutSolidCubeRGB_VAO(dsize));
  drawSolidCubeRGB();
  deleteVAOList(GLUT_CUBE_RGB);
}
```

3.1. VAO: Comanda draw*() per objecte

Exemple: Dibuix del cub segons el model de color RGB.

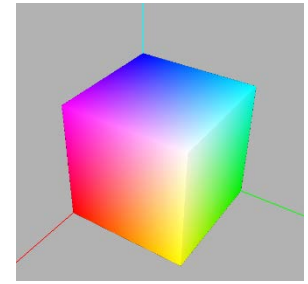


```
// Crear VAO i VBOs, transferir-hi geometria del cub (un sol cop)
CVAO loadglutSolidCubeRGB_VAO(GLdouble dsize) {
GLdouble size = dSize * 0.5f;
// ----------------------- VAO
CVAO auxVAO;   auxVAO.vaoId = 0;auxVAO.vboId = 0;auxVAO.nVertexs = 0;
// Vertex Coord Array
std::vector<double> vertices = {size,size,size,-size,size,size,-size,-size,size, // v0-v1-v2 (front)
-size, -size,  size,  size, -size,  size,  size,  size,  size,      // v2-v3-v0
 size,  size,  size,  size, -size,  size,  size, -size, -size,      // v0-v3-v4 (right)
 size, -size, -size,  size,  size, -size,  size,  size,  size,      // v4-v5-v0
 size,  size,  size,  size,  size, -size, -size,  size, -size,      // v0-v5-v6 (top)
-size,  size, -size, -size,  size,  size,  size,  size,  size,      // v6-v1-v0
-size,  size,  size, -size,  size, -size, -size, -size, -size,      // v1-v6-v7 (left)
-size, -size, -size, -size, -size,  size, -size,  size,  size,      // v7-v2-v1
-size, -size, -size,  size, -size, -size,  size, -size,  size,      // v7-v4-v3 (bottom)
 size, -size,  size, -size, -size,  size, -size, -size, -size,      // v3-v2-v7
 size, -size, -size, -size, -size, -size, -size,  size, -size,      // v4-v7-v6 (back)
-size,  size, -size,  size,  size, -size,  size, -size, -size };    // v6-v5-v4

// Normal array
std::vector<double> normals = {
 0,  0,  1,  0,  0,  1,  0,  0,  1,   // v0-v1-v2 (front)
 0,  0,  1,  0,  0,  1,  0,  0,  1,   // v2-v3-v0
 1,  0,  0,  1,  0,  0,  1,  0,  0,   // v0-v3-v4 (right)
 1,  0,  0,  1,  0,  0,  1,  0,  0,   // v4-v5-v0
 0,  1,  0,  0,  1,  0,  0,  1,  0,   // v0-v5-v6 (top)
 0,  1,  0,  0,  1,  0,  0,  1,  0,   // v6-v1-v0
-1,  0,  0, -1,  0,  0, -1,  0,  0,   // v1-v6-v7 (left)
-1,  0,  0, -1,  0,  0, -1,  0,  0,   // v7-v2-v1
 0, -1,  0,  0, -1,  0,  0, -1,  0,   // v7-v4-v3 (bottom)
 0, -1,  0,  0, -1,  0,  0, -1,  0,   // v3-v2-v7
 0,  0, -1,  0,  0, -1,  0,  0, -1,   // v4-v7-v6 (back)
 0,  0, -1,  0,  0, -1,  0,  0, -1 }; // v6-v5-v4
[...]
```

3.1. VAO: Comanda draw*() per objecte

Exemple: Dibuix del cub segons el model de color RGB.



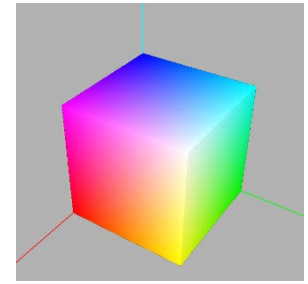
```
// Crear VAO i VBOs, transferir-hi geometria del cub (un sol cop)
CVAO loadglutSolidCubeRGB_VAO(GLdouble dsize) {

[...]
// Color array
std::vector<double> colors= { 1, 1, 1, 1, 0, 1, 1, 1, 0, 0, 1, 1,   // v0-v1-v2 (front)
0, 0, 1, 1, 1, 0, 1, 1, 1, 1, 1, 1,   // v2-v3-v0
1, 1, 1, 1, 1, 0, 1, 1, 1, 0, 0, 1,   // v0-v3-v4 (right)
1, 0, 0, 1, 1, 1, 0, 1, 1, 1, 1, 1,   // v4-v5-v0
1, 1, 1, 1, 1, 1, 0, 1, 0, 1, 0, 1,   // v0-v5-v6 (top)
0, 1, 0, 1, 0, 1, 1, 1, 1, 1, 1, 1,   // v6-v1-v0
0, 1, 1, 1, 0, 1, 0, 1, 0, 0, 0, 1,   // v1-v6-v7 (left)
0, 0, 0, 1, 0, 0, 1, 1, 0, 1, 1, 1,   // v7-v2-v1
0, 0, 0, 1, 1, 0, 0, 1, 1, 0, 1, 1,   // v7-v4-v3 (bottom)
1, 0, 1, 1, 0, 0, 1, 1, 0, 0, 0, 1,   // v3-v2-v7
1, 0, 0, 1, 0, 0, 0, 1, 0, 1, 0, 1,   // v4-v7-v6 (back)
0, 1, 0, 1, 1, 1, 0, 1, 1, 0, 0, 1 }; // v6-v5-v4

// Texture Array
std::vector<double> textures = {
1, 1, 1, 0, 0, 0,      // v0-v1-v2 (front)
0, 0, 0, 1, 1, 1,      // v2-v3-v0
1, 1, 0, 1, 0, 0,      // v0-v3-v4 (right)
0, 0, 1, 0, 1, 1,      // v4-v5-v0
0, 1, 0, 0, 1, 0,      // v0-v5-v6 (top)
1, 0, 1, 1, 0, 1,      // v6-v1-v0
0, 1, 0, 0, 1, 0,      // v1-v6-v7 (left)
1, 0, 1, 1, 0, 1,      // v7-v2-v1
0, 0, 1, 0, 1, 1,      // v7-v4-v3 (bottom)
1, 1, 0, 1, 0, 0,      // v3-v2-v7
0, 0, 1, 0, 1, 1,      // v4-v7-v6 (back)
1, 1, 0, 1, 0, 0 };    // v6-v5-v4
[...]
```

3.1. VAO: Comanda draw*() per objecte

Exemple: Dibuix del cub segons el model de color RGB.



```
// Crear VAO, transferir-hi geometria del cub (un sol cop)
CVAO loadlutSolidCubeRGB_VAO(GLdouble dsize) {

[...]

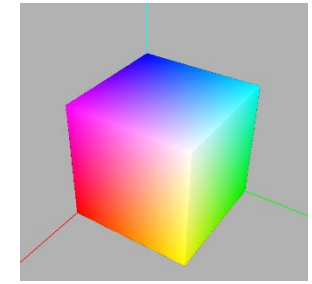
// Creació d'un VAO i un VBO i càrrega de la geometría a la placa gràfica. Guardar identificador VAO.
auxVAO = load_TRIANGLES_VAO(vertices, normals, colors, textures);

return auxVAO;
}
```

`load_TRIANGLES_VAO()`: Funció que crea els buffers VAO i VBO, hi assigna la geometria continguda en els vectors `vertices`, `normals`, `colors`, `textures` i la transfereix a la placa gràfica. Retorna els identificadors i el nombre de vèrtexs amb l'estructura CVAO.

3.1. VAO: Comanda draw*() per objecte

Exemple: Dibuix del cub segons el model de color RGB.



```
// ---------- TRIANGLES VAO
CVAO load_TRIANGLES_VAO(std::vector <double> vertices, std::vector <double> normals, std::vector <double> colors,
std::vector <double> textures) {

// ----------------------- VAO
CVAO tmpVAO;
tmpVAO.vaoId = 0;tmpVAO.vboId = 0;tmpVAO.nVertexs = 0;

std::vector <int>::size_type nv = vertices.size();// Tamany del vector vertices en elements.

// Create Vertex Array Object (VAO) for 3D Model Cube
glGenVertexArrays(1, &tmpVAO.vaoId);

// Create vertex buffer objects for 3D Model attributes in the VAO
glGenBuffers(1, &tmpVAO.vboId);

// Bind our Vertex Array Object as the current used object
glBindVertexArray(tmpVAO.vaoId);

// Bind our Vertex Buffer Object as the current used object
glBindBuffer(GL_ARRAY_BUFFER, tmpVAO.vboId);

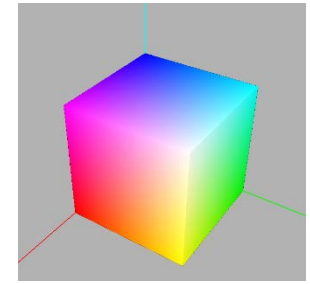
glBufferData(GL_ARRAY_BUFFER, (vertices.size() + normals.size() + textures.size() + colors.size()) * sizeof(double),
0, GL_STATIC_DRAW);

// Position Vertex attributes
glBufferSubData(GL_ARRAY_BUFFER, 0, vertices.size() * sizeof(double), &vertices[0]);// Copy geometry data to VBO
starting from 0 offset
glEnableVertexAttribArray(0);// Enable attribute index 0 as being used (position)
glVertexAttribPointer(0, 3, GL_DOUBLE, GL_FALSE, 3 * sizeof(double), 0); // Specify that our coordinate data is going
into attribute index 0 and contains 3 double

[...]
```

3.1. VAO: Comanda draw*() per objecte

Exemple: Dibuix del cub segons el model de color RGB.



```
// ---------- TRIANGLES VAO
CVAO load_TRIANGLES_VAO(std::vector <double> vertices, std::vector <double> normals, std::vector <double> colors,
std::vector <double> textures) {
[...]

// Normal Vertex Attributes
glBufferSubData(GL_ARRAY_BUFFER, vertices.size() * sizeof(double), normals.size() * sizeof(double), &normals[0]);// Copy normal data to VBO
starting from 0 offest
glEnableVertexAttribArray(1); // Enable attribute index 1 as being used (normals)
glVertexAttribPointer(1, 3, GL_DOUBLE, GL_FALSE, 3 * sizeof(double), (GLvoid*)(vertices.size() * sizeof(double))); // Specify that our
color data is going into attribute index 0 and contains 3 double

// Texture Coordinates Vertex Attributes
// Copy normal data to VBO starting from 0 offset
glBufferSubData(GL_ARRAY_BUFFER, (vertices.size() + normals.size()) * sizeof(double), textures.size() * sizeof(double), &textures[0]);

glEnableVertexAttribArray(2);              // Enable attribute index 2 as being used (texture coordinates)
// Specify that our color data is going into attribute index 0 and contains 3 double
glVertexAttribPointer(2, 2, GL_DOUBLE, GL_FALSE, 2 * sizeof(double), (GLvoid*)((vertices.size() + normals.size()) * sizeof(double)));

// Color Vertex Attributes
glBufferSubData(GL_ARRAY_BUFFER, (vertices.size() + normals.size() + textures.size()) * sizeof(double), colors.size() * sizeof(double),
&colors[0]);                               // Copy normal data to VBO starting from 0 offest
glEnableVertexAttribArray(3);              // Enable attribute index 3 as being used
glVertexAttribPointer(3, 4, GL_DOUBLE, GL_FALSE, 4 * sizeof(double), (GLvoid*)((vertices.size() + normals.size() + textures.size()) *
sizeof(double)));            // Specify that our color data is going into attribute index 0 and contains 3 double

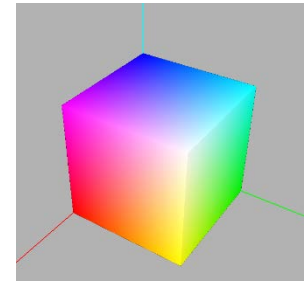
// Unbind VAO, to prevent bugs
glBindVertexArray(0);

tmpVAO.nVertexs = (int)vertices.size() / 3;
tmpVAO.nIndices = 0;

return tmpVAO; // retorna CVAO.
}
```

3.1. VAO: Comanda draw*() per objecte

Exemple: Dibuix del cub segons el model de color RGB.



```
void drawSolidCubeRGB() {
if (VAOList[GLUT_CUBE_RGB].vaoId != -1) {
            glBindBufferArray(VAOList[GLUT_CUBE_RGB].vaoId);                // Activar l'objecte VAO
            glDrawArrays(GL_TRIANGLES, 0, VAOList[GLUT_CUBE_RGB].nVertexs); // Dibuixar el cub
            glBindBufferArray(0);                                           // Desactivar l'objecte VAO
            }
}
```

3.2. VAO: Única comanda de dibuix

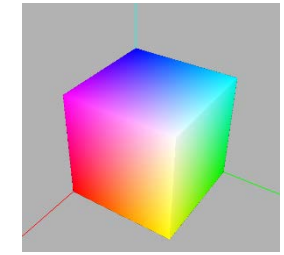
Unica comanda de dibuix per a tots els objectes → funció `draw_TriVAO_Object()`

PASSOS:

1. Mantenir estructura CVAO que contingui totes les variables de VAO
2. Mantenir vector de tipus CVAO (`VAOList[]`) on guardar paràmetres de cada objecte a visualitzar.
3. Mantenir les funcions per a inicialitzar vector (`initVAOList()`), configurar un element del vector (`SetVAOList()`), esborrar un element del vector (`deleteVAOList()`)

3.2. VAO: Única comanda de dibuix

Exemple: Dibuix del cub segons el model de color RGB.



```
void glutSolidCubeRGB(GLdouble dsize)
{
  Set_VAOList(GLUT_CUBE_RGB, loadglutSolidCubeRGB_VAO(dsize));
  draw_TriVAO_Object(GLUT_CUBE_RGB);
  deleteVAOList(GLUT_CUBE_RGB);
}
```

```
void draw_TriVAO_Object(GLint k) {
// Recuperar identificadors VAO i nvertexs dels vector VAOList pr a dibuixar VAO
if (VAOList[k].vaoId != 0) {
            glBindVertexArray(VAOList[k].vaoId);                // Activar l'objecte VAO
            glDrawArrays(GL_TRIANGLES, 0, VAOList[k].nVertexs); // Dibuixar el cub
            glBindVertexArray(0);                               // Desactivar l'objecte VAO

            }
}
```

3.2. VAO: Única comanda de dibuix

Si tenim varis models 3D a visualitzar:

1. Generem-configurem els VAO's i els respectius VBO's, transferint-ho al hardware gràfic (funcions `load*_VAO()`)
2. Per a renderitzar una còpia (funció `drawTriVAO_Object*()`):
   a) *Bind* el corresponent VAO
   b) Renderitzem (`glDrawArrays()`)
   c) *Unbind* el corresponent VAO
3. Quan haguem acabat de renderitzar còpies: (funció `deleteVAOList()`):
   a) *Unbind* el corresponent VAO de la llista `VAOList[]`
   b) Esborrar el VAO (`glDeleteVertexArrays()`)

3.2. VAO: Única comanda de dibuix

Exemple: Dibuix d'una matriu de cubs.

```
SetVAOList(GLUT_CUBE_RGB, loadglutSolidCubeRGB_VAO(5.0));
SetVAOList(GLUT_TEAPOT, loadglutSolidTeapot_VAO());
ModelMatrix=matriuTG;
for (i = 0; i < 10; i++)
    for (j = 0; j < 10; j++)
        for (k = 0; k < 10; k++)
        { ModelMatrix = glm::translate(ModelMatrix, vec3(i * 15.0f, j * 15.0f, k * 15.0f));
          // Pas ModelViewMatrix a shader
          glUniformMatrix4fv(glGetUniformLocation(sh_programID,"modelMatrix"),1,GL_FALSE,&ModelMatrix[0][0]);
          // Pas NormalMatrix a shader
          NormalMatrix = transpose(inverse(MatriuVista * ModelMatrix));
          glUniformMatrix4fv(glGetUniformLocation(sh_programID,"normalMatrix"),1,GL_FALSE,&NormalMatrix[0][0]);
          drawTriVAO_Object(GLUT_CUBE_RGB);
        }

ModelMatrix=matriuTG;
for (i = 0; i < 10; i++)
    for (j = 0; j < 10; j++)
    {    ModelMatrix = glm::translate(ModelMatrix, vec3(i * 15.0f, j * 15.0f, 200.0));
         // Pas ModelViewMatrix a shader
         glUniformMatrix4fv(glGetUniformLocation(sh_programID,"modelMatrix"),1,GL_FALSE,&ModelMatrix[0][0]);
         // Pas NormalMatrix a shader
         NormalMatrix = transpose(inverse(MatriuVista * ModelMatrix));
         glUniformMatrix4fv(glGetUniformLocation(sh_programID,"normalMatrix"),1,GL_FALSE,&NormalMatrix[0][0]);
         drawTriVAO_Object(GLUT_TEAPOT);
    }
deleteVAOList(GLUT_CUBE_RGB);       // Eliminar Cub RGB
deleteVAOList(GLUT_TEAPOT);         // Eliminar Tetera
```

3.2. VAO: Única comanda de dibuix

També funció `drawLinVAO_Object*()` per a dibuixar segments de recta, utilitzada en el dibuix de corbes.

```
void draw_LinVAO_Object(GLint k)
{
// Recuperar identificador VAO a vector VAOId
if (VAOList[k].vaoId != 0) {
           glBindVertexArray(VAOList[k].vaoId);
           glDrawArrays(GL_LINES, 0, VAOList[k].nVertexs);
           glBindVertexArray(0);
           }
}
```

4. Element Buffer Object (EBO)

Objecte OpenGL que permet incloure un vector d'índex com atribut al model 3D per a disminuir el nombre de vèrtexs repetits que s'envien al hardware gràfic en el VBO.

Exemple: Cub de 12 triangles (6 quadrats x 2 triangles) cares amb atributs de vèrtexs, normals, textura i color.

Nombre d'elements en VAO **sense EBO**:

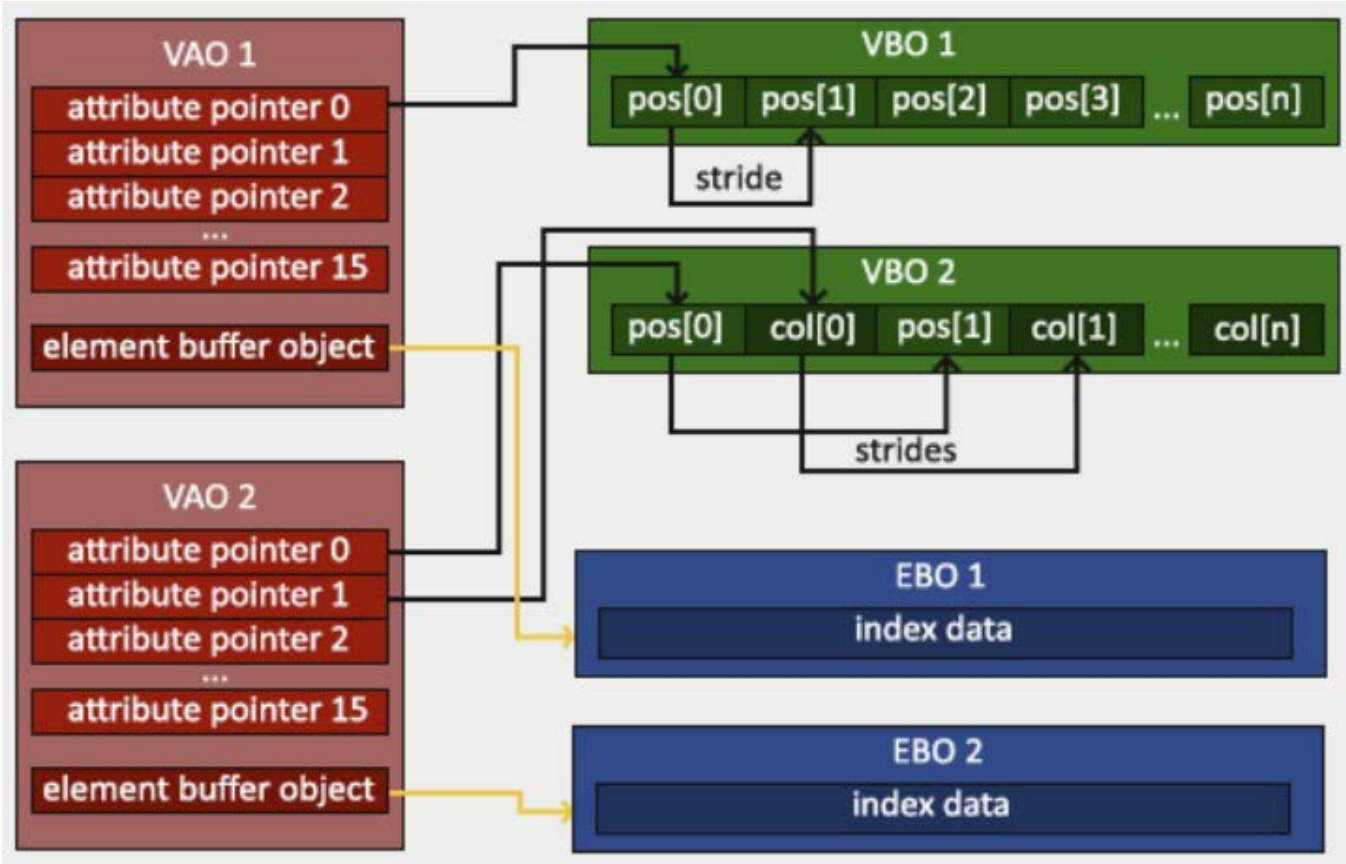
- Vèrtexs: 12 triangles x 3 vèrtexs x 3 coordenades *(x,y,z)* = 108 `GLdouble`
- Normal: 12 triangles x 3 vèrtexs x 3 coordenades *(x,y,z)* = 108 `GLdouble`
- Textura: 12 triangles x 3 vèrtexs x 2 coordenades *(s,t)* = 72 `GLdouble`
- Color:12 triangles x3vèrtexs x4 coordenades*(r,g,b,a)*=144`GLdouble`→**TOTAL:432 `GLdouble`**

Nombre d'elements en VAO **amb EBO**:

- Vèrtexs: 6 quadrats x 4 vèrtexs x 3 coordenades *(x,y,z)* = 72 `GLdouble`
- Normal: 6 quadrats x 4 vèrtexs x 3 coordenades *(x,y,z)* = 72 `GLdouble`
- Textura: 6 quadrats x 4 vèrtexs x 2 coordenades *(s,t)* = 48 `GLdouble`
- Color: 6 quadrats x 4 vèrtexs x 4 coordenades *(r,g,b,a)* = 96 `GLdouble`
- Vector índexs: 12 cares x 3 vèrtexs x 1 índex = 36 `GLuint` → **TOTAL: 288 + 9 `GLdouble`**

Un 33,3% de reducció.

4. Element Buffer Object (EBO)



Qualsevol geometria 3D, sigui fixe (`gluSolidCube(size)`), com variable (`gluSphere(radius, slices, stacks)`), es pot representar tant en VAO com en EBO, incloent corbes i superfícies (Bezier, B-Spline, etc.).

4. Element Buffer Object (EBO)

Passos i comandes:

- Generar VAO's amb identificador → `glGenVertexArrays()`
- Activar el VAO a configurar o visualitzar → `glBindVertexArray()`
- Generar buffers VBO i **EBO** amb identificador → **`glGenBuffers()`**
- Activar el buffer VBO a configurar o visualitzar → `glBindBuffer()`
- Reservar, i transferir dades al VBO → `glBufferData()`
- Com interpretar les dades del VBO → `glVertexAttribPointer()`
- Activar atribut al VAO → `glEnableVertexAttribArray()`
- **Reserva, i transfereix dades al EBO → `glBufferData()`**
- **Com interpretar les dades del EBO → `glVertexAttribPointer()`**
- **Dibuixar l'objecte contingut en el VAO → `glDrawElements()`**
- Desactivar atribut al VAO → `glDisableVertexAttribArray()`
- Eliminar VAO's amb identificador → `glDeleteVertexArrays()`

4. Element Buffer Object (EBO)

```
void glGenBuffers(GLsizei n, GLuint *buffers);
```

Acció: Genera vector d'identificadors de EBO i els propis EBO.

```
void glDeleteBuffers(GLsize n, const GLuint *buffers);
```

Acció: Elimina els buffers EBO i el vector d'identificadors.

Paràmetres:

`n`: Nombre de buffers a generar o eliminar.

`buffers`: Vector on es guarden els identificadors de EBO.

4. Element Buffer Object (EBO)

```
void glBindBuffer(GLenum target, GLuint buffer);
```

Acció: Activa el buffer EBO identificat per `buffer`.

Paràmetres:

`target`: Tipus d'informació. Pot tenir entre altres valors:

`GL_ELEMENT_ARRAY_BUFFER`: Índexs a vèrtexs (EBO).

`GL_ARRAY_BUFFER`: Atributs de vèrtexs (VBO).

`buffer`: Identificador de l'element *Buffer Object*. **Si té el valor 0, desactiva el EBO que estigui activat.**

4. Element Buffer Object (EBO)

```
void glBufferData(GLenum target, GLsizeiptr size, const GLvoid *data, GLenum usage);
```

Acció: Crea i inicialitza les dades d'un EBO. Per introduir el primer atribut de vèrtex.

Paràmetres:

`target`: Tipus d'informació. Pot tenir entre altres valors:

- **`GL_ELEMENT_ARRAY_BUFFER`: Índexs a vèrtexs (EBO).**
- `GL_ARRAY_BUFFER`: Atributs de vèrtexs (VBO).
- `GL_TEXTURE_BUFFER`: Dades de textura.

`size`: Especifica el tamany de les dades a introduir.

`data`: Apuntador a la taula que conté els valors a introduir.

`usage`: Especifica el tipus de dades a emmagatzemar. Pot tenir els valors: `GL_STREAM_DRAW`, `GL_STREAM_READ`, `GL_STREAM_COPY`, `GL_STATIC_DRAW`, `GL_STATIC_READ`, `GL_STATIC_COPY`, `GL_DYNAMIC_DRAW`, `GL_DYNAMIC_READ`, o `GL_DYNAMIC_COPY`.

4. Element Buffer Object (EBO)

```
void glDrawElements(GLenum mode, Glsizei count, GLenum type, const void* indices);
```

Acció: Dibuixa l'objecte contingut en els buffers VAO, VBO i EBO.

Paràmetres:

`mode`: Especifica quina primitiva es vol dibuixar. Pot tenir els valors: `GL_POINTS`, `GL_LINE_STRIP`, `GL_LINE_LOOP`, `GL_LINES`, `GL_TRIANGLE_STRIP`, `GL_TRIANGLE_FAN`, `GL_TRIANGLES`, `GL_QUAD_STRIP` i `GL_PATCHES`.

`GL_LINE_STRIP_ADJACENCY`, `GL_LINES_ADJACENCY`, `GL_TRIANGLE_STRIP_ADJACENCY` i `GL_TRIANGLES_ADJACENCY` per a OpenGL 3.2 o superior.

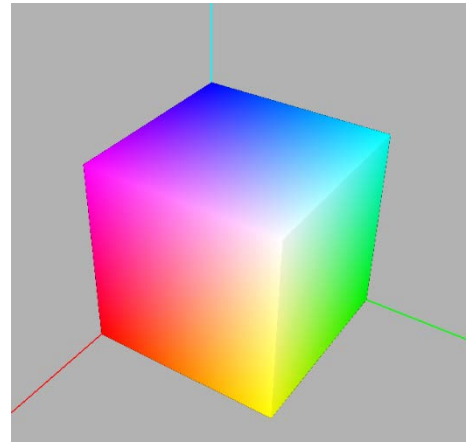
`count`: Nombre de punts a dibuixar.

`type`: Tipus dels índexs. Pot agafar els valors `GL_UNSIGNED_INT`, `GL_UNSIGNED_SHORT`, `GL_UNSIGNED_BYTE`.

`indices`: Apuntador al vector d'índexs.

4. Element Buffer Object (EBO)

Exemple: Dibuix del cub segons el model de color RGB.



Única solució amb vectors dinàmics i comandes separades:

1. Única comanda de dibuix per a tots els objectes

4.1. EBO: Única comanda de dibuix

Una comanda de dibuix pròpia per a cada objecte → funció draw*()

PASSOS:

1. Modificar estructura CVAO que contingui totes les variables de VAO

```
// --------------- VGI: Estructura LLista VAO
struct CVAO
{           GLuint vaoId;           // Identificador del VAO
            GLuint vboId;           // Identificador del VBO
            GLuint eboId;           // Identificador del EBO
            GLint nVertexs;         // Nombre de vèrtexs del VAO
            GLint nIndices;         // Nombre d'índex de l'EBO
};
```

2. Crear vector de tipus CVAO (`VAOList[]`) on guardar paràmetres de cada objecte a visualitzar

```
//--------------- VGI: Màxim tamany vector VAOList
#define MAX_SIZE_VAOID 45

CVAO VAOList[MAX_SIZE_VAOID];
```

4.1. EBO: Única comanda de dibuix

3. Modificar les funcions per a inicialitzar vector (`initVAOList()`), configurar un element del vector (`SetVAOList()`), esborrar un element del vector (`deleteVAOList()`)

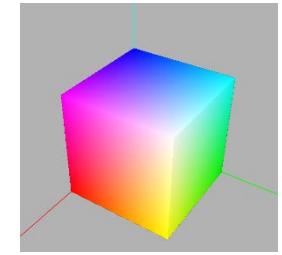
```
void initVAOList()
{  int i;
   for (i = 0; i < MAX_SIZE_VAOID; i++){
      VAOList[i].vaoId = 0;     VAOList[i].vboId = 0;     VAOList[i].eboId = 0;
      VAOList[i].nVertexs = 0;  VAOList[i].nIndices = 0;

      }
}
void Set_VAOList(GLint k, CVAO auxVAO)
{     VAOList[k].vaoId = auxVAO.vaoId;  VAOList[k].vboId = auxVAO.vboId;  VAOList[k].eboId = auxVAO.eboId;
      VAOList[k].nVertexs = auxVAO.nVertexs;  VAOList[k].nIndices = auxVAO.nIndices;
}
void deleteVAOList(GLint k)
{ if (VAOList[k].vaoId != 0) {
   glBindVertexArray(VAOList[k].vaoId); // Activació VAO
   glDisableVertexAttribArray(0); glDisableVertexAttribArray(1); glDisableVertexAttribArray(2);      glDisableVertexAttribArray(3);
   // It is good idea to release VBOs with ID 0 after use.
   glBindBuffer(GL_ARRAY_BUFFER, 0);
   glDeleteBuffers(1, &VAOList[k].vboId); // Esborrar buffer VBO
   // Delete EBO
   glDeleteBuffers(1, &VAOList[k].eboId);// glDeleteBuffers(1, &eboId);
   // Unbind and delete VAO
   glBindVertexArray(0);
   glDeleteVertexArrays(1, &VAOList[k].vaoId);

   // Eliminar valors de la posició VAOList[]
   VAOList[k].vboId = 0; VAOList[k].vaoId = 0; VAOList[k].eboId = 0; VAOList[k].nVertexs = 0; VAOList[k].nIndices = 0;     }
}
```

4.1. EBO: Única comanda de dibuix

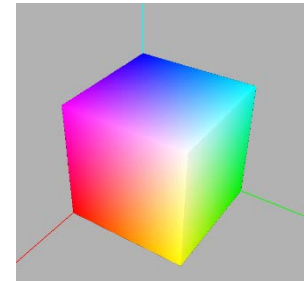
Exemple: Dibuix del cub segons el model de color RGB.



```
void glutSolidCubeRGB(GLdouble dsize)
{
  Set_VAOList(GLUT_CUBE_RGB, loadglutSolidCubeRGB_EBO(dsize));
  drawTriEBO_Object(GLUT_CUBE_RGB);
  deleteVAOList(GLUT_CUBE_RGB);
}
```

4.1. EBO: Única comanda de dibuix

Exemple: Dibuix del cub segons el model de color RGB.

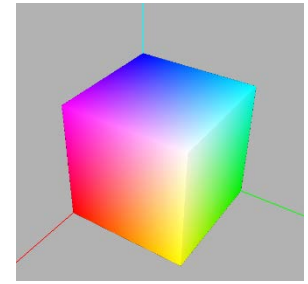


```
// Crear VAO i VBOs, transferir-hi geometria del cub (un sol cop)
CVAO loadglutSolidCubeRGB_EBO(GLdouble dsize) {
GLdouble size = dSize * 0.5f;
// ----------------------- VAO
CVAO auxVAO;   auxVAO.vaoId = 0;   auxVAO.vboId = 0;  auxVAO.eboId = 0;  auxVAO.nVertexs = 0;  auxVAO.nIndices = 0;
// Vertex Coord Array
std::vector<double> vertices = {size,size,size,-size,size,size,-size,-size,size, // v0-v1-v2 (front)
-size, -size,  size,  size, -size,  size,  size,  size,  size,      // v2-v3-v0
 size,  size,  size,  size, -size,  size,  size, -size, -size,      // v0-v3-v4 (right)
 size, -size, -size,  size,  size, -size,  size,  size,  size,      // v4-v5-v0
 size,  size,  size,  size,  size, -size, -size,  size, -size,      // v0-v5-v6 (top)
-size,  size, -size, -size,  size,  size,  size,  size,  size,      // v6-v1-v0
-size,  size,  size, -size,  size, -size, -size, -size, -size,      // v1-v6-v7 (left)
-size, -size, -size, -size, -size,  size, -size,  size,  size,      // v7-v2-v1
-size, -size, -size,  size, -size, -size,  size, -size,  size,      // v7-v4-v3 (bottom)
 size, -size,  size, -size, -size,  size, -size, -size, -size,      // v3-v2-v7
 size, -size, -size, -size, -size, -size, -size,  size, -size,      // v4-v7-v6 (back)
-size,  size, -size,  size,  size, -size,  size, -size, -size };    // v6-v5-v4

// Normal array
std::vector<double> normals = {
 0,  0,  1,  0,  0,  1,  0,  0,  1,   // v0-v1-v2 (front)
 0,  0,  1,  0,  0,  1,  0,  0,  1,   // v2-v3-v0
 1,  0,  0,  1,  0,  0,  1,  0,  0,   // v0-v3-v4 (right)
 1,  0,  0,  1,  0,  0,  1,  0,  0,   // v4-v5-v0
 0,  1,  0,  0,  1,  0,  0,  1,  0,   // v0-v5-v6 (top)
 0,  1,  0,  0,  1,  0,  0,  1,  0,   // v6-v1-v0
-1,  0,  0, -1,  0,  0, -1,  0,  0,   // v1-v6-v7 (left)
-1,  0,  0, -1,  0,  0, -1,  0,  0,   // v7-v2-v1
 0, -1,  0,  0, -1,  0,  0, -1,  0,   // v7-v4-v3 (bottom)
 0, -1,  0,  0, -1,  0,  0, -1,  0,   // v3-v2-v7
 0,  0, -1,  0,  0, -1,  0,  0, -1,   // v4-v7-v6 (back)
 0,  0, -1,  0,  0, -1,  0,  0, -1 }; // v6-v5-v4
[...]
```

4.1. EBO: Única comanda de dibuix

Exemple: Dibuix del cub segons el model de color RGB.



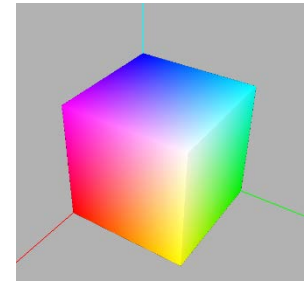
```
// Crear VAO i VBOs, transferir-hi geometria del cub (un sol cop)
CVAO loadglutSolidCubeRGB_EBO(GLdouble dsize) {

[...]
// Color array
std::vector<double> colors= { 1, 1, 1, 1, 0, 1, 1, 1, 0, 0, 1, 1,   // v0-v1-v2 (front)
0, 0, 1, 1, 1, 0, 1, 1, 1, 1, 1, 1,   // v2-v3-v0
1, 1, 1, 1, 1, 0, 1, 1, 1, 0, 0, 1,   // v0-v3-v4 (right)
1, 0, 0, 1, 1, 1, 0, 1, 1, 1, 1, 1,   // v4-v5-v0
1, 1, 1, 1, 1, 1, 0, 1, 0, 1, 0, 1,   // v0-v5-v6 (top)
0, 1, 0, 1, 0, 1, 1, 1, 1, 1, 1, 1,   // v6-v1-v0
0, 1, 1, 1, 0, 1, 0, 1, 0, 0, 0, 1,   // v1-v6-v7 (left)
0, 0, 0, 1, 0, 0, 1, 1, 0, 1, 1, 1,   // v7-v2-v1
0, 0, 0, 1, 1, 0, 0, 1, 1, 0, 1, 1,   // v7-v4-v3 (bottom)
1, 0, 1, 1, 0, 0, 1, 1, 0, 0, 0, 1,   // v3-v2-v7
1, 0, 0, 1, 0, 0, 0, 1, 0, 1, 0, 1,   // v4-v7-v6 (back)
0, 1, 0, 1, 1, 1, 0, 1, 1, 0, 0, 1 }; // v6-v5-v4

// Texture Array
std::vector<double> textures = {
1, 1, 1, 0, 0, 0,      // v0-v1-v2 (front)
0, 0, 0, 1, 1, 1,      // v2-v3-v0
1, 1, 0, 1, 0, 0,      // v0-v3-v4 (right)
0, 0, 1, 0, 1, 1,      // v4-v5-v0
0, 1, 0, 0, 1, 0,      // v0-v5-v6 (top)
1, 0, 1, 1, 0, 1,      // v6-v1-v0
0, 1, 0, 0, 1, 0,      // v1-v6-v7 (left)
1, 0, 1, 1, 0, 1,      // v7-v2-v1
0, 0, 1, 0, 1, 1,      // v7-v4-v3 (bottom)
1, 1, 0, 1, 0, 0,      // v3-v2-v7
0, 0, 1, 0, 1, 1,      // v4-v7-v6 (back)
1, 1, 0, 1, 0, 0 };    // v6-v5-v4
[...]
```

4.1. EBO: Única comanda de dibuix

Exemple: Dibuix del cub segons el model de color RGB.



```
// Crear VAO, transferir-hi geometria del cub (un sol cop)
CVAO loadlutSolidCubeRGB_EBO(GLdouble dsize) {

[...]
// Indices Array
std::vector<uint> indices = {
0, 1, 2, 2, 3, 0,            // v0-v1-v2-v3 (top)
4, 5, 6, 6, 7, 4,            // v0-v3-v4-v5 (front)
8, 9, 10, 10, 11, 8,         // v0-v5-v6-v1 (right)
12, 13, 14, 14, 15, 12,      // v1-v6-v7-v2 (back)
16, 17, 18, 18, 19, 16,      // v7-v4-v3-v2 (left)
20, 21, 22, 22, 23, 20 };    // v4-v7-v6-v5 (bottom)

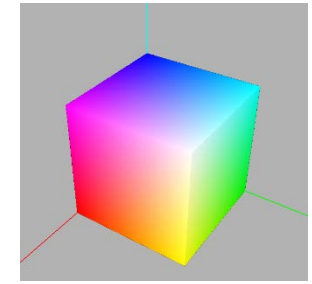
// Creació d'un VAO, VBO i EBO carregant hi de la geometria a la placa gràfica. Guardar identificadors VAO, VBO, EBO, vertex i indexs a
//  struct CVAO.
auxVAO = load_TRIANGLES_EBO(vertices, normals, colors, textures, indices);

return auxVAO;
}
```

`load_TRIANGLES_EBO()`: Funció que crea els buffers VAO, VBO i EBO, hi assigna la geometria continguda en els vectors `vertices`, `normals`, `colors`, `textures`, `indices` i la transfereix a la placa gràfica. Retorna els identificadors i el nombre de vèrtexs i índexs amb l'estructura CVAO.

4.1. EBO: Única comanda de dibuix

Exemple: Dibuix del cub segons el model de color RGB.



```
// ---------- TRIANGLES EBO
CVAO load_TRIANGLES_EBO(std::vector <double> vertices, std::vector <double> normals, std::vector <double> colors,
std::vector <double> textures) {

// ----------------------- VAO
CVAO tmpVAO;
tmpVAO.vaoId = 0;  tmpVAO.vboId = 0; tmpVAO.eboId = 0; tmpVAO.nVertexs = 0; tmpVAO.nIndices = 0;

std::vector <int>::size_type nv = vertices.size();// Tamany del vector vertices en elements.

// Create Vertex Array Object (VAO) for 3D Model Cube
glGenVertexArrays(1, &tmpVAO.vaoId);

// Create vertex buffer objects for 3D Model attributes in the VAO
glGenBuffers(1, &tmpVAO.vboId);

// Create vertex buffer objects for 3D Model EBO Elements attributes in the VAO
glGenBuffers(1, &tmpVAO.eboId);

// Bind our Vertex Array Object as the current used object
glBindVertexArray(tmpVAO.vaoId);

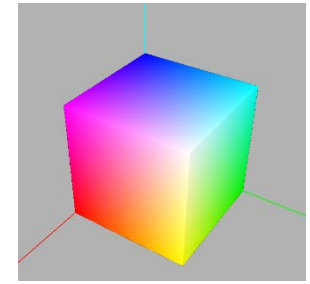
// Bind our Vertex Buffer Object as the current used object
glBindBuffer(GL_ARRAY_BUFFER, tmpVAO.vboId);

glBufferData(GL_ARRAY_BUFFER, (vertices.size() + normals.size() + textures.size() + colors.size()) * sizeof(double),
0, GL_STATIC_DRAW);

// Position Vertex attributes
glBufferSubData(GL_ARRAY_BUFFER, 0, vertices.size() * sizeof(double), &vertices[0]);// Copy geometry data to VBO
starting from 0 offset
glEnableVertexAttribArray(0);              // Enable attribute index 0 as being used (position)
glVertexAttribPointer(0, 3, GL_DOUBLE, GL_FALSE, 3 * sizeof(double), 0);              // Specify that our coordinate
data is going into attribute index 0 and contains 3 double
[...]
```

4.1. EBO: Única comanda de dibuix

Exemple: Dibuix del cub segons el model de color RGB.



```
// ---------- TRIANGLES EBO
CVAO load_TRIANGLES_EBO(std::vector <double> vertices, std::vector <double> normals, std::vector <double> colors,
std::vector <double> textures) {

[...]

// Normal Vertex Attributes
glBufferSubData(GL_ARRAY_BUFFER, vertices.size() * sizeof(double), normals.size() * sizeof(double), &normals[0]);// Copy normal data to VBO
starting from 0 offest
glEnableVertexAttribArray(1);// Enable attribute index 1 as being used (normals)
glVertexAttribPointer(1, 3, GL_DOUBLE, GL_FALSE, 3 * sizeof(double), (GLvoid*)(vertices.size() * sizeof(double)));// Specify that our color
data is going into attribute index 0 and contains 3 double

// Texture Coordinates Vertex Attributes
glBufferSubData(GL_ARRAY_BUFFER, (vertices.size() + normals.size()) * sizeof(double), textures.size() * sizeof(double), &textures[0]);//
Copy normal data to VBO starting from 0 offest
glEnableVertexAttribArray(2);              // Enable attribute index 2 as being used (texture coordinates)
glVertexAttribPointer(2, 2, GL_DOUBLE, GL_FALSE, 2 * sizeof(double), (GLvoid*)((vertices.size() + normals.size()) * sizeof(double)));//
Specify that our color data is going into attribute index 0 and contains 3 double

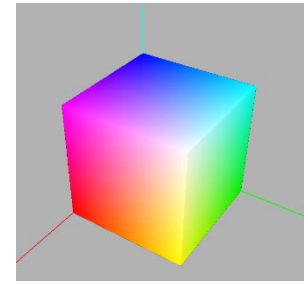
// Color Vertex Attributes
glBufferSubData(GL_ARRAY_BUFFER, (vertices.size() + normals.size() + textures.size()) * sizeof(double), colors.size() * sizeof(double),
&colors[0]);  // Copy normal data to VBO starting from 0 offset
glEnableVertexAttribArray(3);// Enable attribute index 3 as being used
glVertexAttribPointer(3, 4, GL_DOUBLE, GL_FALSE, 4 * sizeof(double), (GLvoid*)((vertices.size() + normals.size() + textures.size()) *
sizeof(double)));           // Specify that our color data is going into attribute index 0 and contains 3 double

// Bind our Element Buffer Object as the current used object
glBindBuffer(GL_ELEMENT_ARRAY_BUFFER, tmpVAO.eboId);
glBufferData(GL_ELEMENT_ARRAY_BUFFER, indices.size() * sizeof(GLuint), &indices[0], GL_STATIC_DRAW);

[...]
```

4.1. EBO: Única comanda de dibuix

Exemple: Dibuix del cub segons el model de color RGB.



```
// ---------- TRIANGLES EBO
CVAO load_TRIANGLES_EBO(std::vector <double> vertices, std::vector <double> normals, std::vector <double> colors,
std::vector <double> textures) {

[...]

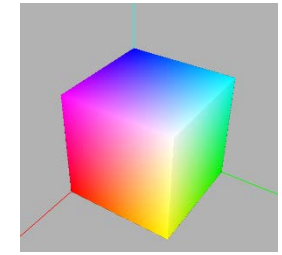
// Unbind VAO, to prevent bugs
glBindVertexArray(0);

tmpVAO.nVertexs = (int)vertices.size() / 3;
tmpVAO.nIndices = indices.size();

return tmpVAO; // retorna CVAO.
}
```

4.1. EBO: Única comanda de dibuix

Exemple: Dibuix del cub segons el model de color RGB.



```
void glutSolidCubeRGB(GLdouble dsize)
{
  Set_VAOList(GLUT_CUBE_RGB, loadglutSolidCubeRGB_VAO(dsize));
  draw_TriEBO_Object(GLUT_CUBE_RGB);
  deleteVAOList(GLUT_CUBE_RGB);
}


void draw_TriEBO_Object(GLint k) {
// Recuperar identificadors VAO i nvertexs dels vector VAOList pr a dibuixar VAO
if (VAOList[k].vaoId != 0) {
            glBindVertexArray(VAOList[k].vaoId);                    // Activar l'objecte VAO

            // Dibuixar el cub
            glBindBuffer(GL_ELEMENT_ARRAY_BUFFER, VAOList[k].eboId);
            glDrawElements(GL_TRIANGLES, VAOList[k].nIndices, GL_UNSIGNED_INT, nullptr);

            glBindBuffer(GL_ELEMENT_ARRAY_BUFFER, 0);               // Desactivar l'objecte EBO
            glBindVertexArray(0);                                   // Desactivar l'objecte VAO
            }
}
```

4.1. EBO: Única comanda de dibuix

Si tenim varis models 3D a visualitzar:

1. Generem-configurem els VAO's i els respectius VBO's i EBO, transferint-ho al hardware gràfic (funcions `load*_EBO()`)
2. Per a renderitzar una còpia (funció `drawTriEBO_Object*()`):
   a) *Bind* el corresponent VAO
   b) Renderitzem (`glDrawArrays()`)
   c) *Unbind* el corresponent VAO
3. Quan haguem acabat de renderitzar còpies: (funció `deleteVAOList()`):
   a) *Unbind* el corresponent VAO de la llista `VAOList[]`
   b) Esborrar el VAO (`glDeleteVertexArrays()`)

4.1. EBO: Única comanda de dibuix

Exemple: Dibuix d'una matriu de cubs.

```
SetVAOList(GLUT_CUBE_RGB, loadglutSolidCubeRGB_EBO(5.0));
SetVAOList(GLUT_TEAPOT, loadglutSolidTeapot_EBO());
ModelMatrix=matriuTG;
for (i = 0; i < 10; i++)
    for (j = 0; j < 10; j++)
        for (k = 0; k < 10; k++)
        { ModelMatrix = glm::translate(ModelMatrix, vec3(i * 15.0f, j * 15.0f, k * 15.0f));
          // Pas ModelViewMatrix a shader
          glUniformMatrix4fv(glGetUniformLocation(sh_programID,"modelMatrix"),1,GL_FALSE,&ModelMatrix[0][0]);
          // Pas NormalMatrix a shader
          NormalMatrix = transpose(inverse(MatriuVista * ModelMatrix));
          glUniformMatrix4fv(glGetUniformLocation(sh_programID,"normalMatrix"),1,GL_FALSE,&NormalMatrix[0][0]);
          drawTriEBO_Object(GLUT_CUBE_RGB);
        }

ModelMatrix=matriuTG;
for (i = 0; i < 10; i++)
    for (j = 0; j < 10; j++)
    {    ModelMatrix = glm::translate(ModelMatrix, vec3(i * 15.0f, j * 15.0f, 200.0));
         // Pas ModelViewMatrix a shader
         glUniformMatrix4fv(glGetUniformLocation(sh_programID,"modelMatrix"),1,GL_FALSE,&ModelMatrix[0][0]);
         // Pas NormalMatrix a shader
         NormalMatrix = transpose(inverse(MatriuVista * ModelMatrix));
         glUniformMatrix4fv(glGetUniformLocation(sh_programID,"normalMatrix"),1,GL_FALSE,&NormalMatrix[0][0]);
         drawTriEBO_Object(GLUT_TEAPOT);
    }
deleteVAOList(GLUT_CUBE_RGB);       // Eliminar Cub RGB
deleteVAOList(GLUT_TEAPOT);         // Eliminar Tetera
```

5. Pas de valors als shaders

Vectors dels buffers de VAO, VBO i EBO definits en l'aplicació es passen al *vertex shader* com variables d'entrada (`in`).

En aplicació (funció `dibuixa_EscenaGL()` en *escena.cpp*):

```
// Attribute Locations
glBindAttribLocation(sh_programID, 0, "in_Vertex");    // Vèrtexs
glBindAttribLocation(sh_programID, 1, "in_Normal");    // Normals
glBindAttribLocation(sh_programID, 2, "in_TexCoord"); // Textura
glBindAttribLocation(sh_programID, 3, "in_Color");     // Color
```

En shader (vertex program .vert):

```
// --- Variables in
layout (location = 0) in vec3 in_Vertex;    // Coordenades (x,y,z) posicio Vertex.
layout (location = 1) in vec3 in_Normal;    // Coordenades (x,y,z) Vector Normal.
layout (location = 2) in vec2 in_TexCoord; // Coordenades (s,t) Coordenada textura.
layout (location = 3) in vec4 in_Color;     // Coordenades (r,g,b,a) Color.
```

6. *Instancing*

També anomenat *Instanced rendering*

Dibuixar múltiples instàncies d'un VAO en una única crida `glDraw*()` i proveir d'atributs (matriu Transformacions Geomètriques - TG) únics a cada instància dins el Vertex Shader.

Exemples *instancing*:

- Asteroids al voltant d'un planeta
- Esquadra de naus espacials
- Exèrcit de soldats
- Ramat d'animals

6. *Instancing*

Passos:

1. Calcular matriu TG per a cada instància
2. Passar nombre d'instàncies i matrius TG al shader des de l'aplicació
3. Dibuixar les instàncies (`gl_DrawArrayInstanced()`)
4. Modificar el codi del Vertex Shader per a dibuixar les instàncies

6.1. Calcular TG per a cada instància

Definir les matrius TG úniques per a cada instància

Tres formes de fer-ho:

1. Definir-les dins el Vertex Shader
2. Definir-les dins l'aplicació i passar-les com variables `uniform`
3. Definir-les dins l'aplicació i passar-les com VBO dins el VAO

6.1.1. Dins Vertex Shader

Funció per a calcular les matrius TG (posició i orientació) per a cada instància i utilitzar com a índex la variable predefinida `gl_InstanceID` que identifica quina instància es processa en el VS.

```
mat4 transform = funcio_calcul_matriu(gl_InstanceID);
gl_Position = transform * in_Vertex;
```

6.1.2. Dins aplicació i passar-les uniform

Definir el vector de matrius TG, calcular-les en l'aplicació i passar el vector com uniform.

Aplicació:

```
glm::mat4 tr[100];
// Càlcul matrius
...
glUniformMatrix4fv(glGetUniformLocation(sh_programID,"trans"),100,GL_FALSE,&tr[0]);
```

VS:

```
uniform mat4 = trans[100];
gl_Position = trans[gl_InstanceID] * in_Vertex;
```

6.1.3. Dins aplicació i passar-les pel VAO

Millor opció.

Definir vector de matrius TG i calcular-les en l'aplicació i passar el vector com a VBO dins el VAO.

VAO pot ser sense EBO o amb EBO.

Dos casos:

a) **Instanciació per grid 3D.** Distribució matricial per a X, Y, Z. Esquadra naus.

b) **Instanciació asteroid.** Elements en posició i orientació aleatòria formant un anell de radi *R*.

6.1.3. Dins aplicació i passar-les pel VAO

a) Instanciació per grid 3D (1)

Passos:

1. Definir el vector TG

```
std::vector <glm::mat4> instanceMatrix;  // Definició del vector dinàmic de
                                         // matrius de transformació
instanceMatrix.clear();  // Reinicialitzar vector
```

6.1.3. Dins aplicació i passar-les pel VAO

a) Instanciació per grid 3D (2)

2. Definir matrius TG dins l'aplicació i passar-les com VBO dins el VAO

```
// Nombre d'elements del grix per a X, Y, Z
const GLint NINSTANCES_GRIDX = 10;
const GLint NINSTANCES_GRIDY = 10;
const GLint NINSTANCES_GRIDZ = 10;
// Separació entre elements del grix per a X, Y, Z
const GLint SEP_GRIDX = 15;
const GLint SEP_GRIDY = 15;
const GLint SEP_GRIDZ = 15;

glm::mat4 ModelMatrix(1.0);
for (int i = 0; i < NINSTANCES_GRIDX; i++)
  for (int j = 0; j < NINSTANCES_GRIDY; j++)
     for (int k = 0; k < NINSTANCES_GRIDZ; k++)
{     // Càlcul de la matriu de transformació dins el grid.
      ModelMatrix = glm::translate(MatriuTG, vec3(i * SEP_GRIDX, j * SEP_GRIDY, k * SEP_GRIDZ));
      instanceMatrix.push_back(ModelMatrix); // Guardar matriu a instanceMatrix.
}
...
```

6.1.3. Dins aplicació i passar-les pel VAO

a) Instanciació per grid 3D (3)

2. Definir matrius TG dins l'aplicació i passar-les com VBO dins el VAO

```
...
// Bind our Vertex Array Object as the current used object (o càrrega VAO amb 4 VBO's: vertex
(0),color (1),vector normal 2) i coordenades textura (3) )
glBindVertexArray(VAOList[indexVAO].vaoId); // indexVAO: enter que indexa un VAO actiu en VAOList

// Create vertex buffer objects for 3D Model attributes in the VAO
glGenBuffers(1, &Mat_InstId);

// Bind our Vertex Buffer Object as the current used object
glBindBuffer(GL_ARRAY_BUFFER, Mat_InstId);
glBufferData(GL_ARRAY_BUFFER, instanceMatrix.size() * sizeof(glm::mat4),
glm::value_ptr(instanceMatrix[0]), GL_STATIC_DRAW);

// First column instanceMatrix attributes
glEnableVertexAttribArray(4);    // Enable attribute index 4 as being used (first column glm::mat4)
// Specify that our column data is going into attribute index 0 and contains 4 float vec4
glVertexAttribPointer(4, 4, GL_FLOAT, GL_FALSE, 4 * sizeof(glm::vec4), (GLvoid*)0);
...
```

6.1.3. Dins aplicació i passar-les pel VAO

a) Instanciació per grid 3D (4)

2. Definir matrius TG dins l'aplicació i passar-les com VBO dins el VAO

```
...
// Second column instanceMatrix attributes
glEnableVertexAttribArray(5); // Enable attribute index 5 as being used (second column glm::mat4)
// Specify that our column data is going into attribute index sizeof(glm::vec4) and contains 4 float glm::vec4
glVertexAttribPointer(5, 4, GL_FLOAT, GL_FALSE, 4 * sizeof(glm::vec4), (void*)(sizeof(glm::vec4)));

// Third column instanceMatrix attributes
glEnableVertexAttribArray(6); // Enable attribute index 6 as being used (second column glm::mat4)

glVertexAttribPointer(6, 4, GL_FLOAT, GL_FALSE, 4 * sizeof(glm::vec4), (void*)(2*sizeof(glm::vec4)));
// Specify that our column data is going into attribute index 2*sizeof(glm::vec4) and contains 4 float glm::vec4
// Fourth column instanceMatrix attributes
glEnableVertexAttribArray(7); // Enable attribute index 7 as being used (second column glm::mat4)
// Specify that our column data is going into attribute index 3*sizeof(glm::vec4) and contains 4 float glm::vec4
glVertexAttribPointer(7, 4, GL_FLOAT, GL_FALSE, 4 * sizeof(glm::vec4), (void*)(3*sizeof(glm::vec4)));
...
```

6.1.3. Dins aplicació i passar-les pel VAO

a) Instanciació per grid 3D (5)

2. Definir matrius TG dins l'aplicació i passar-les com VBO dins el VAO

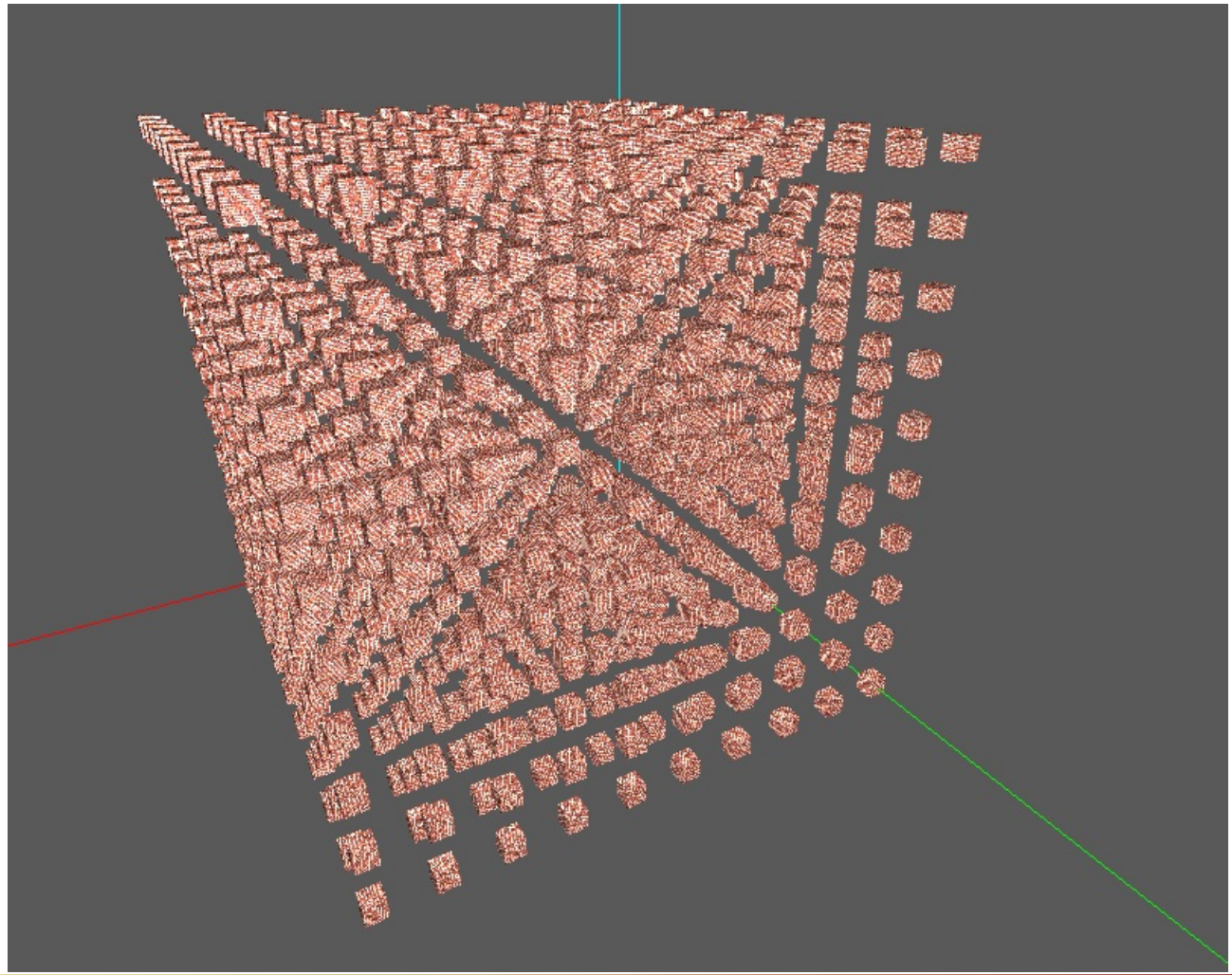
```
...
// Assignació de les 4 columnes a la matriu a cada instància.
glVertexAttribDivisor(4, 1);
glVertexAttribDivisor(5, 1);
glVertexAttribDivisor(6, 1);
glVertexAttribDivisor(7, 1);

// Unbind the registered VBO as current VBO
glBindBuffer(GL_ARRAY_BUFFER, 0);

// Unbind VAO, to prevent bugs
glBindVertexArray(0);
```

6.1.3. Dins aplicació i passar-les pel VAO

a) Instanciació per grid 3D (6)



6.1.3. Dins aplicació i passar-les pel VAO

b) Instanciació asteroid (1)

Passos:

1. Definir el vector TG

```
unsigned int amount = NUM_INSTANCIES; // Definició nombre d'instàncies
glm::mat4* modelMatrices;
modelMatrices = new glm::mat4[amount];// Definició vector de matrius glm::mat4
```

6.1.3. Dins aplicació i passar-les pel VAO

b) Instanciació asteroid (2)

2. Definir matrius TG dins l'aplicació i passar-les com VBO dins el VAO

```
srand(static_cast<unsigned int>(clock())); // Initialize random seed
float radius = 150.0;           // Radi localització asteroids
float offset = 25.0f;           // Separació entre elements
for (unsigned int i = 0; i < amount; i++)
{ glm::mat4 model = glm::mat4(1.0f);
  // 1. translation: displace along circle with 'radius' in range [-offset, offset]
  float angle = (float)i / (float)amount * 360.0f;
  float displacement = (rand() % (int)(2 * offset * 100)) / 100.0f - offset;

  float x = sin(angle) * radius + displacement;
  displacement = (rand() % (int)(2 * offset * 100)) / 100.0f - offset;

  float y = displacement * 0.4f; // keep height of asteroid field smaller compared to width of x and z
  displacement = (rand() % (int)(2 * offset * 100)) / 100.0f - offset;

  float z = cos(angle) * radius + displacement;
  model = glm::translate(model, glm::vec3(x, y, z));
...
```

6.1.3. Dins aplicació i passar-les pel VAO

b) Instanciació asteroid (3)

2. Definir matrius TG dins l'aplicació i passar-les com VBO dins el VAO

```
...
  // 2. scale: Scale between 0.05 and 0.25f
  float scale = static_cast<float>((rand() % 20) / 100.0 + 0.05);
  model = glm::scale(model, glm::vec3(scale));

  // 3. rotation: add random rotation around a (semi)randomly picked rotation axis vector
  float rotAngle = static_cast<float>((rand() % 360));
  model = glm::rotate(model, rotAngle, glm::vec3(0.4f, 0.6f, 0.8f));

  // 4. now add to list of matrices
  modelMatrices[i] = model;
}
```

6.1.3. Dins aplicació i passar-les pel VAO

b) Instanciació asteroid (4)

2. Definir matrius TG dins l'aplicació i passar-les com VBO dins el VAO

```
...
// Bind our Vertex Array Object as the current used object
//      (o càrrega VAO amb 4 VBO's: vertex (0),color (1),vector normal 2) i coordenades textura (3) )
glBindVertexArray(VAOList[indexVAO].vaoId); // indexVAO: enter que indexa un VAO actiu en VAOList

// Create vertex buffer objects for 3D Model attributes in the VAO
glGenBuffers(1, &Mat_InstId);

// Bind our Vertex Buffer Object as the current used object
glBindBuffer(GL_ARRAY_BUFFER, Mat_InstId);
glBufferData(GL_ARRAY_BUFFER, NUM_INSTANCIES* sizeof(glm::mat4), &modelMatrices[0], GL_STATIC_DRAW);

// First column instanceMatrix attributes
glEnableVertexAttribArray(4);    // Enable attribute index 4 as being used (first column glm::mat4)
// Specify that our column data is going into attribute index 0 and contains 4 float vec4
glVertexAttribPointer(4, 4, GL_FLOAT, GL_FALSE, 4 * sizeof(glm::vec4), (GLvoid*)0);
...
```

6.1.3. Dins aplicació i passar-les pel VAO

b) Instanciació asteroid (5)

2. Definir matrius TG dins l'aplicació i passar-les com VBO dins el VAO

```
...
// Second column instanceMatrix attributes
glEnableVertexAttribArray(5); // Enable attribute index 5 as being used (second column glm::mat4)
// Specify that our column data is going into attribute index sizeof(glm::vec4) and contains 4 float glm::vec4
glVertexAttribPointer(5, 4, GL_FLOAT, GL_FALSE, 4 * sizeof(glm::vec4), (void*)(sizeof(glm::vec4)));

// Third column instanceMatrix attributes
glEnableVertexAttribArray(6); // Enable attribute index 6 as being used (second column glm::mat4)
glVertexAttribPointer(6, 4, GL_FLOAT, GL_FALSE, 4 * sizeof(glm::vec4), (void*)(2*sizeof(glm::vec4)));
// Specify that our column data is going into attribute index 2*sizeof(glm::vec4) and contains 4 float glm::vec4

// Fourth column instanceMatrix attributes
glEnableVertexAttribArray(7); // Enable attribute index 7 as being used (second column glm::mat4)
// Specify that our column data is going into attribute index 3*sizeof(glm::vec4) and contains 4 float glm::vec4
glVertexAttribPointer(7, 4, GL_FLOAT, GL_FALSE, 4 * sizeof(glm::vec4), (void*)(3*sizeof(glm::vec4)));
...
```

6.1.3. Dins aplicació i passar-les pel VAO

b) Instanciació asteroid (6)

2. Definir matrius TG dins l'aplicació i passar-les com VBO dins el VAO

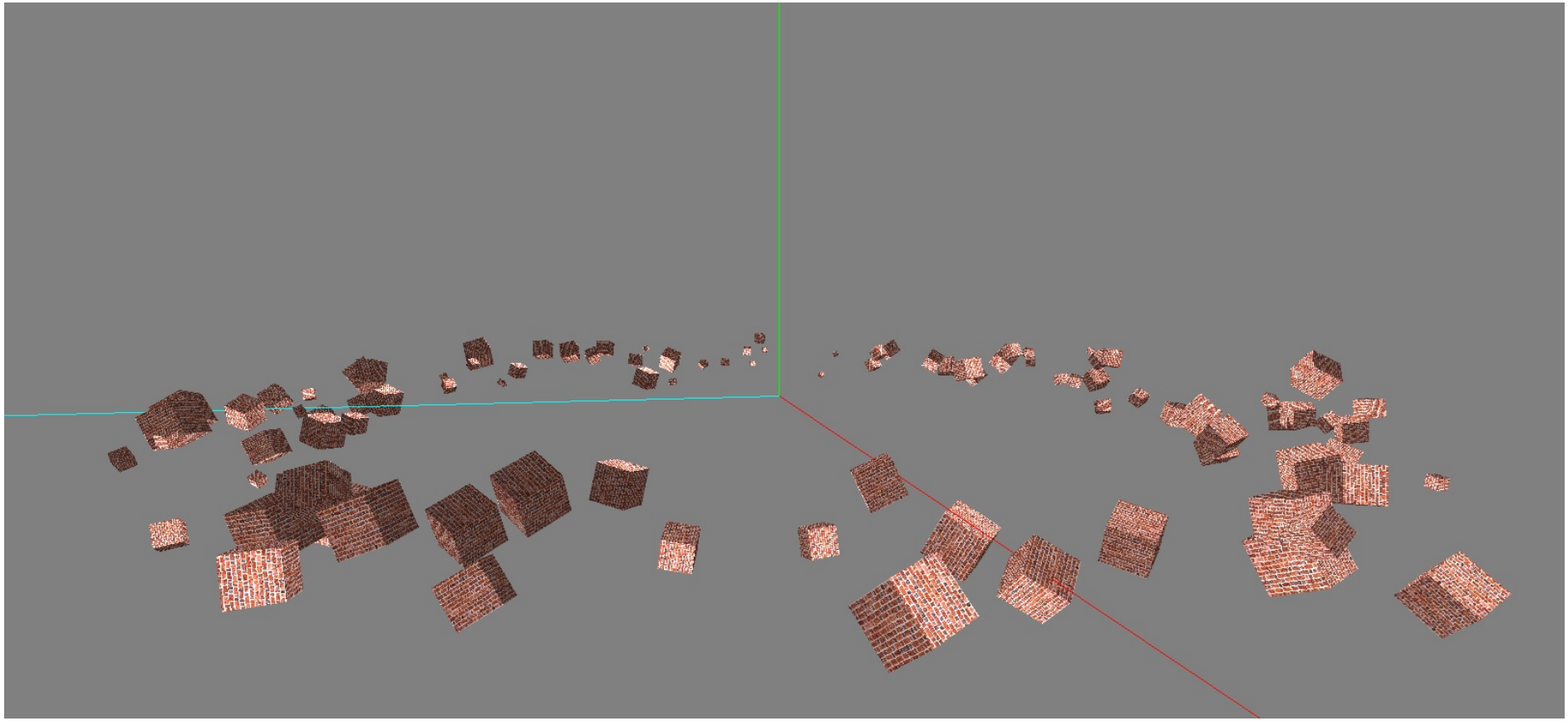
```
...
// Assignació de les 4 columnes a la matriu a cada instància.
glVertexAttribDivisor(4, 1);
glVertexAttribDivisor(5, 1);
glVertexAttribDivisor(6, 1);
glVertexAttribDivisor(7, 1);

// Unbind the registered VBO as current VBO
glBindBuffer(GL_ARRAY_BUFFER, 0);

// Unbind VAO, to prevent bugs
glBindVertexArray(0);
```

6.1.3. Dins aplicació i passar-les pel VAO

b) Instanciació asteroid (7)



6.2. Passar instàncies i TG des de l'aplicació

Definir un nou atribut pel VAO en l'aplicació

```
// Attribute Locations must be setup before calling glLinkProgram()
glBindAttribLocation(sh_programID, 0, "in_Vertex");     // Vèrtexs
glBindAttribLocation(sh_programID, 1, "in_Color");      // Color
glBindAttribLocation(sh_programID, 2, "in_Normal");     // Normals
glBindAttribLocation(sh_programID, 3, "in_TexCoord");   // Textura
glBindAttribLocation(sh_programID, 4, "instanceMatrix");// Instancing
```

6.3. Dibuixar les instàncies

Comanda `gl_DrawArrayInstanced()`

VAO sense EBO:

```
glDrawArraysInstanced(GL_TRIANGLES, 0, VAOList[k].nVertexs, NINSTANCIES);
```

```
void draw_TriVAO_Object(GLint k, bool sw_instanc)
{
// Recuperar identificadors VAO i nvertexs dels vector VAOList pr a dibuixar VAO
if (VAOList[k].vaoId != 0)
   {     glBindVertexArray(VAOList[k].vaoId);
         if (!sw_instanc) glDrawArrays(GL_TRIANGLES, 0, VAOList[k].nVertexs);
                  else glDrawArraysInstanced(GL_TRIANGLES, 0, VAOList[k].nVertexs,
                           VAOList[k].nInstances); // Instancing
         glBindVertexArray(0);
   }
}
```

6.3. Dibuixar les instàncies

Comanda `gl_DrawArrayInstanced()`

VAO amb EBO:

```
glDrawArraysInstanced(GL_TRIANGLES, VAOList[k].nIndices, GL_UNSIGNED_INT,
                                    nullptr, NINSTANCIES);

void draw_TriEBO_Object(GLint k, bool sw_instanc)
{
// Recuperar identificadors VAO i nvertexs dels vector VAOList pr a dibuixar VAO
if (VAOList[k].vaoId != 0)
 {  glBindVertexArray(VAOList[k].vaoId);
    if (!sw_instanc)
        glDrawElements(GL_TRIANGLES, VAOList[k].nIndices, GL_UNSIGNED_INT, nullptr);
       else glDrawArraysInstanced(GL_TRIANGLES, VAOList[k].nIndices, GL_UNSIGNED_INT,
                                    nullptr, VAOList[k].nInstances); // Instancing
       glBindVertexArray(0);
   }
}
```

6.4. Modificar Vertex Shader per a instàncies

Inclusió d'un nou atribut al VAO (`instanceMatrix`), diferent per a cada instància dins el Vertex Shader.

EN VS:

```
// --- L40- Variables in
layout (location = 0) in vec3 in_Vertex;     // Coordenades (x,y,z) posicio Vertex.
layout (location = 1) in vec4 in_Color;      // Coordenades (r,g,b,a) Color.
layout (location = 2) in vec3 in_Normal;     // Coordenades (x,y,z) Vector Normal.
layout (location = 3) in vec2 in_TexCoord;   // Coordenades (s,t) Coordenada textura.
layout (location = 4) in mat4 instanceMatrix; // Matriu TG de les instàncies.

// Definició vèrtex en coordenades càmera per a càlculs il.luminació
vec3 vertexPV = vec3(viewMatrix * instanceMatrix * modelMatrix *  vec4(in_Vertex,1.0));
vec3 V = normalize(-vertexPV);

// Transformacio de Visualitzacio amb Matriu Projeccio (projectionMatrix), Matriu
Càmera (viewMatrix), matriu instancing i Matriu TG (modelMatrix)
gl_Position = vec4(projectionMatrix * viewMatrix * instanceMatrix * modelMatrix *
vec4(in_Vertex,1.0));
```

6.4. Modificar Vertex Shader per a instàncies

En cas de que objecte a dibuixar estigui format per a varis VAO's, dues opcions a l'hora de multiplicar `instanceMatrix`) en VS:

- Abans `modelMatrix`

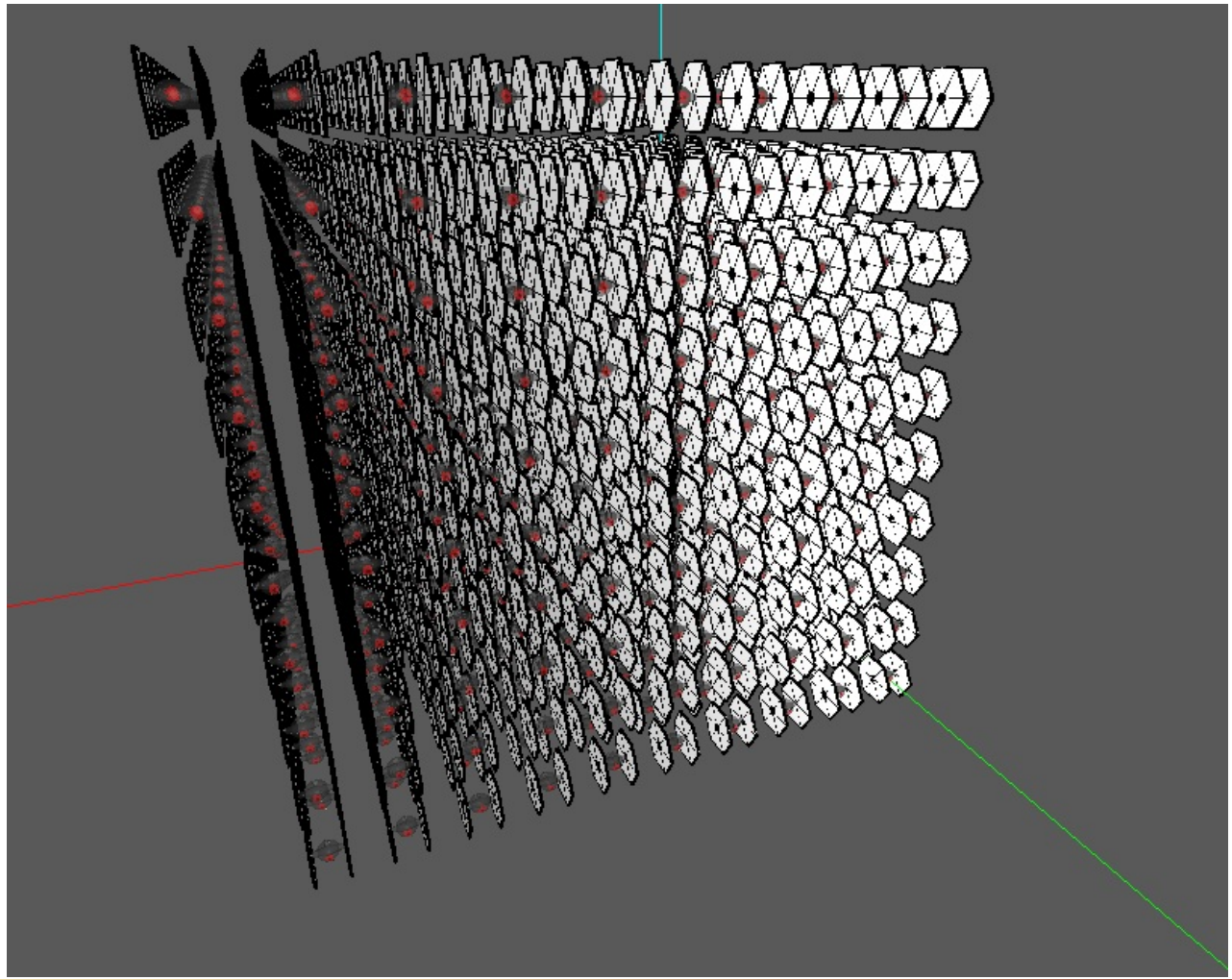
```
// Transformacio de Visualitzacio amb Matriu Projeccio (projectionMatrix), Matriu Càmera (viewMatrix), matriu instancing i Matriu TG (modelMatrix)
gl_Position = vec4(projectionMatrix * viewMatrix * instanceMatrix * modelMatrix * vec4(in_Vertex,1.0));
```

- Després `modelMatrix`

```
// Transformacio de Visualitzacio amb Matriu Projeccio (projectionMatrix), Matriu Càmera (viewMatrix), Matriu TG (modelMatrix) i matriu instancing
gl_Position = vec4(projectionMatrix * viewMatrix * modelMatrix * instanceMatrix * vec4(in_Vertex,1.0));
```

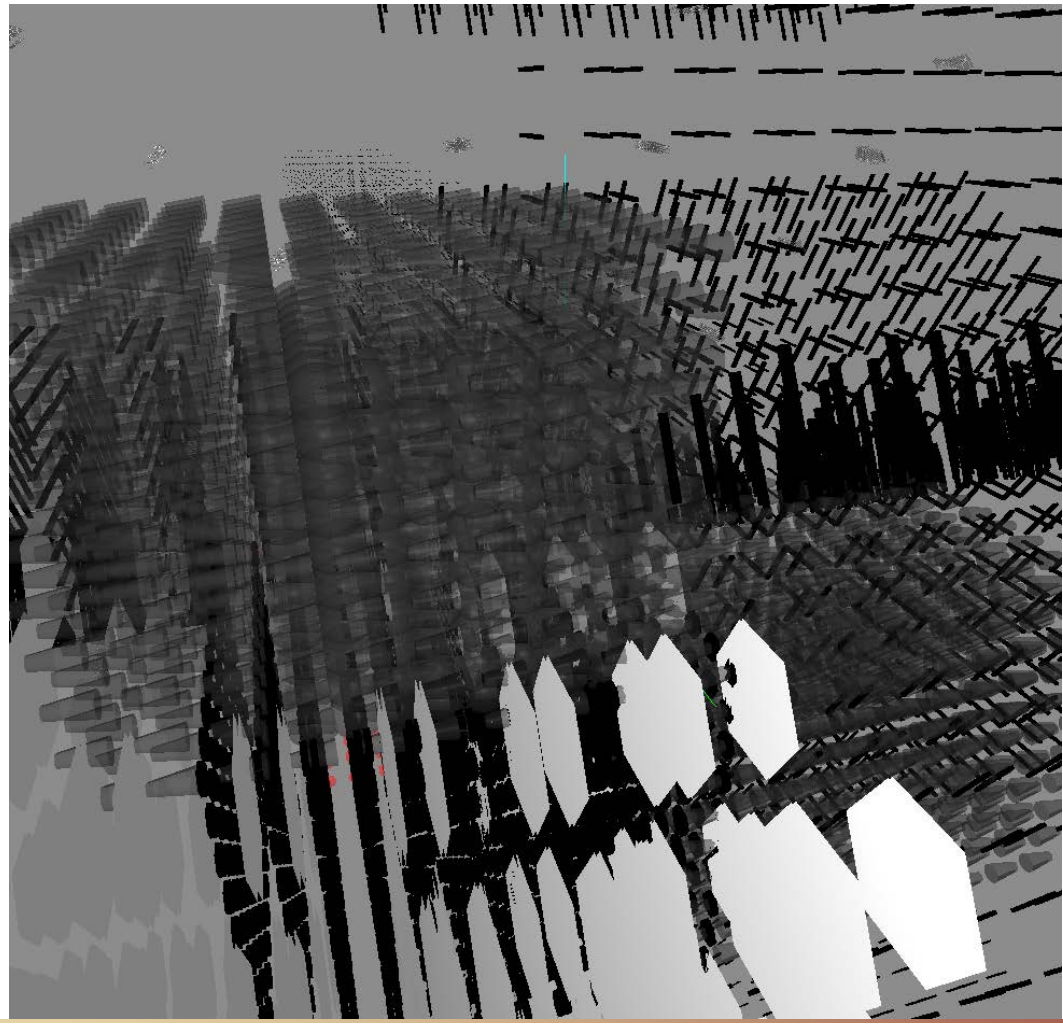
6.4. Modificar Vertex Shader per a instàncies

Abans `modelMatrix` (grid 3D)



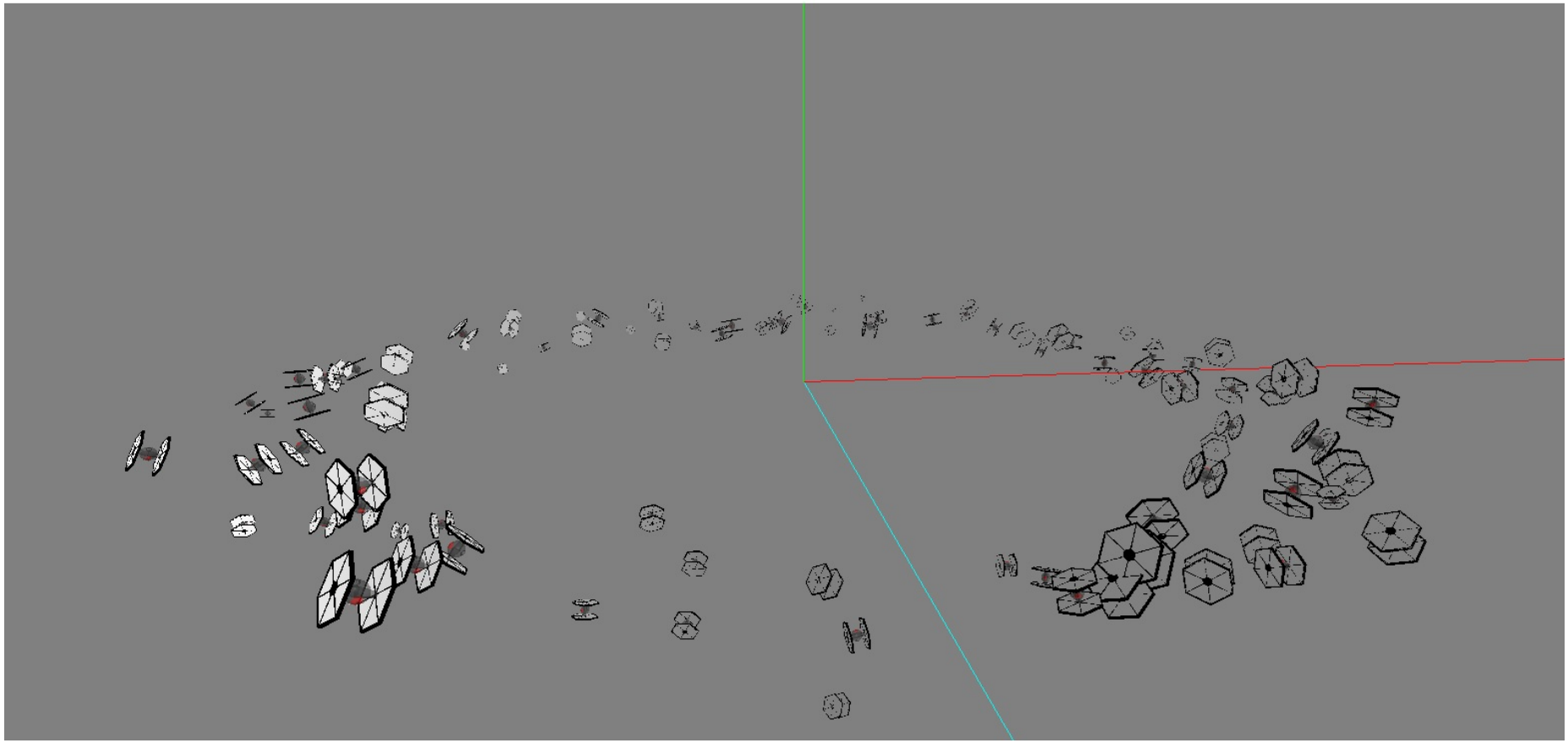
6.4. Modificar Vertex Shader per a instàncies

Després `modelMatrix` (grid 3D)



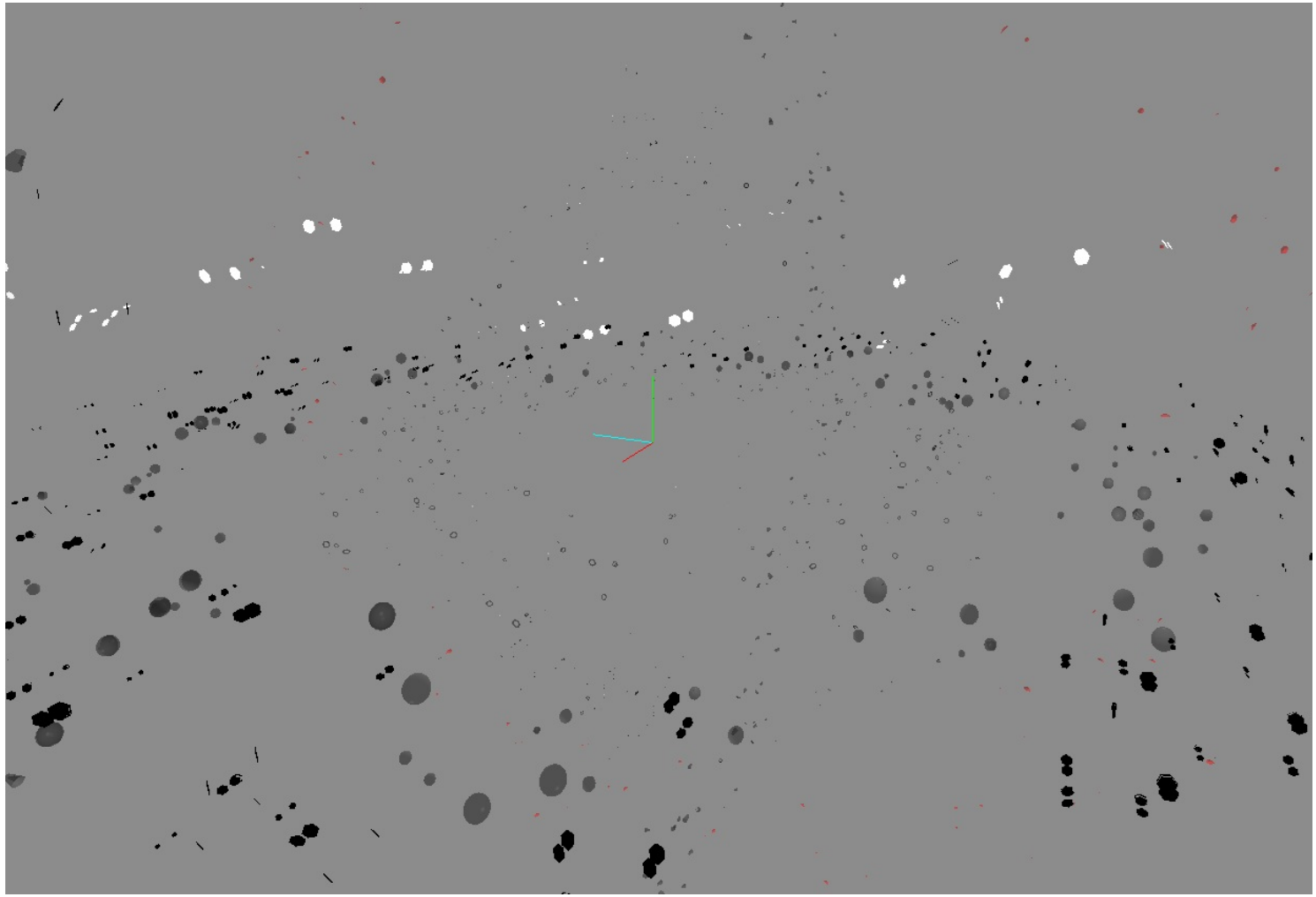
6.4. Modificar Vertex Shader per a instàncies

Abans `modelMatrix` (asteroid)



6.4. Modificar Vertex Shader per a instàncies

Després `modelMatrix` (asteroid)



7. Propietats de Dibuix

Propietats configurables dels VBO, VAO i EBO:

- `glPointSize()`: Tamany del punt en `size` píxels.
- `glLineWidth()`: Amplada de la línia en `size` píxels.
- `glEnable()`: Activació de l'aspecte de línia.
- `glLineStipple()`: Definició d'aspecte (interliniat) de línia.
- `glDisable()`: Desactivació de l'aspecte de línia.

7. Propietats de Dibuix

Propietats configurables dels VBO, VAO i EBO quan dibuixen triangles:

- `glFrontFace()`: Definició de les cares frontals segons el recorregut dels vèrtex (*clockwise* o *counterclockwise*).
- `glPoligonMode()`: Defineix quines cares (`GL_FRONT` i/o `GL_BACK`) es dibuixen i l'aspecte (punts, línies o sòlid).

7. Propietats de Dibuix

Propietats configurables dels VBO, VAO i EBO quan dibuixem triangles (2):

- `glEnable(GL_CULL_FACE)`: Activar text de visibilitat (*back-face culling*).
- `glCullFace()`: Defineix quines cares (`GL_FRONT` i/o `GL_BACK`) s'eliminen en el text de visibilitat.
- `glDisable(GL_CULL_FACE)`: Desactivar test de visibilitat (*back-face culling*).
- `glEnable(GL_DEPTH_TEST)`: Activar algorisme d'ocultacions Z-Buffer.
- `glDisable(GL_DEPTH_TEST)`: Desactivar algorisme d'ocultacions Z-Buffer.

8. Primitives Gràfiques

Primitives en codi font que hem adaptat a VBO, VAO i EBO en `glut_primitives.cpp`.

1. Primitives quàdriques GLU

2. Primitives GLUT

Cada primitiva té una funció de dibuix `glu*()` que carrega el VAO, dibuixa i elimina el VAO. La funció conté les funcions separades:

- `loadglu*_VAO()`, `loadglu*_EBO()` de creació i càrrega del VAO a placa
- `draw_TriVAO_Object()`, `draw_TriEBO_Object()` de dibuix de l'objecte
- `deleteVAOList()` d'eliminació del VAO, VBO i EBO

8.1. Primitives quàdriques GLU

Primitives basades en superfícies quàdriques definides per l'equació:

$$a_1x^2 + a_2y^2 + a_3z^3 + a_4xy + a_6xz + a_7x + a_8y + a_9z + a_{10} = 0$$

Primitives:

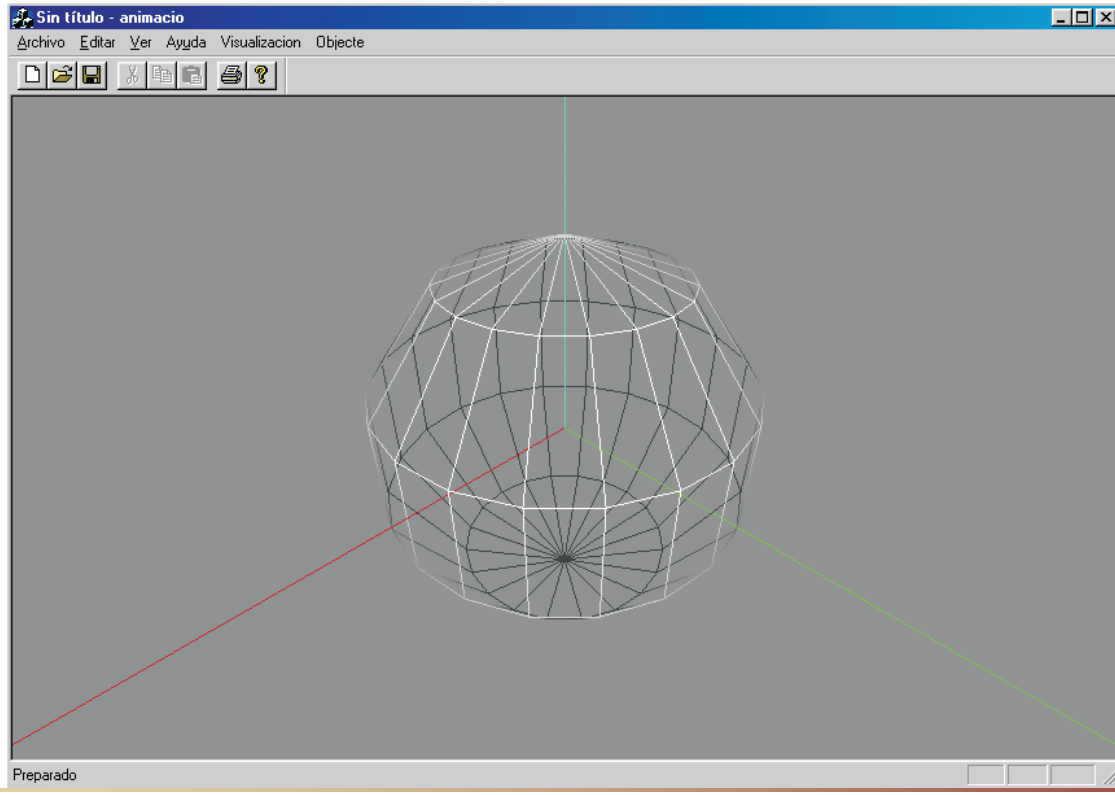
- Esfera (`gluSphere()`)
- Cilindre (`gluCylinder()`)
- Disc (`gluDisk()`)
- Arc de disc (`gluPartialDisk()`)

8.1. Primitives quàdriques GLU

```
void gluSphere(GLdouble radius, GLint slices, GLint stacks);
```

Acció: Dibuixa una esfera de radi `radius` centrada a l'origen `(0,0,0)` amb `slices` particions radials i `stacks` particions horitzontals.

Exemple: `gluSphere(1.0,20,5);`



8.1. Primitives quàdriques GLU

```
void gluSphere(GLdouble radius, GLint slices, GLint stacks);
```

Funció amb VAO sense EBO:

```
void gluSphere(GLdouble radius, GLint slices, GLint stacks) {
      Set_VAOList(GLU_SPHERE, loadgluSphere_VAO(radius, slices, stacks));
      draw_TriVAO_Object(GLU_SPHERE);
      deleteVAOList(GLU_SPHERE);
}
```

Funció amb VAO amb EBO:

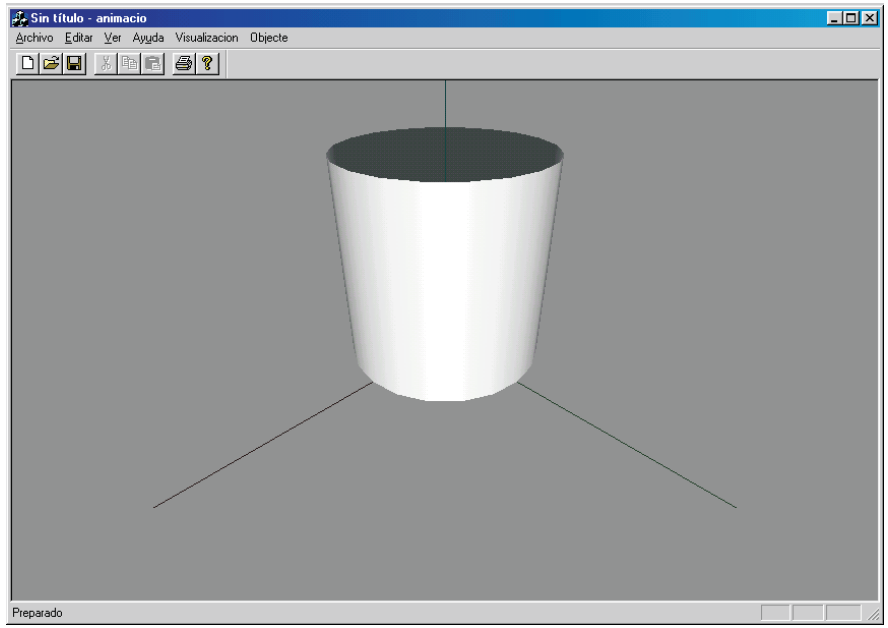
```
void gluSphere(GLdouble radius, GLint slices, GLint stacks) {
      Set_VAOList(GLU_SPHERE, loadgluSphere_EBO(radius, slices, stacks));
      draw_TriEBO_Object(GLU_SPHERE);
      deleteVAOList(GLU_SPHERE);
}
```

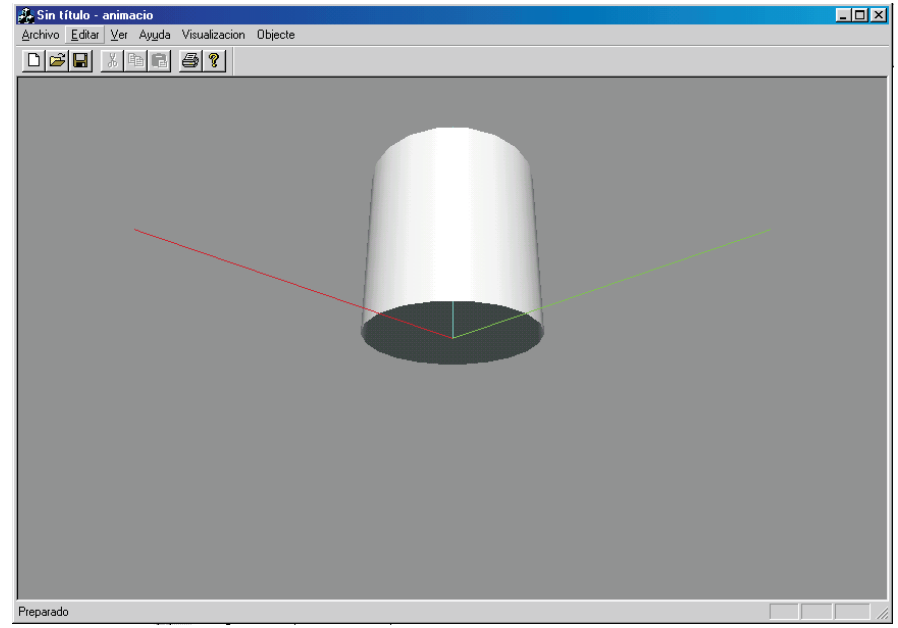
8.1. Primitives quàdriques GLU

```
void gluCylinder(GLdouble baseRadius, GLdouble topRadius, GLdouble height, GLint slices, GLint stacks);
```

Acció: Dibuixa un cilindre sense tapes sobre el pla z=0 amb radi inferior `baseRadius`, amb radi superior `topRadius` i alçada `heigh`, amb `slices` particions radials i `stacks` particions horitzontals.

Exemple: `gluCylinder(1.0,1.0,2.0,20,5);`





8.1. Primitives quàdriques GLU

```
void gluCylinder(GLdouble baseRadius, GLdouble topRadius, GLdouble
height, GLint slices, GLint stacks);
```

Funció amb VAO sense EBO:

```
void gluCylinder(GLdouble baseRadius, GLdouble topRadius, GLdouble height, GLint slices, GLint
stacks) {
     Set_VAOList(GLU_CYLINDER, loadgluCylinder_VAO(baseRadius, topRadius, height, slices, stacks));
     draw_TriVAO_Object(GLU_CYLINDER);
     deleteVAOList(GLU_CYLINDER);
}
```

Funció amb VAO i amb EBO:

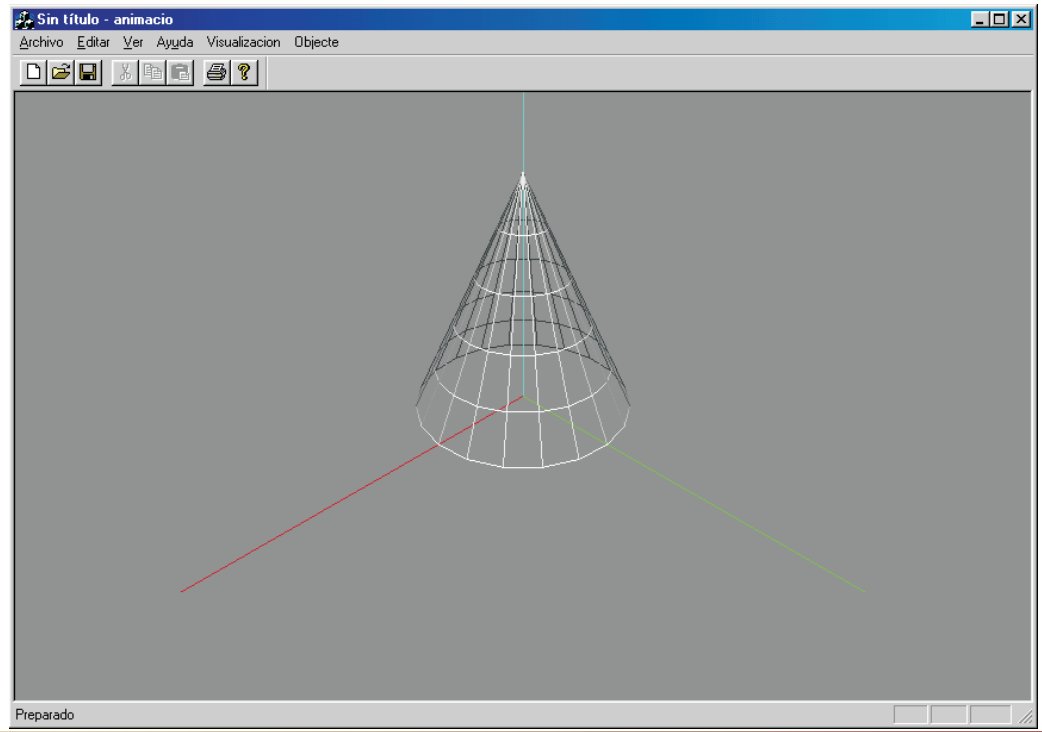
```
void gluCylinder(GLdouble baseRadius, GLdouble topRadius, GLdouble height, GLint slices, GLint
stacks) {
     Set_VAOList(GLU_CYLINDER, loadgluCylinder_EBO(baseRadius, topRadius, height, slices, stacks));
     draw_TriEBO_Object(GLU_CYLINDER);
     deleteVAOList(GLU_CYLINDER);
}
```

8.1. Primitives quàdriques GLU

```
void gluCylinder(GLdouble baseRadius, GLdouble topRadius, GLdouble
height, GLint slices, GLint stacks);
```

Funció molt versàtil que permet dibuixar diferents objectes geomètrics, com cub sense tapes o con.

Exemple con: `gluCylinder(1.0,0.0,2.0,20,5);`

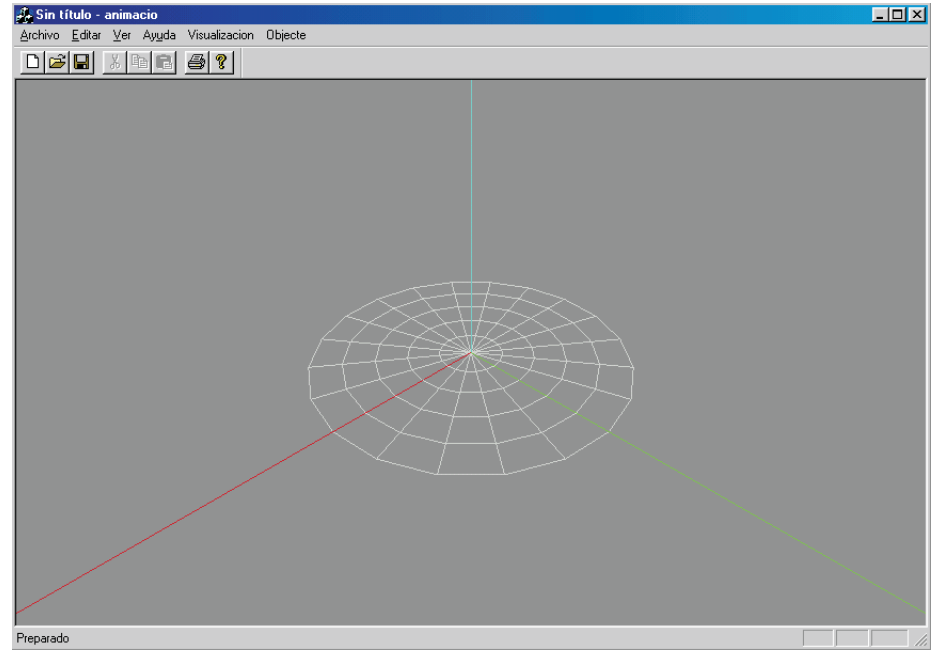


8.1. Primitives quàdriques GLU

```
void gluDisk(GLdouble innerRadius, GLdouble outerRadius, GLint slices, GLint rings);
```

Acció: Dibuixa un disc sobre el pla $z=0$ amb radi interior (forat) `innerRadius`, amb radi exterior `outerRadius`, amb `slices` particions radials i `rings` particions concèntriques.

Exemple: `gluDisk(0.0,1.0,20,5);`



8.1. Primitives quàdriques GLU

```
void gluDisk(GLdouble innerRadius, GLdouble outerRadius, GLint slices, GLint rings);
```

Funció amb VAO sense EBO:

```
void gluDisk(GLdouble innerRadius, GLdouble outerRadius, GLint slices, GLint loops) {
  Set_VAOList(GLU_DISK, loadgluDisk_VAO(innerRadius, outerRadius, slices, loops) );
  draw_TriVAO_Object(GLU_DISK);
  deleteVAOList(GLU_DISK);
}
```

Funció amb VAO amb EBO:

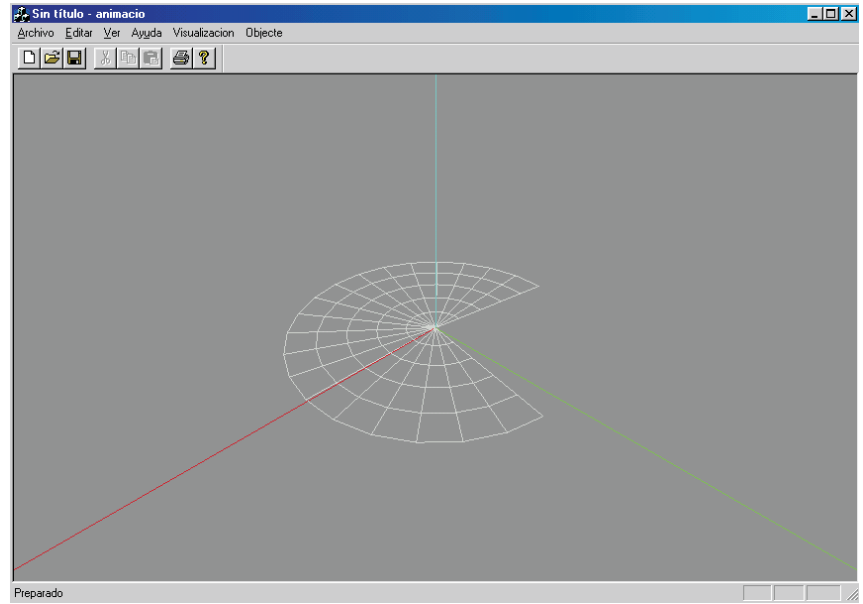
```
void gluDisk(GLdouble innerRadius, GLdouble outerRadius, GLint slices, GLint loops) {
  Set_VAOList(GLU_DISK, loadgluDisk_EBO(innerRadius, outerRadius, slices, loops) );
  draw_TriEBO_Object(GLU_DISK);
  deleteVAOList(GLU_DISK);
}
```

8.1. Primitives quàdriques GLU

```
void gluPartialDisk(GLdouble innerRadius, GLdouble outerRadius,
GLint slices, GLint rings, GLdouble startAngle, GLdouble sweep);
```

Acció: Dibuixa un disc sobre el pla $z=0$ amb radi interior (forat) `innerRadius`, amb radi exterior `outerRadius`, des de l'angle inicial `startAngle` amb `sweep` graus, amb `slices` particions radials i `rings` particions concèntriques.

Exemple: `gluPartialDisk(0.0,1.0,20,5,10.0,270.0);`



8.1. Primitives quàdriques GLU

```
void gluPartialDisk(GLdouble innerRadius, GLdouble outerRadius,
GLint slices, GLint rings, GLdouble startAngle, GLdouble sweep);
```

Funció amb VAO sense EBO:

```
void gluPartialDisk(GLdouble innerRadius, GLdouble outerRadius, GLint slices, GLint
rings, GLdouble startAngle, GLdouble sweep); {
     Set_VAOList(GLU_PARTIALDISK, loadgluPartialDisk_VAO(innerRadius, outerRadius,
                                                  slices, rings, startAngle, sweep));
     draw_TriVAO_Object(GLU_ PARTIALDISK);
     deleteVAOList(GLU_ PARTIALDISK);
}
```

Funció amb VAO amb EBO:

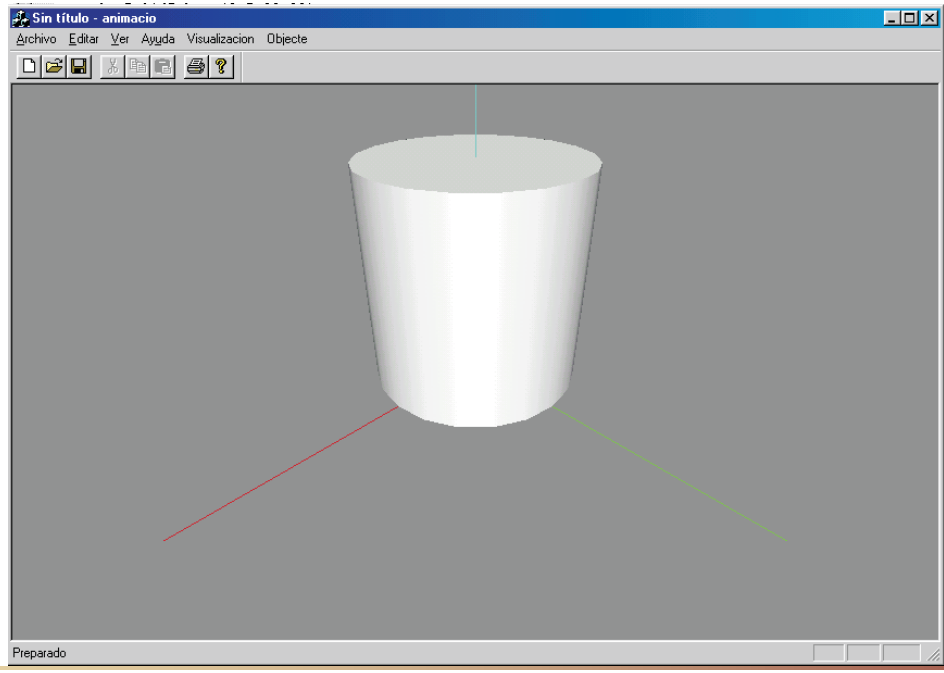
```
void gluPartialDisk(GLdouble innerRadius, GLdouble outerRadius, GLint slices, GLint
rings, GLdouble startAngle, GLdouble sweep);{
     Set_VAOList(GLU_PARTIALDISK, loadgluPartialDisk_EBO(innerRadius, outerRadius,
                                                  slices, rings, startAngle, sweep));
     draw_TriEBO_Object(GLU_PARTIALDISK);
     deleteVAOList(GLU_PARTIALDISK);
}
```

8.1. Primitives quàdriques GLU

```
void cilindre(GLdouble baseRadius, GLdouble topRadius, GLdouble height, GLint slices, GLint stacks)
```

Acció: Dibuixa un cilindre AMB TAPES sobre el pla z=0 amb radi inferior `baseRadius`, amb radi superior `topRadius` i alçada `heigh`, amb `slices` particions radials i `stacks` particions horitzontals.

Exemple: `cilindre(1.0,1.0,2.0,20,5);`



8.1. Primitives quàdriques GLU

```
void cilindre(GLdouble baseRadius, GLdouble topRadius, GLdouble
height, GLint slices, GLint stacks)
```

Funció amb VAO sense EBO:

```
void cilindre(GLdouble baseRadius, GLdouble topRadius, GLdouble height, GLint slices,
GLint stacks) {
     Set_VAOList(GLU_CILINDRE_SENCER, loadCilindre_VAO(baseradius, topRadius, height,
                                                       slices, stacks));
     draw_TriVAO_Object(GLU_CILINDRE_SENCER);
     deleteVAOList(GLU_CILINDRE_SENCER);
}
```

Funció amb VAO amb EBO:

```
void cilindre(GLdouble baseRadius, GLdouble topRadius, GLdouble height, GLint slices,
GLint stacks) {
     Set_VAOList(GLU_CILINDRE_SENCER, loadCilindre_EBO(baseradius, topRadius, height,
                                                       slices, stacks));
     draw_TriEBO_Object(GLU_CILINDRE_SENCER);
     deleteVAOList(GLU_CILINDRE_SENCER);
}
```

8.2. Primitives GLUT

Primitives gràfiques centrades a l'origen, amb definició explícita de coordenades textura i dibuixades amb triangles.

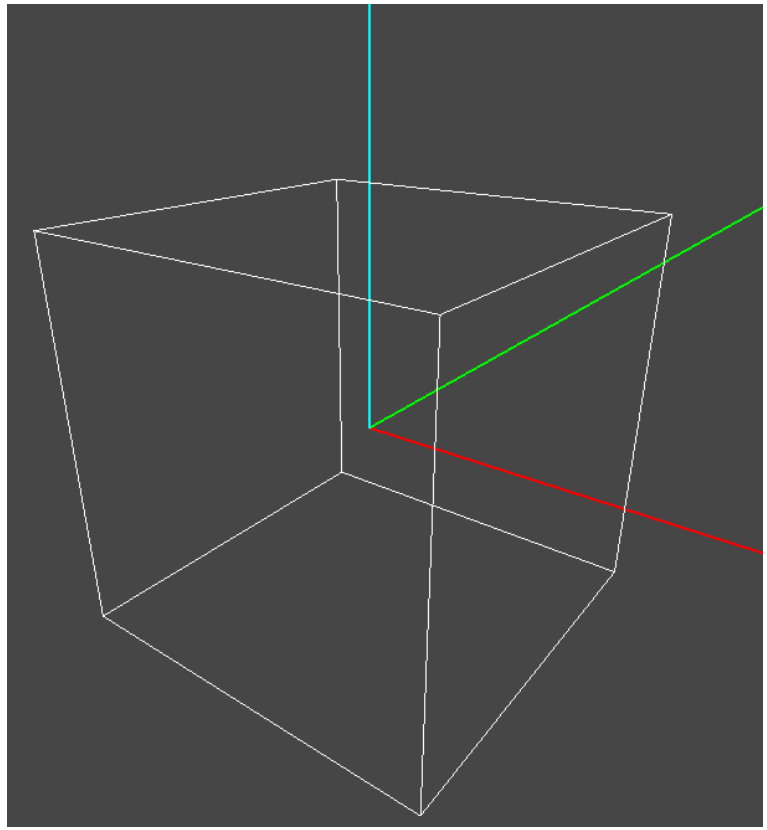
- Cub (`glutSolidCube( )`)
- Esfera (`glutSolidSphere()`)
- Torus (`glutSolidTorus()`)
- Icosaedre (`glutSolidIcosahedron()`)
- Octaedre (`glutSolidOctahedron()`)
- Tetraedre (`glutSolidTetraedron()`)
- Dodecaedre (`glutSolidDodecahedron()`)
- Conus (`glutSolidCone()`)
- Tetera (`glutSolidTeapot()`)

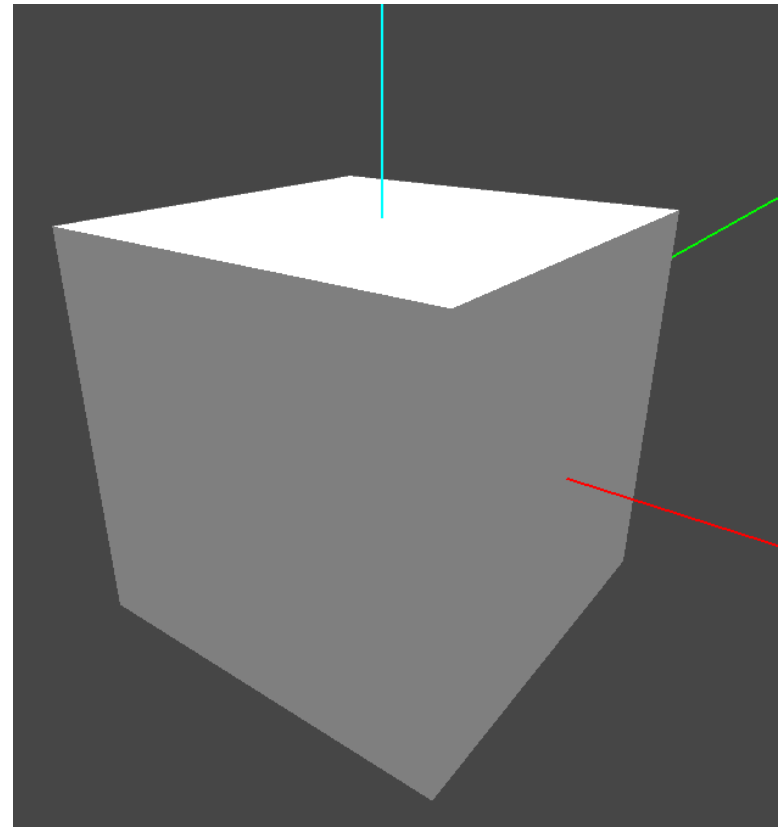
8.2. Primitives GLUT

```
void glutSolidCube(GLdouble size);
```

Acció: Dibuixa un cub de costat `size` centrat a l'origen `(0,0,0)`.

Exemple: `glutSolidCube(5.0);`





8.2. Primitives GLUT

```
void glutSolidCube(GLdouble size);
```

Funció amb VAO sense EBO:

```
void glutSolidCube(GLdouble size) {
      Set_VAOList(GLUT_CUBE, loadglutSolidCube_VAO(size));
      draw_TriVAO_Object(GLUT_CUBE);
      deleteVAOList(GLUT_CUBE);
}
```

Funció amb VAO amb EBO:

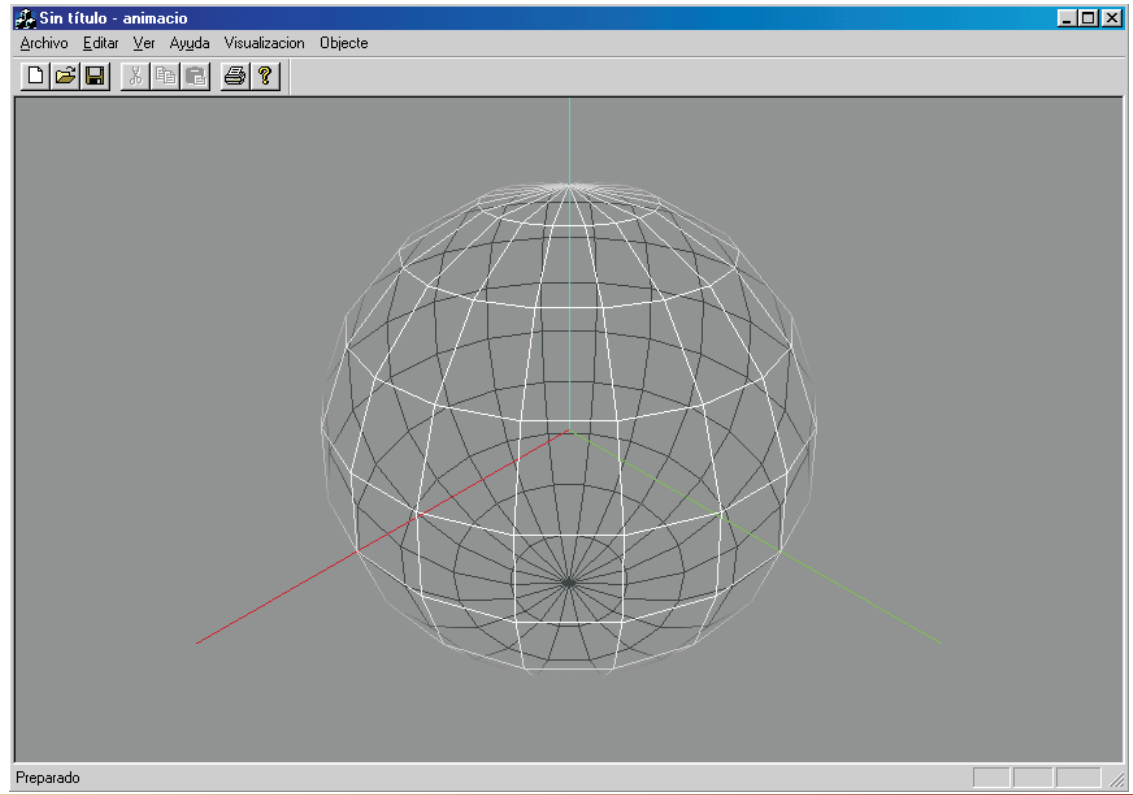
```
void glutSolidCube(GLdouble size) {
      Set_VAOList(GLUT_CUBE, loadglutSolidCube_EBO(size));
      draw_TriEBO_Object(GLUT_CUBE);
      deleteVAOList(GLUT_CUBE);
}
```

8.2. Primitives GLUT

```
void glutSolidSphere(GLdouble radius, GLint slices, GLint stacks);
```

Acció: Dibuixa una esfera de radi `radius` centrada a l'origen `(0,0,0)` amb `slices` particions radials i `stacks` particions horitzontals.

Exemple: `glutSolidSphere(1.5,20,10);`



8.2. Primitives GLUT

```
void glutSolidSphere(GLdouble radius, GLint slices, GLint stacks);
```

Funció amb VAO sense EBO:

```
void glutSolidSphere(GLdouble radius, GLint slices, GLint stacks) {
     Set_VAOList(GLUT_SPHERE, loadglutSolidSphere_VAO(radius, slices, stacks));
     draw_TriVAO_Object(GLUT_SPHERE);
     deleteVAOList(GLUT_SPHERE);
}
```

Funció amb VAO amb EBO:

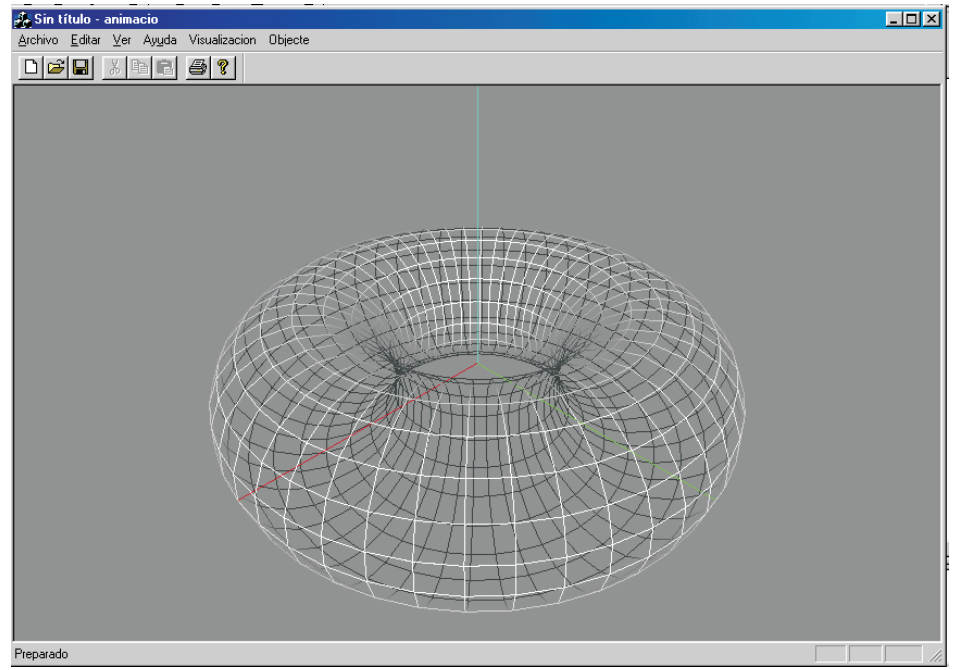
```
void glutSolidSphere(GLdouble radius, GLint slices, GLint stacks); {
     Set_VAOList(GLUT_SPHERE, loadglutSolidSphere_EBO(radius, slices, stacks));
     draw_TriEBO_Object(GLUT_SPHERE);
     deleteVAOList(GLUT_SPHERE);
}
```

8.2. Primitives GLUT

```
void glutSolidTorus(GLdouble innerRadius, GLdouble outerRadius,
GLint nsides, GLint rings);
```

Acció: Dibuixa una torus centrat a l'origen `(0,0,0)`, de radi interior `innerRadius`, radi exterior `outerRadius`, amb `nsides` particions horitzontals i `rings` anells.

Exemple: `glutSolidTorus(1.0,2.0,20,50);`



8.2. Primitives GLUT

```
void glutSolidTorus(GLdouble innerRadius, GLdouble outerRadius,
GLint nsides, GLint rings);
```

Funció amb VAO sense EBO:

```
void glutSolidTorus(GLdouble innerRadius, GLdouble outerRadius, GLint nsides, GLint
rings)
{     Set_VAOList(GLUT_TORUS, loadglutSolidTorus_VAO(innerRadius, outerRadius,
                                                     nsides, rings));
      draw_TriVAO_Object(GLUT_TORUS);
      deleteVAOList(GLUT_TORUS);
}
```

Funció amb VAO amb EBO:

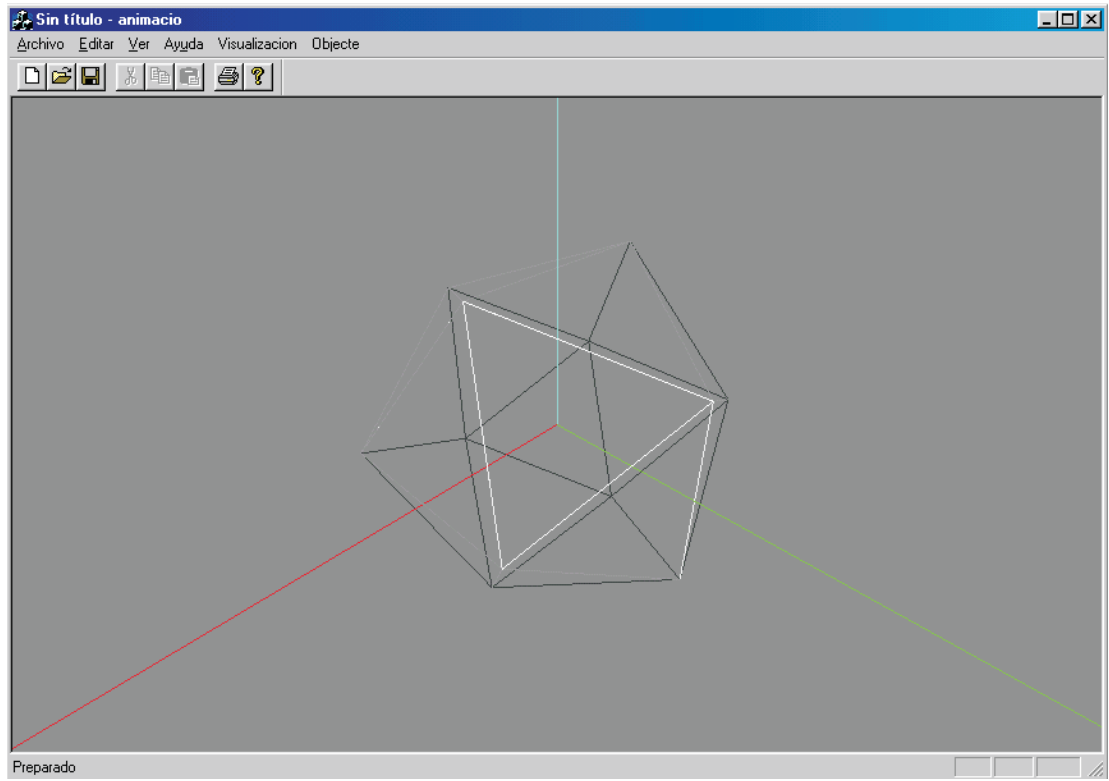
```
void glutSolidTorus(GLdouble innerRadius, GLdouble outerRadius, GLint nsides, GLint
rings)
{     Set_VAOList(GLUT_TORUS, loadglutSolidTorus_EBO(innerRadius, outerRadius,
                                                     nsides, rings));
      draw_TriEBO_Object(GLUT_TORUS);
      deleteVAOList(GLUT_TORUS);
}
```

8.2. Primitives GLUT

```
void glutSolidIcosahedron();
```

Acció: Dibuixa un icosaedre centrat a l'origen `(0,0,0)` i circumscrit en una esfera de radi 1.

Exemple: `glutSolidIcosahedron();`



8.2. Primitives GLUT

```
void glutSolidIcosahedron();
```

Funció amb VAO sense EBO:

```
void glutSolidIcosahedron() {
      Set_VAOList(GLUT_ICOSAHEDRON, loadglutSolidIcosahedron_VAO());
      draw_TriVAO_Object(GLUT_ICOSAHEDRON);
      deleteVAOList(GLUT_ICOSAHEDRON);
}
```

Funció amb VAO amb EBO:

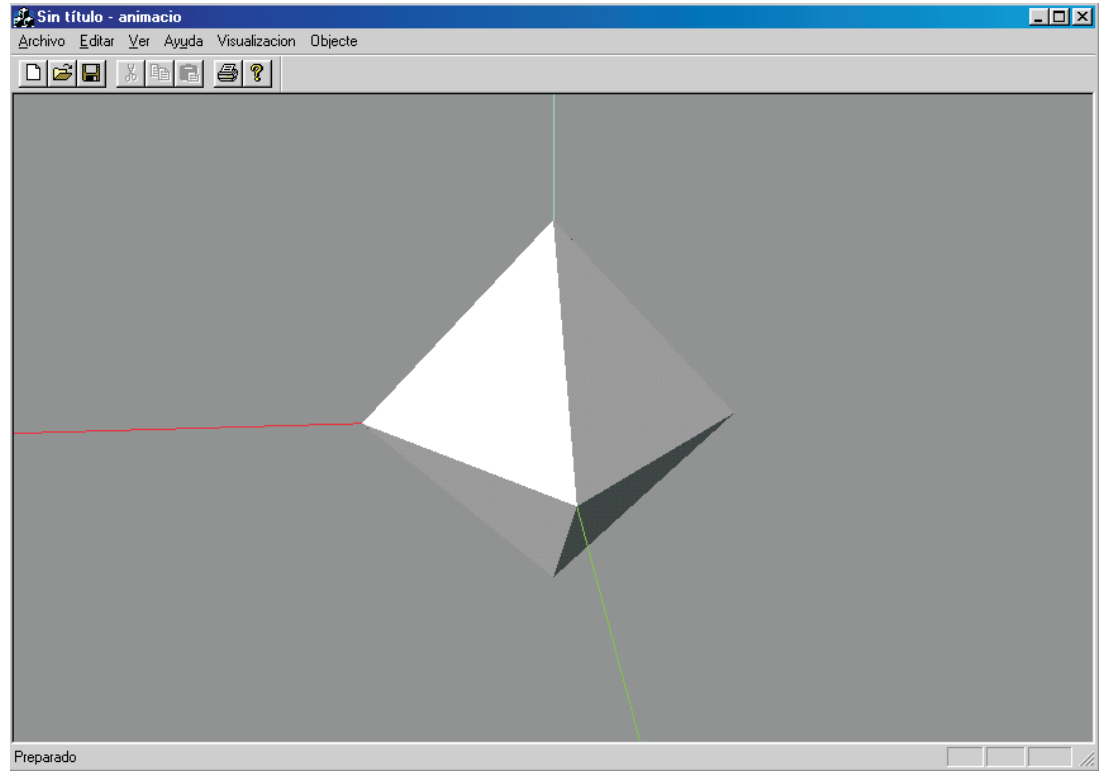
```
void glutSolidIcosahedron() {
      Set_VAOList(GLUT_ICOSAHEDRON, loadglutSolidIcosahedron_EBO());
      draw_TriEBO_Object(GLUT_ICOSAHEDRON);
      deleteVAOList(GLUT_ICOSAHEDRON);
}
```

8.2. Primitives GLUT

```
void glutSolidOctahedron();
```

Acció: Dibuixa un octaedre centrat a l'origen `(0,0,0)` i circumscrit en una esfera de radi 1.

Exemple: `glutSolidOctahedron();`



8.2. Primitives GLUT

```
void glutSolidOctahedron();
```

Funció amb VAO sense EBO:

```
void glutSolidOctahedron() {
      Set_VAOList(GLUT_OCTAHEDRON, loadglutSolidOctahedron_VAO());
      draw_TriVAO_Object(GLUT_OCTAHEDRON);
      deleteVAOList(GLUT_OCTAHEDRON);
}
```

Funció amb VAO amb EBO:

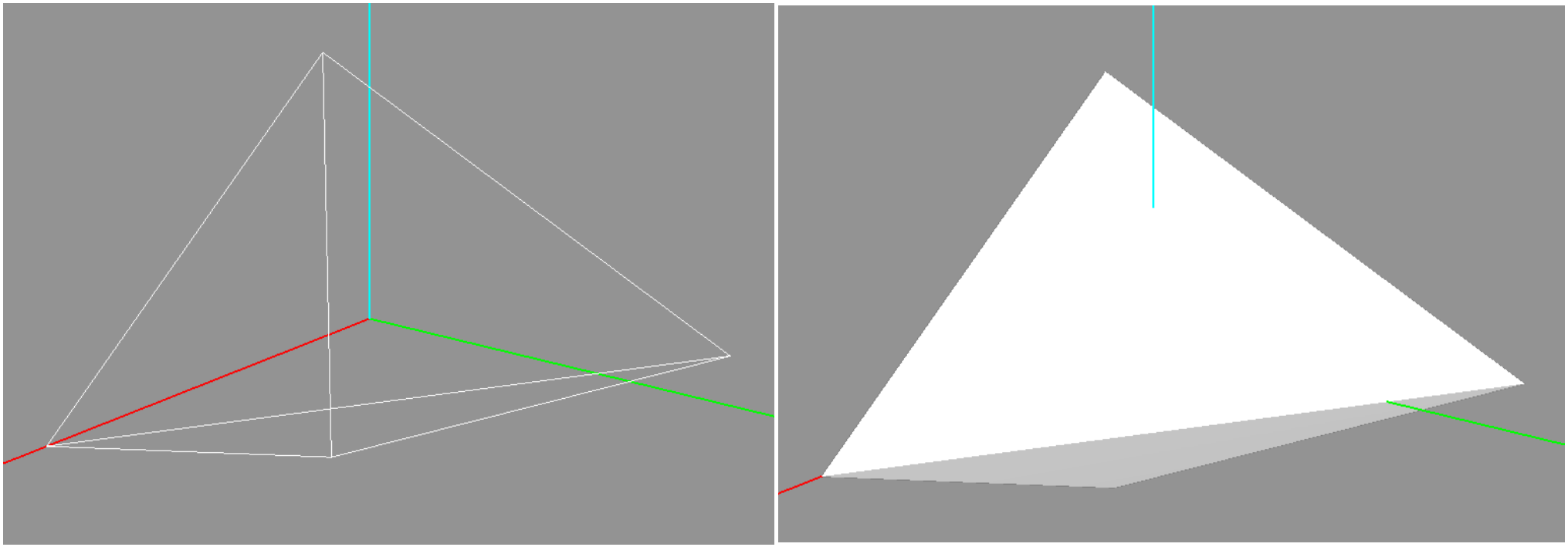
```
void glutSolidOctahedron() {
      Set_VAOList(GLUT_OCTAHEDRON, loadglutSolidOctahedron_EBO());
      draw_TriEBO_Object(GLUT_OCTAHEDRON);
      deleteVAOList(GLUT_OCTAHEDRON);
}
```

8.2. Primitives GLUT

```
void glutSolidTetrahedron();
```

Acció: Dibuixa un tetraedre centrat a l'origen `(0,0,0)` i circumscrit en una esfera de radi 1.

Exemple: `glutSolidTetrahedron();`



8.2. Primitives GLUT

```
void glutSolidTetrahedron();
```

Funció amb VAO sense EBO:

```
void glutSolidTetrahedron() {
      Set_VAOList(GLUT_TETRAHEDRON, loadglutSolidTetrahedron_VAO());
      draw_TriVAO_Object(GLUT_TETRAHEDRON);
      deleteVAOList(GLUT_TETRAHEDRON);
}
```

Funció amb VAO amb EBO:

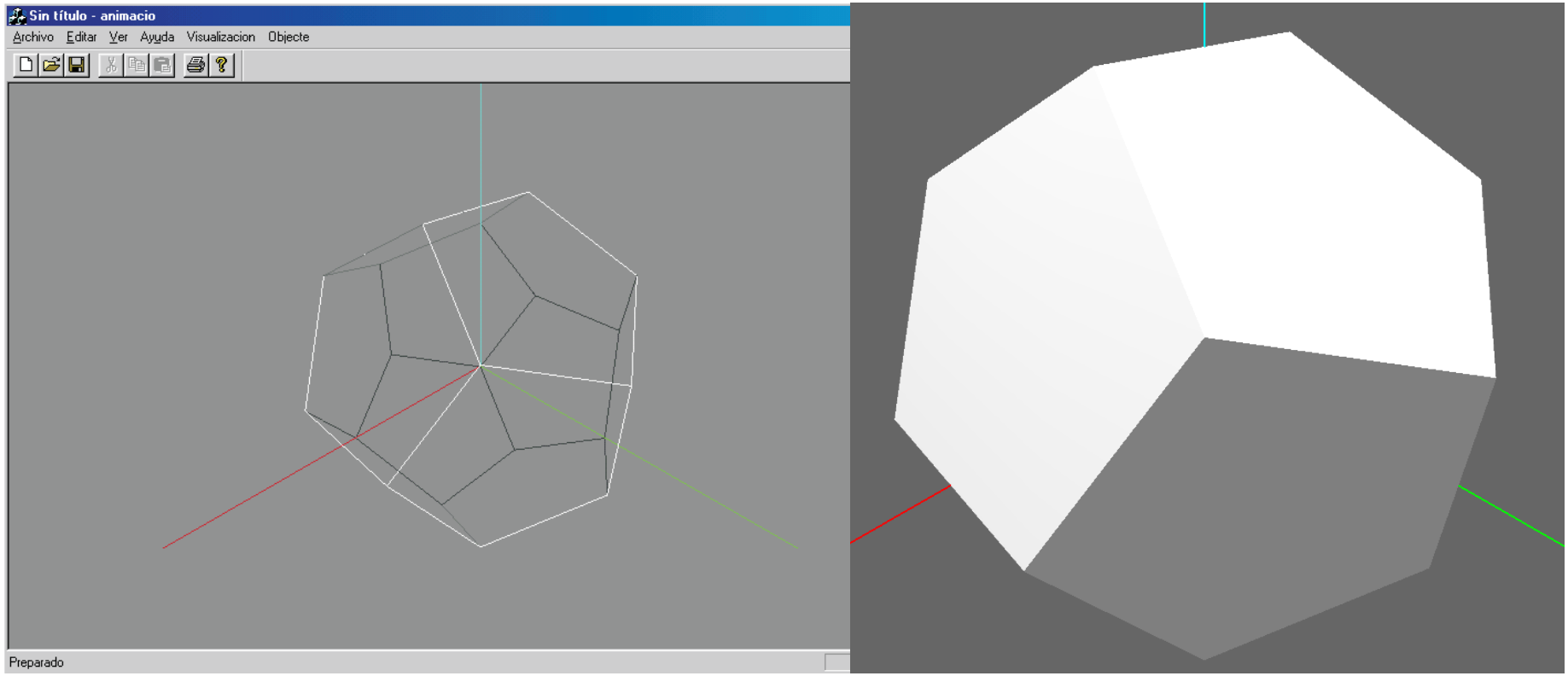
```
void glutSolidTetrahedron() {
      Set_VAOList(GLUT_TETRAHEDRON, loadglutSolidTetrahedron_EBO());
      draw_TriEBO_Object(GLUT_TETRAHEDRON);
      deleteVAOList(GLUT_TETRAHEDRON);
}
```

8.2. Primitives GLUT

```
void glutSolidDodecahedron();
```

Acció: Dibuixa un dodecaedre centrat a l'origen `(0,0,0)` i circumscrit en una esfera de radi 1.

Exemple: `glutSolidDodecahedron();`



8.2. Primitives GLUT

```
void glutSolidDodecahedron();
```

Funció amb VAO sense EBO:

```
void glutSolidDodecahedron() {
      Set_VAOList(GLUT_DODECAHEDRON, loadglutSolidDodecahedron_VAO());
      draw_TriVAO_Object(GLUT_DODECAHEDRON);
      deleteVAOList(GLUT_DODECAHEDRON);
}
```

Funció amb VAO amb EBO:

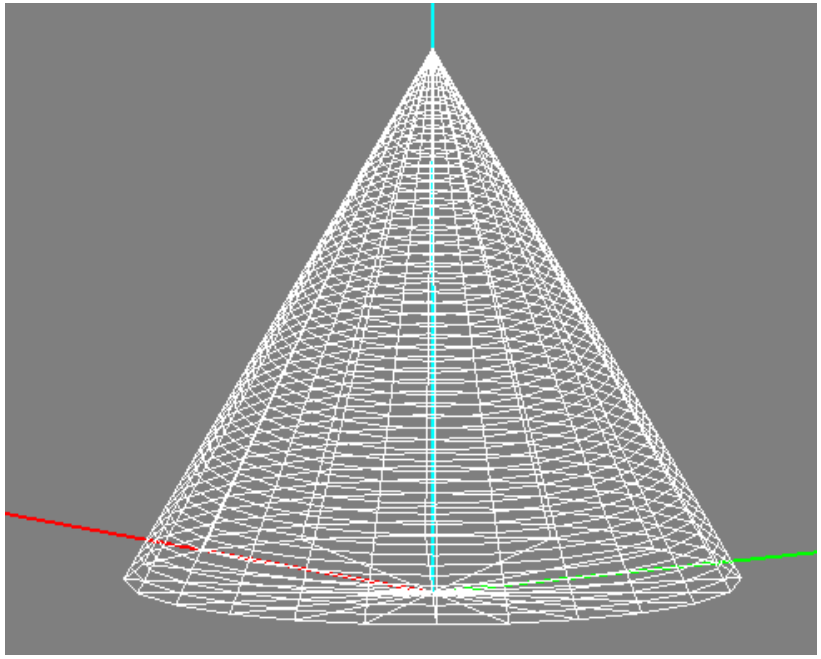
```
void glutSolidDodecahedron() {
      Set_VAOList(GLUT_DODECAHEDRON, loadglutSolidDodecahedron_EBO());
      draw_TriEBO_Object(GLUT_DODECAHEDRON);
      deleteVAOList(GLUT_DODECAHEDRON);
}
```

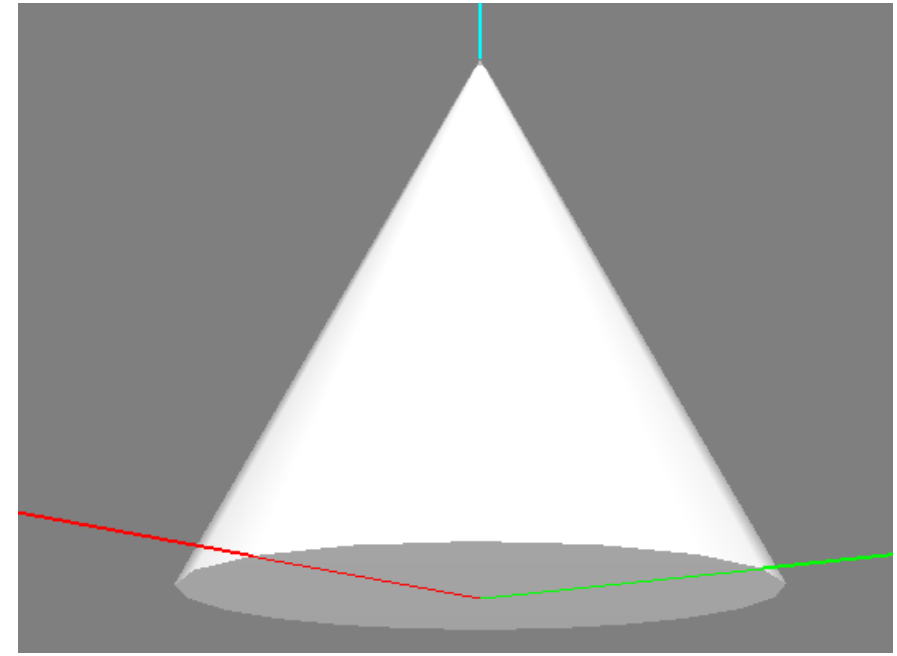
8.2. Primitives GLUT

```
void glutSolidCone(GLdouble base, GLdouble height, GLint slices,
GLint stacks);
```

Acció: Dibuixa un con centrat a l'origen `(0,0,0)`, de radi inferior `radius`, alçada `height`, mb `slices` particions radials i `stacks` particions horitzontals.

Exemple: `glutSolidCone(1.0,2.0,20,5);`





8.2. Primitives GLUT

```
void glutSolidCone(GLdouble base, GLdouble height, GLint slices,
GLint stacks);
```

Funció amb VAO sense EBO:

```
void glutSolidCone(GLdouble base, Gldouble height, GLint slices, GLint stacks) {
     Set_VAOList(GLUT_CONE, loadglutSolidCone_VAO(base, height, slices, stacks));
     draw_TriVAO_Object(GLUT_CONE);
     deleteVAOList(GLUT_CONE);
}
```

Funció amb VAO amb EBO:

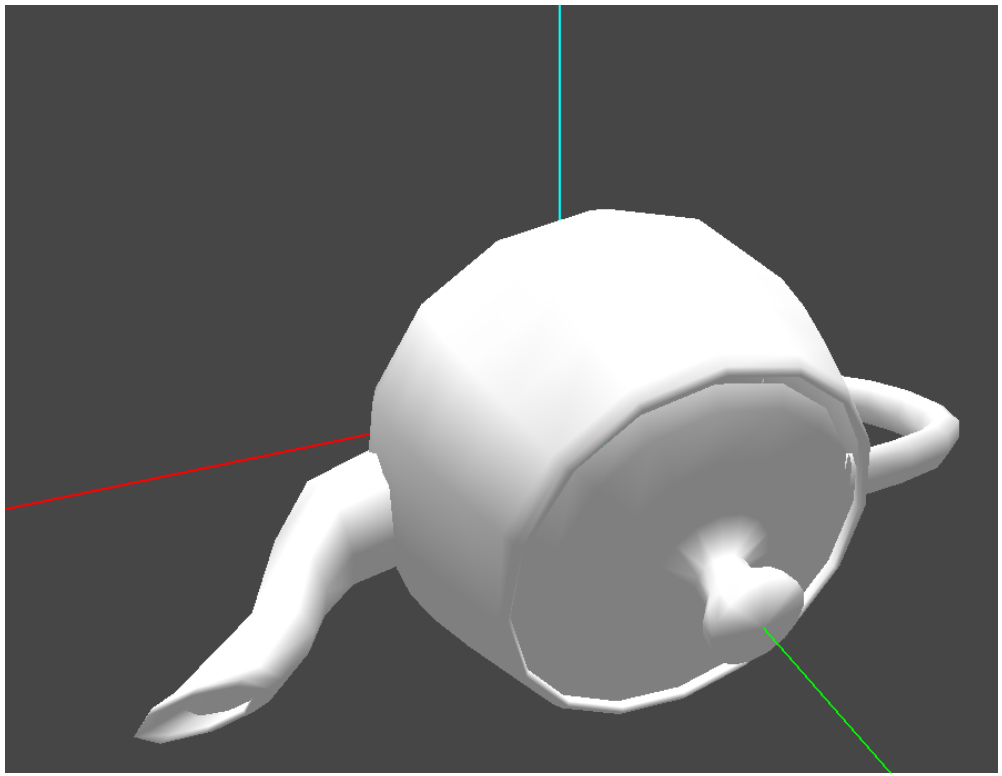
```
void glutSolidCone(GLdouble base, Gldouble height, GLint slices, GLint stacks) {
     Set_VAOList(GLUT_CONE, loadglutSolidCone_EBO(base, height, slices, stacks));
     draw_TriEBO_Object(GLUT_CONE);
     deleteVAOList(GLUT_CONE);
}
```

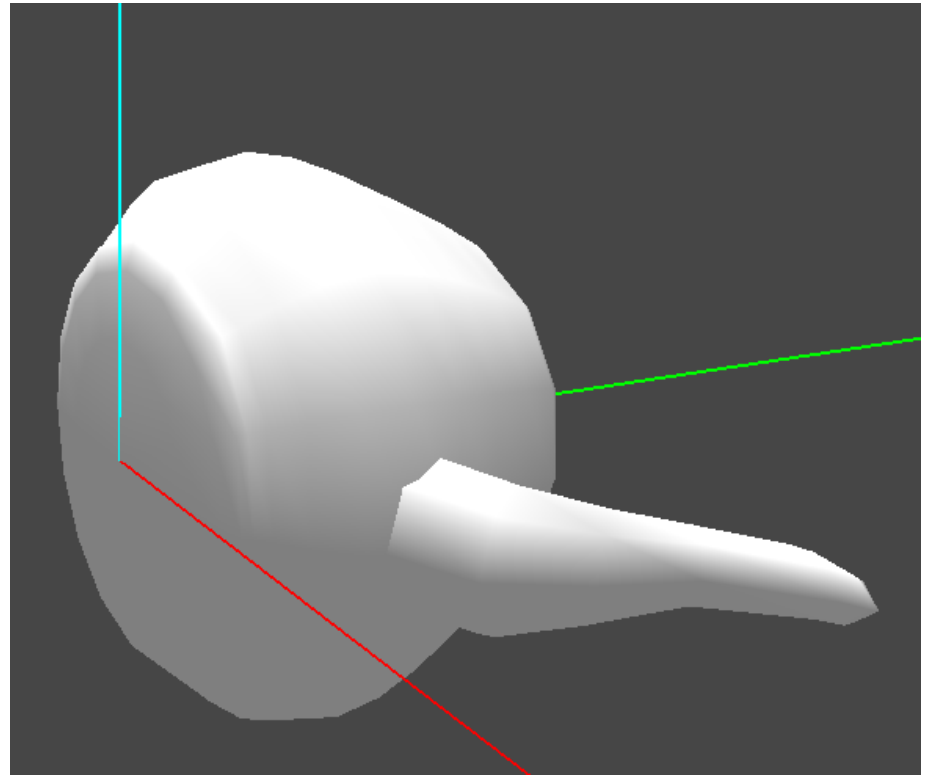
8.2. Primitives GLUT

```
void glutSolidTeapot();
```

Acció: Dibuixa una tetera sobre el pla $y=0$, circumscrita en una esfera de radi `1.`

Exemple: `glutSolidTeapot();`





8.2. Primitives GLUT

```
void glutSolidTeapot();
```

Funció amb VAO sense EBO:

```
void glutSolidTeapot() {
      Set_VAOList(GLUT_TEAPOT, loadglutSolidTeapot_VAO());
      draw_TriVAO_Object(GLUT_TEAPOT);
      deleteVAOList(GLUT_TEAPOT);
}
```

Funció amb VAO amb EBO:

```
void glutSolidTeapot() {
      Set_VAOList(GLUT_TEAPOT, loadglutSolidTeapot_EBO());
      draw_TriEBO_Object(GLUT_TEAPOT);
      deleteVAOList(GLUT_TEAPOT);
}
```

9. Malla. Classe `Mesh`

Classe definida i explicada en [deVries15].

Conté una estructura de dades per a contenir una malla (mesh) definida per vèrtexs, vectors normals i coordenades textura per a dibuixar-la en objectes OpenGL VAO, VBO i EBO.

9. Malla. Classe Mesh

Definició de la classe Mesh:

```
// Estructures de la classe Mesh.
// Vertex: Estructura amb atributs d'un vertex de la malla.
struct Vertex {
    glm::vec3 Position;               // Posició (x,y,z) del vertex de la malla.
    glm::vec3 Normal;                 // Vector normal (nx,ny,nz) del vertex de la malla.
    glm::vec2 TexCoords;              // Coordenada textura (s,t) del vertex de la malla.
};

// Texture: Estructura amb atributs de textures de la malla.
struct Texture {
    unsigned int id;      // Identificador de l'objecte textura de la malla.
    string type;          // Tipus de textura de la malla: diffuse map o specular map.
};

class Mesh {
  public:
      // Mesh data
      vector<Vertex> vertices;        // Vector amb atributs als vèrtexs de la malla.
      vector<unsigned int> indices;   // Vector índexs a vèrtexs de cares de la malla per a objecte EBO.
      vector<Texture> textures;       // vector d'atributs de textures de la malla.
      // Functions:
      Mesh(vector<Vertex> vertices, vector<unsigned int> indices, vector<Texture> textures);
      void Draw(Shader shader);
  private:
      // Render data:
      unsigned int VAO, VBO, EBO;
      // Functions:
      void setupMesh();
};
```

9. Malla. Classe `Mesh`

Funcions classe `Mesh`:

- `Mesh()`: Constructor de la classe que assigna contingut a les estructures de dades de la malla i crida a `setupMesh()`.
- `setupMesh()`: Crea objectes VAO, VBO i EBO i els hi carrega el contingut de les estructures de dades de la malla.
- `Draw()`: Activa les textures de la malla i dibuixa la malla en forma de triangles (`glDrawElements(GL_TRIANGLES,0,n);`)

9. Malla. Classe `Mesh`

Funció `Draw()`

Problema: Quantes textures cal definir en els shaders per a dibuixar la malla i com anomenar-les?.

Convenció:

- Definir en shaders el nom `texture_diffuseN` per a diffuse Maps o `texture_specularN` per a specular Maps on N és el número de textura (`unsigned int` 0-n).

```
uniform sampler2D texture_diffuse1;
uniform sampler2D texture_diffuse2;
uniform sampler2D texture_diffuse3;
uniform sampler2D texture_specular1;
uniform sampler2D texture_specular2;
```

- Establir un nombre màxim de textures que pot definir una malla, i si alguna ho sobrepassa, modificar shader.

9. Malla. Classe Mesh

Funció Mesh()

```
Mesh(vector<Vertex> vertices, vector<unsigned int> indices, vector<Texture> textures)
{   this->vertices = vertices;
    this->indices = indices;
    this->textures = textures;
    setupMesh();
}
```

Funció setupMesh()

```
void setupMesh()
{   glGenVertexArrays(1, &VAO);                    // Creació identificadors VAO, VBO i EBO.
    glGenBuffers(1, &VBO);
    glGenBuffers(1, &EBO);
    glBindVertexArray(VAO);                        // Activació VAO i VBO.
    glBindBuffer(GL_ARRAY_BUFFER, VBO);
    // Reservar buffer per a l'objecte VBO
    glBufferData(GL_ARRAY_BUFFER, vertices.size() * sizeof(Vertex), &vertices[0], GL_STATIC_DRAW);
    glBindBuffer(GL_ELEMENT_ARRAY_BUFFER, EBO);    // Activació EBO.
    glBufferData(GL_ELEMENT_ARRAY_BUFFER, indices.size() * sizeof(unsigned int), &indices[0], GL_STATIC_DRAW);

    // Vertex positions
    glEnableVertexAttribArray(0);
    glVertexAttribPointer(0, 3, GL_FLOAT, GL_FALSE, sizeof(Vertex), (void*)0);
    // Vertex normals
    glEnableVertexAttribArray(1);
    glVertexAttribPointer(1, 3, GL_FLOAT, GL_FALSE, sizeof(Vertex), (void*)offsetof(Vertex, Normal));
    // Vertex texture coords
    glEnableVertexAttribArray(2);
    glVertexAttribPointer(2, 2, GL_FLOAT, GL_FALSE, sizeof(Vertex), (void*)offsetof(Vertex, TexCoords));
    glBindVertexArray(0);                          // Desactivació VAO.
}
```

9. Malla. Classe Mesh

Funció Draw()

```
void Mesh::Draw(Shader shader)
{
    unsigned int diffuseNr = 1;
    unsigned int specularNr = 1;
    shader.Use();                                   // Activació del program prèviament compilar i linkat

    for(unsigned int i = 0; i < textures.size(); i++)
    {   glActiveTexture(GL_TEXTURE0 + i);           // Activate proper texture unit before binding

        // Retrieve texture number (the N in diffuse_textureN)
        string number;
        string name = textures[i].type;
        if (name == "texture_diffuse") number = std::to_string(diffuseNr++);
          else if (name == "texture_specular") number = std::to_string(specularNr++);

        shader.setFloat(("material." + name + number).c_str(), i);
        glBindTexture(GL_TEXTURE_2D, textures[i].id);
    }
    glActiveTexture(GL_TEXTURE0);

    // Draw mesh
    glBindVertexArray(VAO);
    glDrawElements(GL_TRIANGLES, indices.size(), GL_UNSIGNED_INT, 0);
    glBindVertexArray(0);
}
```

10. Model. Classe `Model`

Classe definida i explicada en [deVries15].

Classe que utilitza funcions de la llibreria Assimp per a llegir un model 3D contingut en un fitxer gràfic i transformar la informació llegida en dades Assimp a malles per a la classe `Mesh`.

10. Model. Classe Model

Definició de la classe Model:

```
// Inclusió de les funcions de la llibreria Assimp per a importar Models 3D en format gràfic.
#include <assimp/Importer.hpp>
#include <assimp/scene.h>
#include <assimp/postprocess.h>

class Model
{
    public:
        Model(char *path)            // Constructor de la classe.
        {   loadModel(path);
        }
        void Draw(Shader shader);
    private:
        // Model data
        vector<Mesh> meshes;         // Vector de malles de la classe Mesh.
        string directory;            // Path on es troben els fitxers del model 3D.

        void loadModel(string path);
        void processNode(aiNode *node, const aiScene *scene);
        Mesh processMesh(aiMesh *mesh, const aiScene *scene);
        vector<Texture> loadMaterialTextures(aiMaterial *mat, aiTextureType type,
                                             string typeName);
};
```

10. Model. Classe `Model`

Funcions classe `Model`:

- `Model()`: Constructor de la classe que crida a `loadModel()`.
- `loadModel()`: Crida funcions Assimp per a llegir el fitxer gràfic i que el contingut es carregui en les estructures de dades Assimp. Un model es composa de varies estructures tipus `node` (objecte Assimp). Crida a la funció `processNode()` per a processar cada node Assimp.
- `processNode()`: Converteix les malles contingudes en un node Assimp a malles de la classe Mesh. Crida a la funció processMesh().
- `processMesh()`: Converteix una malla Assimp a una malla `Mesh`.

10. Model. Classe Model

Funció loadModel()

```
void loadModel(string path)
{
    Assimp::Importer import;            // Instancia Importer d'Assimp que contindrà tot el model 3D
    // Lectura fitxer gràfic del model 3D, aplicant directives de postprocessament (aiProcess_Triangulate, etc.)
    const aiScene *scene = import.ReadFile(path, aiProcess_Triangulate | aiProcess_FlipUVs);

    if(!scene || scene->mFlags & AI_SCENE_FLAGS_INCOMPLETE || !scene->mRootNode)
    {   printf("ERROR::ASSIMP:: \s \n", import.GetErrorString());
        return;
    }
    directory = path.substr(0, path.find_last_of('/'));          // Extracció del nom de fitxer final al path
    processNode(scene->mRootNode, scene);                        // Processar tots els nodes Assimp.
}
```

Funció processNode()

```
void processNode(aiNode *node, const aiScene *scene)
{   // Process all the node's meshes (if any)
    for(unsigned int i = 0; i < node->mNumMeshes; i++)
    {   aiMesh *mesh = scene->mMeshes[node->mMeshes[i]];  // Extreu les malles d'un node de l'escena Assimp.
        meshes.push_back(processMesh(mesh, scene));       // Malla Assimp → Malla Mesh i la guarda en vector meshes.

    }
    // Then do the same for each of its children
    for(unsigned int i = 0; i < node->mNumChildren; i++)
    {   processNode(node->mChildren[i], scene);
    }
}
```

10. Model. Classe Model

Funció processMesh()

```
Mesh processMesh(aiMesh *mesh, const aiScene *scene)
{
    vector<Vertex> vertices;
    vector<unsigned int> indices;
    vector<Texture> textures;

    for(unsigned int i = 0; i < mesh->mNumVertices; i++)
    {   Vertex vertex;
        // Process vertex positions, normals and texture coordinates
        [...]
        vertices.push_back(vertex);
    }
    // Process indices
    [...]
    // Process material
    if(mesh->mMaterialIndex >= 0)
    {   [...]
    }
    return Mesh(vertices, indices, textures);
}
```

Funció Draw()

```
void Model::Draw(Shader shader)
{
    for(unsigned int i = 0; i < meshes.size(); i++)
        meshes[i].Draw(shader);
}
```

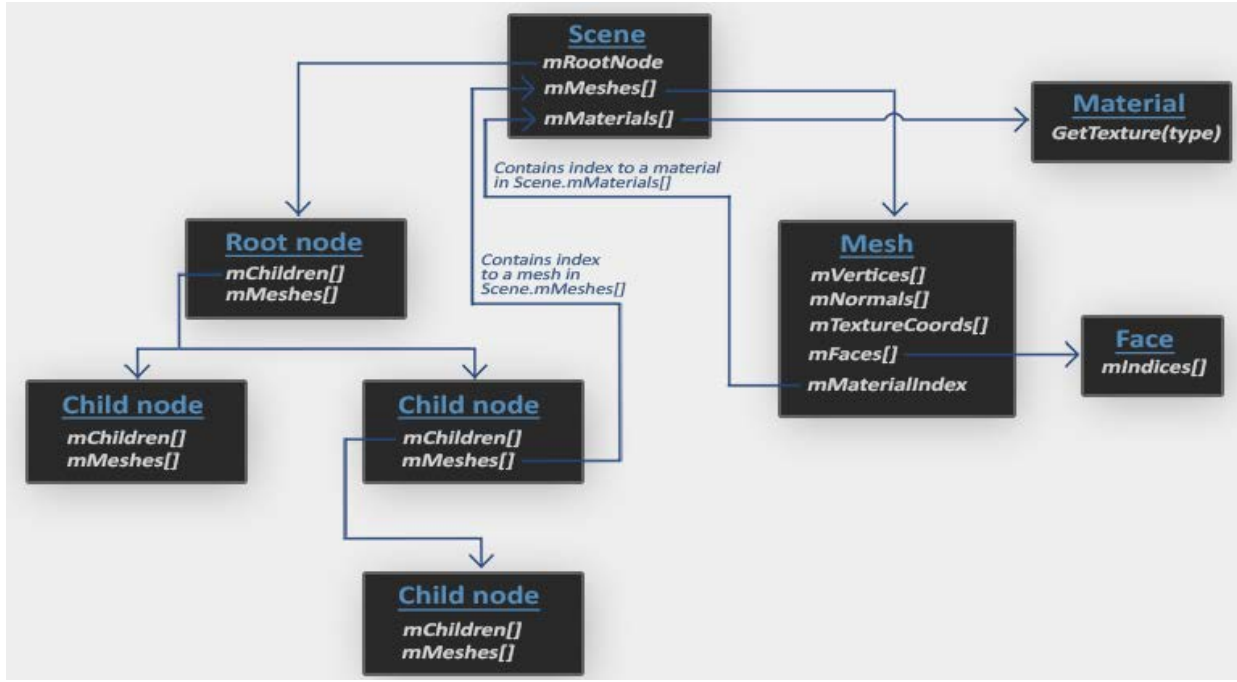
11. Importació Model3D. Llibreria Assimp

*Open **Ass**et **Imp**ort Library*

Llibreria importadora de models 3D que accepta diferents formats gràfics (.OBJ, .3DS, Collada, etc.) i també pot exportar a un fitxer, permetent la conversió de formats de models 3D.

11. Importació Model3D. Llibreria Assimp

Assimp llegeix informació del fitxer gràfic (geometria, normals, textures, etc.) i la transforma a estructures pròpies (scene, node, mesh):



Assimp defineix una estructura jeràrquica de *Scene*, que conté *nodes*, i cada node conté *nMeshes[ ]*, cadascun amb la seva geometria (*mVertices[ ], mNormals[ ], mfaces[ ], mIndices[ ]*), coordenades textura (*mTextureCoords[ ]*) i materials (*mMaterialIndex*).

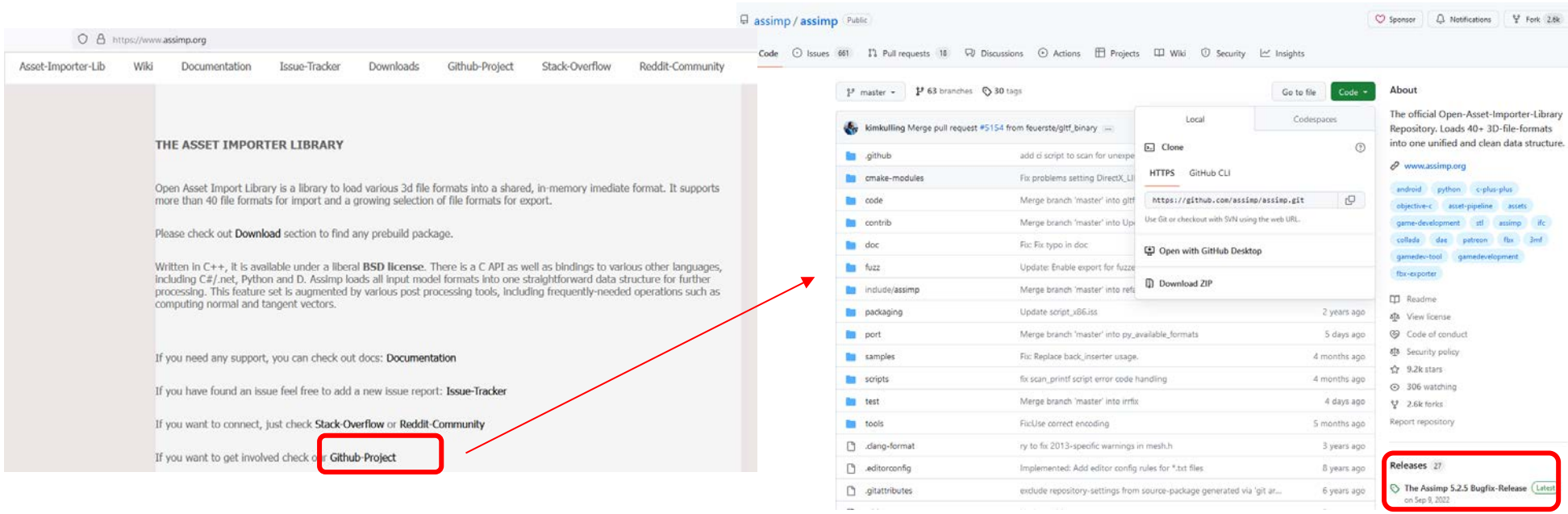
11. Importació Model3D. Llibreria Assimp

Passos per a incorporar la llibreria Assimp i les classes `Mesh` i `Model` al projecte Visual:

1. Instalació llibreria Assimp
2. Inclusió Assimp al projecte
3. Inclusió classes `Mesh` i `Model` al projecte
4. Connexió de llibreria i classes al codi del projecte

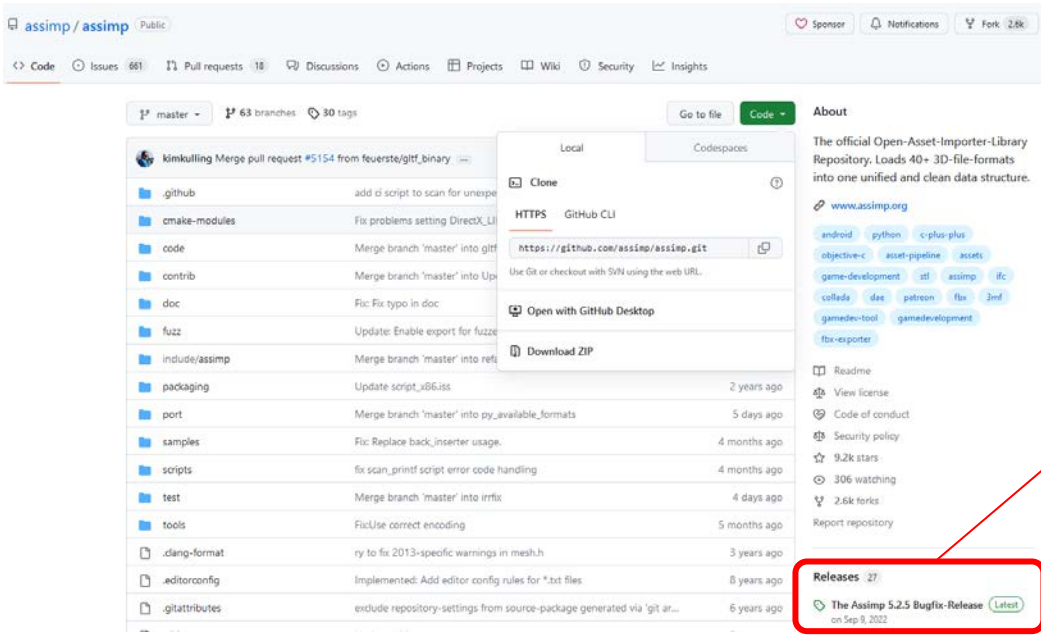
11.1. Instalació llibreria Assimp

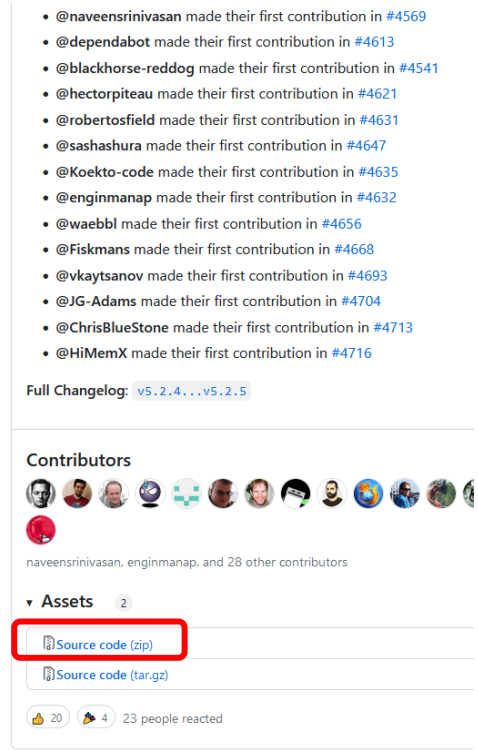
Llibreria de codi obert es pot trobar a https://www.assimp.org/ on hi ha documentació i el codi font de la versió 5.2.5.



11.1. Instalació llibreria Assimp

Llibreria de codi obert es pot trobar a https://www.assimp.org/ on hi ha documentació i el codi font de la versió 5.2.5.





11.1. Instalació llibreria Assimp

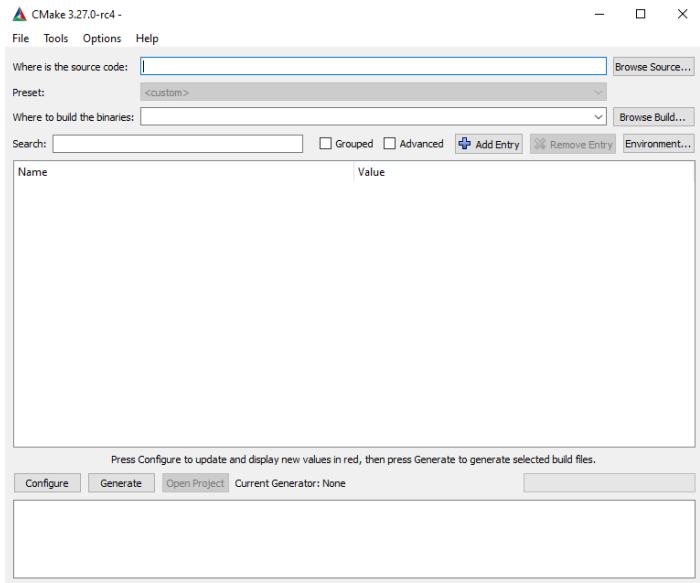
Convenient compilar la llibreria (generar fitxers `.lib` i `.dll`) per a tenir el codi el més actualitzat possible.

Passos per a compilar la llibreria:

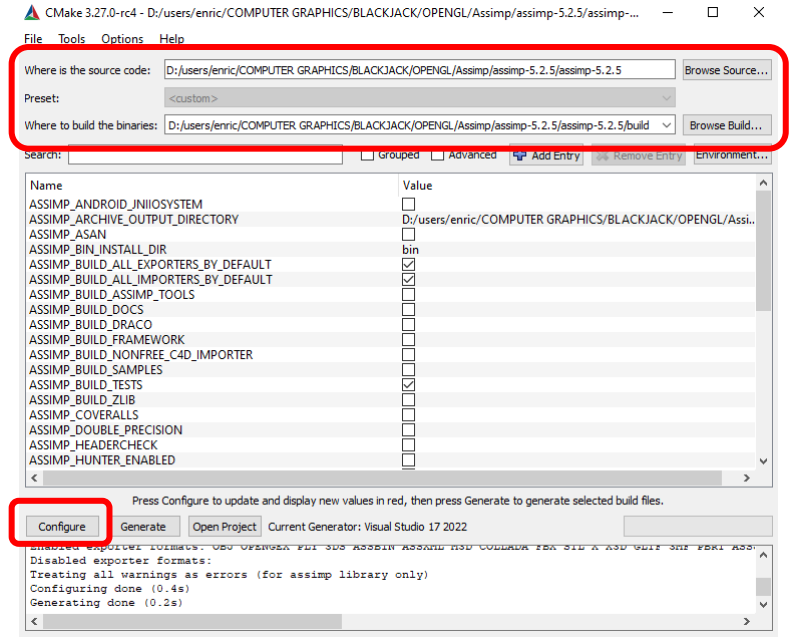
1. Baixar codi font (`assimp-5.2.5.zip`) i descomprimir
2. Baixar i instal·lar el programa `CMake v.3.27.0` segons plataforma (Win32, Win64, MacOS X 10.7, Linux x86_64 (https://cmake.org/)
3. Executar `CMake` per a crear una versió d'Assimp per a ser compilada (per `VS2022` per Win32, `Xcode` per a MacOS o `gcc` per a Linux) al directori `./build`
4. Compilar projecte en `./build` per a generar fitxers `.lib` i .dll

11.1.3. Executar CMake

Quan executem CMake ens apareix la GUI:







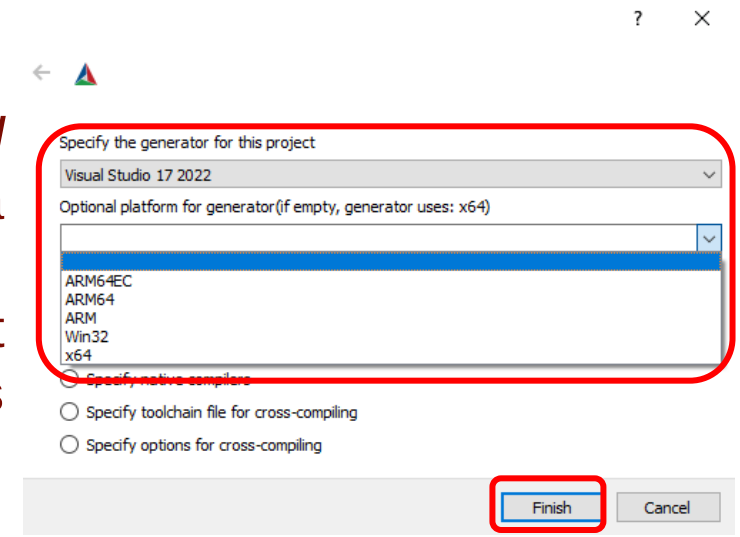
Passos:

1. *Where is the Source code:* Directori `.\assimp-5.2.5\assimp-master`
2. Where to build the binaries: Directori `.\assimp-5.2.4\build` (cal crear directori `.\assimp-5.2.4\build`)
3. Prèmer botó *Configure*

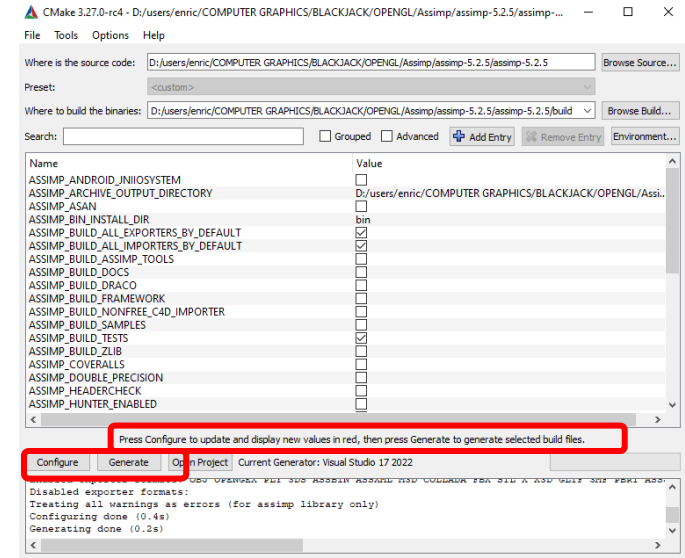
11.1.3. Executar CMake

Passos:

4. En nova finestra, seleccionar *Visual Studio 17 2022.* Per defecte es genera codi per x64, que serà el nostre cas, però en el segon desplegable es pot triar generar codi per a x86. Després prémer botó *Finish*



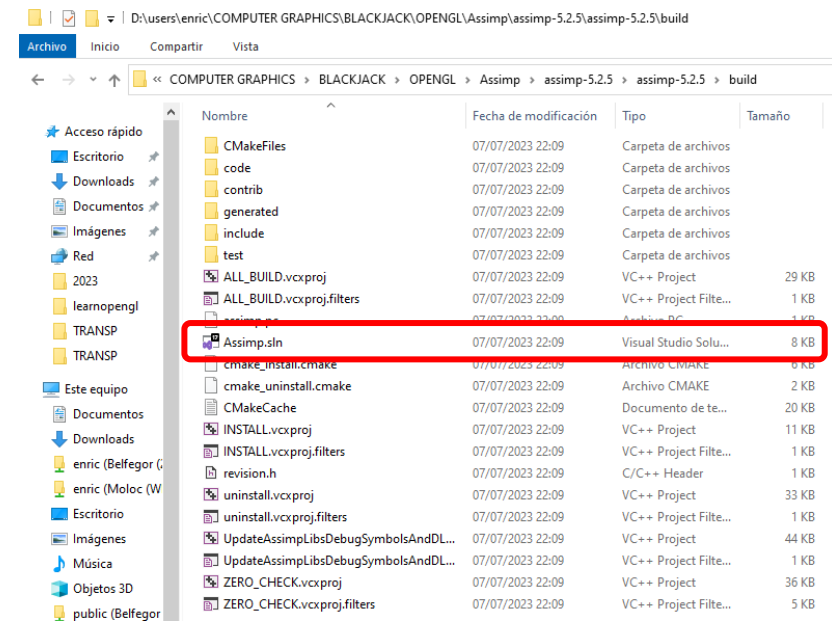
5. Prèmer botó *Configure* i després botó *Generate* i es genera projecte VS2022 en el directori `.\assimp-5.2.5\assimp-master\build`.
Utilitzar els valors definits per defecte.



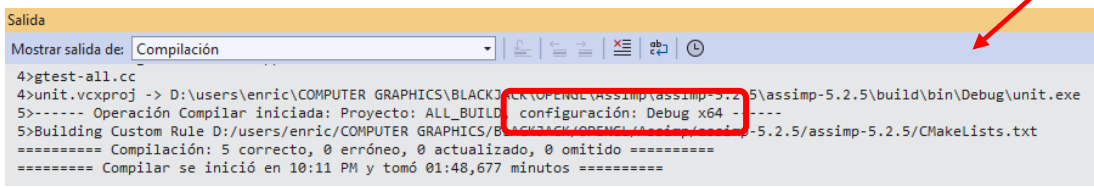
11.1.3. Executar CMake

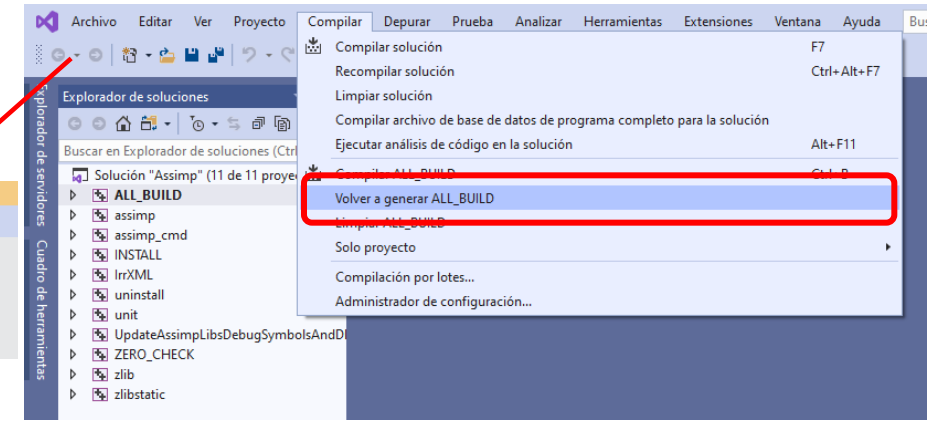
Passos:

6. En carpeta `.\assimp-5.2.4\build` obrir fitxer `Assimp.sln` amb VS2022



7. Compilar projecte per a `Debug – x64`

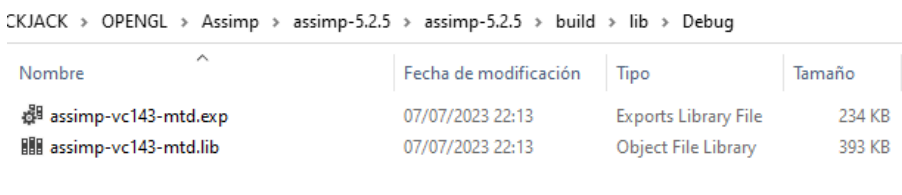


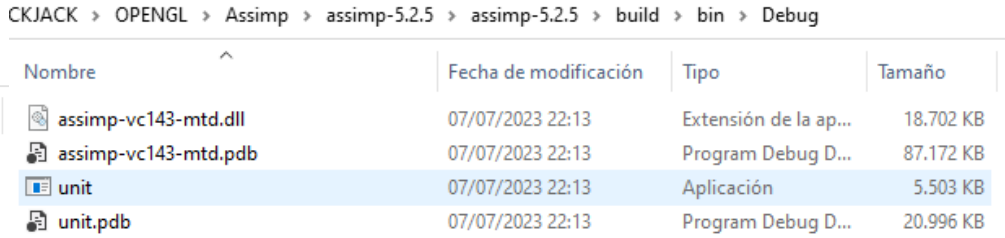


11.1.3. Executar CMake

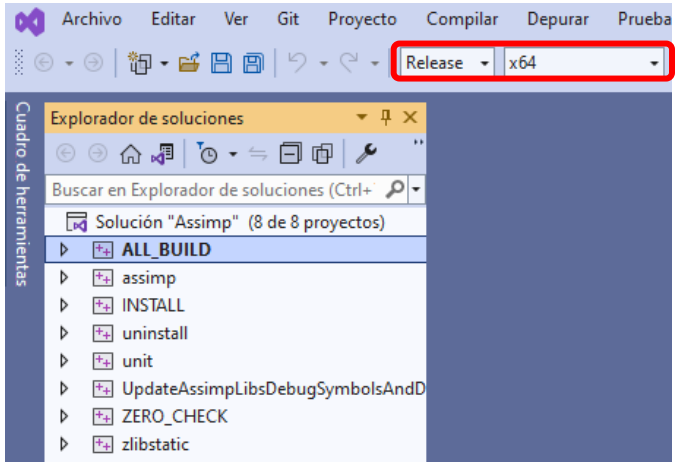
Passos:

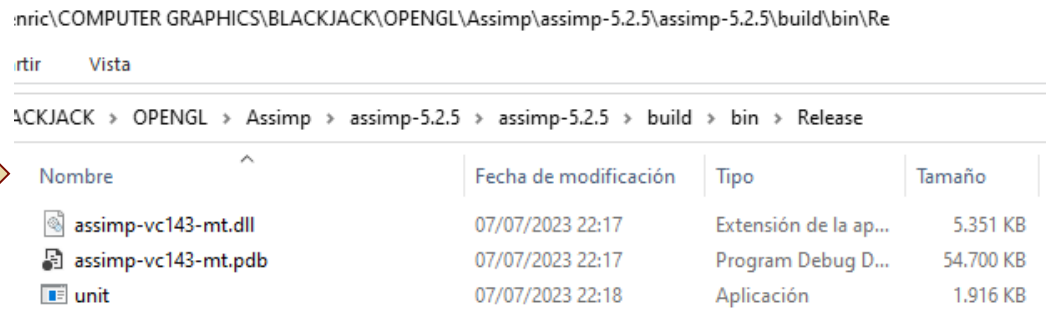
8. S'han generat en la carpeta `.\assimp-5.2.5\build\lib\Debug` els fitxers `.lib` i en la carpeta `.\assimp-5.2.5\build\bin\Debug` els fitxers `.dll` de la llibreria



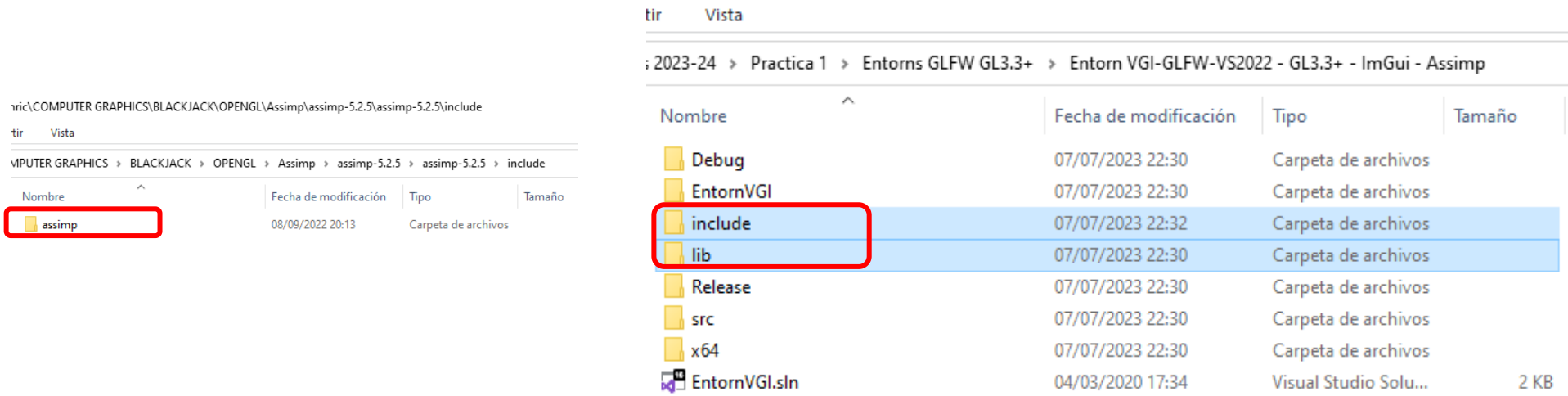


8. Es pot generar també la versió Release de la llibreria





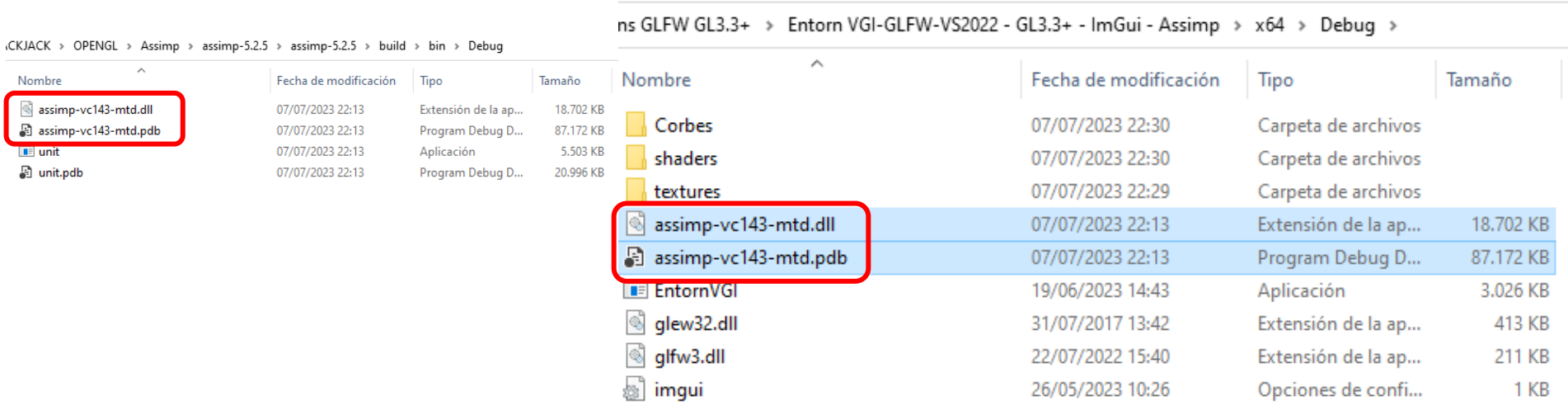
11.2. Inclusió Assimp al projecte



Incloure els fitxers de la llibreria al projecte Visual:

1. Carpeta `.\assimp-5.2.5\include\assimp` dins la carpeta `\include` del projecte
2. Fitxers `.\assimp-5.2.5\build\lib\Debug\assimp-vc143-mtd.lib i .exp` a directori `\lib` del projecte
3. Fitxer `assimp-vc143-mtd.dll` a directori `\Debug` del projecte, si s'ha generat la versió Debug de la llibreria. El fitxer `assimp-vc143-mt.dll` al directori `\Release` si s'ha generat.

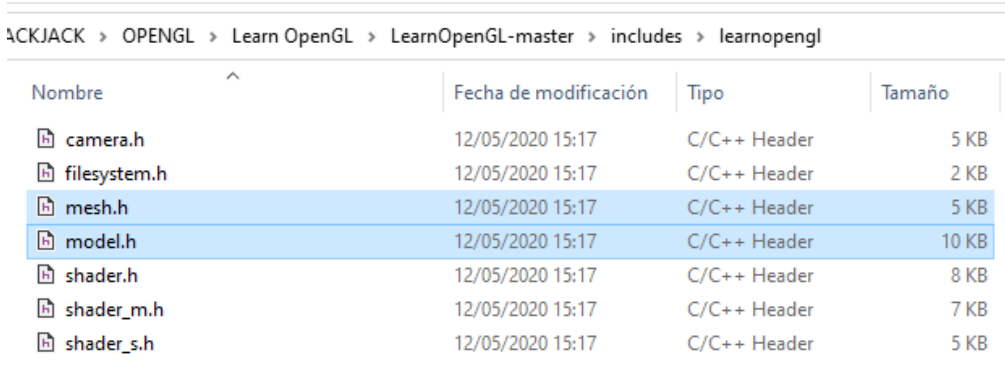
11.2. Inclusió Assimp al projecte

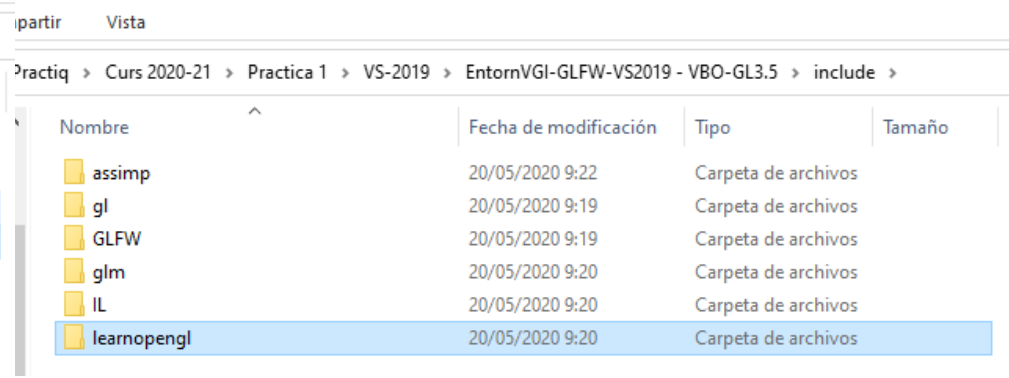


Incloure els fitxers de la llibreria al projecte Visual (2):

3. Fitxer `\assimp-vc143-mtd.dll` a directori `.\x64\Debug` del projecte, si s'ha generat la versió Debug de la llibreria. El fitxer `.\assimp-5.2.5\build\bin\Debug\assimp-vc143-mt.dll` al directori `.\x64\Release` si s'ha generat la versió Release.

11.3. Inclusió classes Mesh i Model al projecte





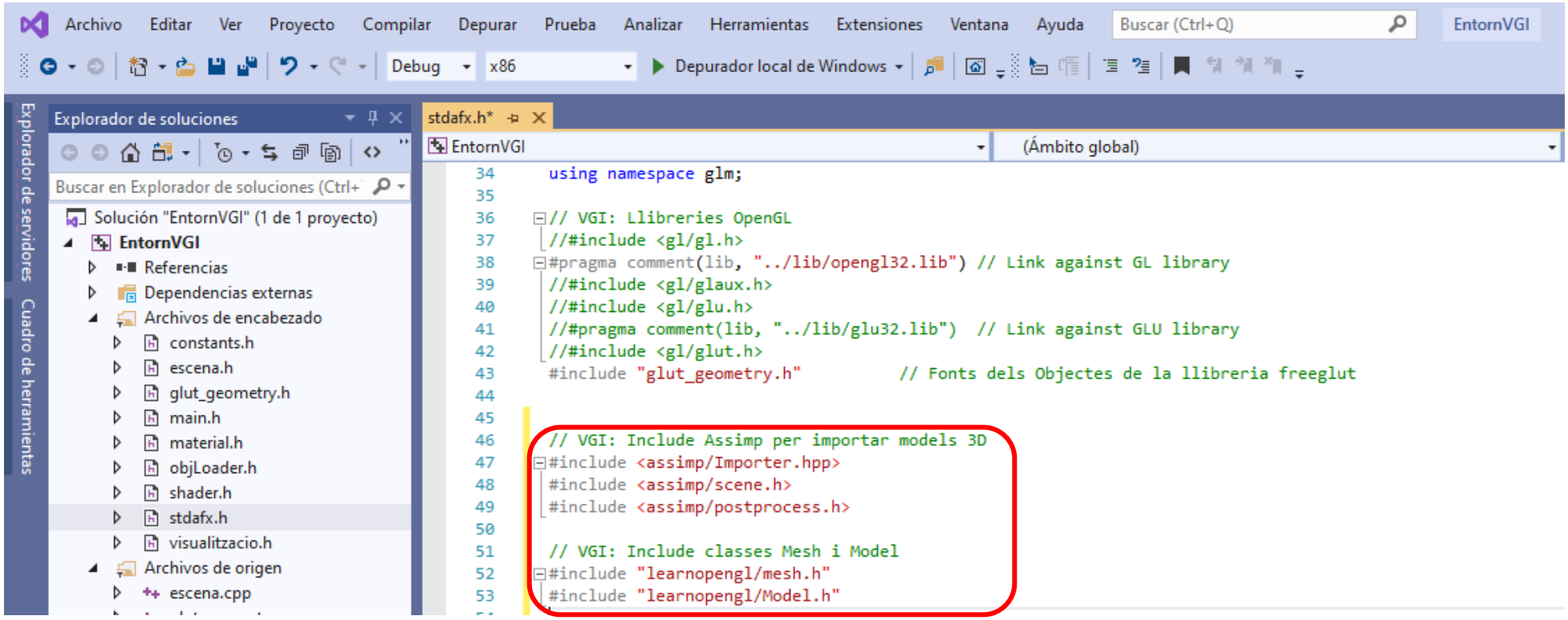
Passos:

1. Crear dins directori del projecte `\include` una carpeta anomenada `\learnopengl`
2. Baixar el codi font de [deVries15] a Githhub (https://github.com/JoeyDeVries/LearnOpenGL) comprimit i descomprimir carpeta `.\LearnOpenGL-master`
3. Copiar els fitxers `.\LearnOpenGL-master\mesh.h` i `model.h` al directori `.\include\learnopengl` del projecte

11.4. Connexió llibreria i classes al codi projecte

Passos:

1. Includes de la llibreria Assimp a `stdafx.h` del projecte:
2. Includes de `mesh.h` i `model.h` a `stdafx.h`



11.4. Connexió llibreria i classes al codi projecte

`ReadFile()`. Funció principal de lectura d'un fitxer gràfic, cridada en la funció `Model::loadModel()`:

```
Assimp::Importer importer;
const aiScene *scene = importer.ReadFile(path, aiProcess_Triangulate | aiProcess_FlipUVs);
```

Paràmetres

- `path`: path i nom del fitxer gràfic principal que conté el model 3D. El path es guarda per a buscar fitxers auxiliars.
- `Segon paràmetre`: Bits de post-processament de la informació gràfica. Poden haver diferents valors:
  - `aiProcess_Triangulate`: Triangularitza tots els polígons del model
  - `aiProcess_FlipUV`: Inversió Y de imatge textura
  - `aiProcess_GenNormals`: Calcula normals dels vèrtexs si model no les te
  - `aiProcess_Tangent`: Calcula vector tangent de cada vèrtexs per a *Normal Mapping*
  - `aiProcess_SplitLargeMeshes`: Divideix malles grans en malles petites per a optimització
  - `aiProcess_OptimizeMeshes`: Junta malles en una més gran per optimització

En resum...

- Els objectes VBO, VAO i EBO són una forma òptima de renderitzar geometria d'una malla 3D doncs l'envien al hardware gràfic un sol cop i això permet ràpidament dibuixar diferents còpies de la malla.
- Com aquests objectes transfereixen la geometria facilita molt l'accés d'aquesta geometria als shaders.
- L'objecte VAO permet tenir vàries malles 3D carregades en el hardware gràfic i activar i dibuixar el que ens interessi, accelerant molt la visualització.
- L'objecte EBO, mitjançant un vector d'índexs als vèrtexs estalvia molta memòria a transferir al hardware gràfic. Quan coneixem la resolució exacta de la malla és molt òptim.
- S'han adaptat les funcions de les primitives GLU i GLUT als objectes VBO, VAO i EBO.
- S'utilitza la llibreria *Assimp* per a importar models 3D en format gràfic .obj, .3ds, etc.

Referències

[deVries15] Joey de Vries**, Learn OpenGL,** https://learnopengl.com/**,** 2015.